
SOFTWARE ENGINEERING

NOTEBOOK

Table of Contents

Table of Contents	2
1 Algorithms	7
1 Time Complexity	7
1.1 Big O notation	8
1.2 Omega notation	8
1.3 Theta notation	8
1.4 Meaning of previous notations - Insertion sort case	8
1.5 Running times of well known algorithms	10
1.6 Methods for solving the recurrence	11
2 Space Complexity	11
Algorithms Bibliography	12
2 Idioms	13
1 C++ Idioms	13
1.1 PImpl Idiom	13
1.2 Fast PImpl Idiom	14
1.3 Handle/Object Idiom	14
1.4 Copy and Swap Idiom	14
1.5 RAII - Resource Acquisition is Initialization	14
1.6 DBDII - Dynamic Binding During Initialization Idiom	14
1.7 Named Constructor Idiom	14
1.8 Construct On First Use Idiom	14
1.9 Nifty Counter Idiom	14
1.10 Named Parameter Idiom	15
1.11 Virtual Friend Function Idiom	15
1.12 CRTP - Curiously recurring template pattern	15
1.13 Type Erasure	15
Idioms Bibliography	16
3 Containers	17
1 Links to the CPP Reference documentation	19

1.1	Sequence Containers	19
1.2	Associative Containers	19
1.3	Unordered Associative Containers	19
1.4	Container Adaptors	20
1.5	Views	20
2	Invalidation of Iterators and References	21
	Containers Bibliography	22
4	Memory Management	23
1	Memory Management	23
1.1	Memory Allocation and Deallocation in C	25
1.2	Memory Allocation and Deallocation in C++	25
	Memory Management Bibliography	26
5	Smart Pointers	27
1	Smart Pointers in C++	27
1.1	std::unique_ptr	28
1.2	std::shared_ptr	30
1.3	std::weak_ptr	31
1.4	std::auto_ptr	32
	Smart Pointers Bibliography	33
6	C++	35
1	C++: General Knowledge and Good Practices	35
1.1	Rule of three/five/zero	35
1.2	The Most Vexing Parse	36
1.3	General Topics	38
1.4	Classes and Inheritance	46
1.5	Beyond Classes	65
1.6	Value Categories	72
1.7	Reference Binding	73
1.8	lvalue references	74
2	C++11 New Features - legacy section to remove	75
2.1	Reference Collapsing	75
3	Contemporary C++: new language and library features	76
3.1	C++11 new language features	76
3.2	C++11 new library features	92
3.3	C++14 new language features	98
3.4	C++14 new library features	101
3.5	C++17 new language features	102

3.6	C++17 new library features:	109
3.7	C++20 new language features	117
3.8	C++20 new library features	125
	C++ Bibliography	130
7	Concurrency	133
1	Threads	133
1.1	Functions managing the current thread	134
2	Atomicity	135
2.1	Atomic operations	135
2.2	Atomic types	135
2.3	Operations on atomic types	135
2.4	Flag type and operations	136
2.5	Initialization	136
2.6	Memory synchronization ordering	137
2.7	Mutual exclusion	137
2.8	Generic mutex management	137
2.9	Generic locking management	138
2.10	Call once	138
2.11	Condition variables	138
2.12	Futures	139
2.13	Future errors	139
2.14	Memory Order	140
	Concurrency Bibliography	142
8	Python	143
1	PEP	143
2	Built-in Data Types	143
2.1	dict	144
2.2	list	144
2.3	set	144
2.4	frozenset	144
2.5	tuple	144
2.6	string	144
2.7	bytes	144
2.8	bytearray	144
3	unittest Module	145
3.1	Classes and functions	145
4	Decorator	146
5	Special Instance Method	147

6	Providing Multiple Ctors	147
	Python Bibliography	148
9	Software Testing	149
1	Seven Tesing Principles	149
2	SDLC vs. STLC	149
3	Python Unit Testing	149
3.1	unittest	149
3.2	doctest	152
4	C++ Unit Testing	152
4.1	gMock	152
5	Integration Test	152
6	Regression Test	153
7	Behavior-Driven Development	153
8	terms from the youtube video	153
9	Continous Integration Systems	153
	Testing Bibliography	155
	Bibliography	157
0	References	162

Algorithms

1 Time Complexity

Time complexity for an algorithm has three different facets

- The *worst-case* complexity of the algorithm is the function defined by the maximum number of steps taken in any instance of size n .
- The *best-case* complexity of the algorithm is the function defined by the minimum number of steps taken in any instance of size n .
- The *average-case* complexity or expected time of the algorithm, which is the function defined by the average number of steps over all instances of size n .

1.1 Big O notation

$$f(n) = O(g(n)) \quad \text{if} \quad 0 \leq f(n) \leq cg(n) \forall n \geq n_0 \quad (1.1)$$

For all values $n \geq n_0$, $f(n)$ is on or below $cg(n)$.

A constant multiple of $g(n)$ is an *asymptotic upper bound* for $f(n)$.

1.2 Omega notation

$$f(n) = \Omega(g(n)) \quad \text{if} \quad 0 \leq cg(n) \leq f(n) \forall n \geq n_0 \quad (1.2)$$

For all values $n \geq n_0$, $f(n)$ is on or above $cg(n)$.

A constant multiple of $g(n)$ is an *asymptotic lower bound* for $f(n)$.

1.3 Theta notation

$$f(n) = \Theta(g(n)) \quad \text{if} \quad 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0 \quad (1.3)$$

For all values $n \geq n_0$, $f(n)$ is equal to $g(n)$ within a constant factor.

$g(n)$ is an *asymptotically tight bound* for $f(n)$.

1.4 Meaning of previous notations - Insertion sort case

Big O notation

describes an *upper bound*. When we use it to *bound the worst-case running time* of an algorithm, we have a bound on the running time of the algorithm on *every input*.

Thus, the $O(n^2)$ bound on worst-case running time of *insertion sort* also applies to its running time on every input.

Omega notation

describes a *lower bound*. When we say that the running time (no modifier) of an algorithm is $\Omega(g(n))$, we mean that *no matter what particular input of size n is chosen* for each value of n , the running time on that input is at least a constant times $g(n)$, for sufficiently large n .

Equivalently, we are giving a *lower bound on the best-case running time* of an algorithm. For example, the best-case running time of insertion sort is $\Omega(n)$, which implies that the running time of insertion sort is $\Omega(n)$.

The *running time of insertion sort therefore belongs to both $\Omega(n)$ and $O(n^2)$* , since it falls anywhere between a linear function of n and a quadratic function of n . Moreover, these bounds are asymptotically as tight as possible: for instance, the running time of insertion sort is not $\Omega(n^2)$, since there exists an input for which insertion sort runs in $\Theta(n)$ time (e.g., when the input is already sorted). It is not contradictory, however, to say that the worst-case running time of insertion sort is $\Omega(n^2)$, since there exists an input that causes the algorithm to take $\Omega(n^2)$ time.

Theta notation

describes a *tight bound*. The $\Theta(n^2)$ notation bound on the worst-case running time of insertion sort, however, does not imply a $\Theta(n^2)$ bound on the running time of insertion

sort on every input. For example, when the input is already sorted, insertion sort runs in $\Theta(n)$ time. Technically, it is an abuse to say that the running time of insertion sort is $O(n^2)$, since for a given n , the actual running time varies, depending on the particular input of size n . When we say "the running time is $O(n^2)$ ", we mean that there is a function $f(n)$ that is $O(n^2)$ such that for any value of n , no matter what particular input of size n is chosen, the running time on that input is bounded from above by the value $f(n)$. Equivalently, we mean that the worst-case running time is $O(n^2)$.

1.5 Running times of well known algorithms

Algorithm	Best	Average	Worst	Worst Space Complexity
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell Sort	$O(n)$	$O((n \log n)^2)$	$O((n \log n)^2)$	$O(1)$
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$

Verbal description of the previous sorting algorithms:

- **Quicksort**

Efficient, general-purpose sorting algorithm, **slightly faster** than **merge sort** and **heapsort** for **randomized data**, particularly on larger distributions.

It selects a **pivot element** from the array and partitions the other elements into **two sub-arrays**, according to whether they are **less than or greater than the pivot**.

For this reason, it is sometimes called **partition-exchange sort**.

The sub-arrays are then **sorted recursively**.

- **Mergesort**

First, divide the list into the smallest unit (1 element), then **compare each element with the adjacent** list to sort and merge the two adjacent lists.

Finally, all the elements are sorted and merged.

- **Timsort**

a hybrid, stable sorting algorithm, **derived from merge sort and insertion sort**, designed to **perform well on many kinds of real-world data**. It was implemented by Tim Peters in 2002 for use in the Python programming language. The algorithm finds **subsequences of the data that are already ordered** (runs) and uses them to **sort the remainder more efficiently**. This is done by merging runs until certain criteria are fulfilled. Timsort was Python's standard sorting algorithm from version 2.3 to version 3.10

- **Heapsort**

comparison-based sorting algorithm which can be thought of as

an implementation of selection sort using the right data structure. Like selection sort, heapsort divides its input into a sorted and an unsorted region, and it iteratively shrinks the unsorted region by extracting the largest element from it and inserting it into the sorted region. Unlike selection sort, heapsort does not waste

time with a linear-time scan of the unsorted region; rather, heap sort maintains the unsorted region in a heap data structure to efficiently find the largest element in each step.

- **Bubble Sort**

a simple sorting algorithm that repeatedly steps through the input list element by element, comparing the current element with the one after it, swapping their values if needed. These passes through the list are repeated until no swaps have to be performed during a pass, meaning that the list has become fully sorted

- **Insertion Sort**

iterates, consuming one input element each repetition, and grows a sorted output list. At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list, and inserts it there. It repeats until no input elements remain.

- **Selection Sort**

repeatedly identifies the smallest remaining unsorted element and puts it at the end of the sorted portion of the array

- **Shell Sort**

- **Bucket Sort**

Each bucket is then sorted individually, either using a different sorting algorithm, or by recursively applying the bucket sorting algorithm. It is a distribution sort, a generalization of pigeonhole sort that allows multiple keys per bucket, and is a cousin of radix sort in the most-to-least significant digit flavor. Bucket sort can be implemented with comparisons and therefore can also be considered a comparison sort algorithm. The computational complexity depends on the algorithm used to sort each bucket, the number of buckets to use, and whether the input is uniformly distributed.

- **Radix Sort**

1.6 Methods for solving the recurrence

There are three methods for solving recurrences:

- the substitution method
- the recursion-tree method
- the master theorem

2 Space Complexity

Algorithms Bibliography

- [90] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley.
- [91] Steven S. Skiena. *The Algorithm Design Manual*. Springer.
- [92] *Quicksort*. [Wikipedia](#).
- [93] *Merge Sort*. [Wikipedia](#).
- [94] *Timsort*. [Wikipedia](#).
- [95] *Heapsort*. [Wikipedia](#).
- [96] *Bubble Sort*. [Wikipedia](#).
- [97] *Insertion Sort*. [Wikipedia](#).
- [98] *Selection Sort*. [Wikipedia](#).
- [99] *Shell Sort*. [Wikipedia](#).
- [100] *Bucket Sort*. [Wikipedia](#).
- [101] *Radix Sort*. [Wikipedia](#).

Idioms

1 C++ Idioms

An idiom can be considered as the most natural way to express something or to solve a problem. More specifically, an idiom is a group of code fragments sharing an equivalent semantic role, which recurs frequently in one or more programming languages or libraries.

A comprehensive list of C++ Idioms can be found at [\[128\]](#)

1.1 PImpl Idiom

[c++reference documentation about the PImpl Idiom.](#)

"Pointer to implementation" or "pImpl" is a C++ programming technique that **removes implementation details of a class from its object representation** by placing them in a separate class, accessed **through an opaque pointer**.

This idiom is also known as **Compiler Firewall Idiom**.

Problematic Scenario

Whenever file class definition changes in a header, all users of that class must be recompiled. The reason is the textual inclusion on which the C++ build model is based. The user is assumed to know two features which could be affected by private members: **size** and **layout** and **functions**.

Solution

```
1 // Pimpl idiom - basic idea
2 class widget {
3     // :::
4 private:
5     struct impl;           // things to be hidden go here
6     impl* pimpl_;         // opaque pointer to forward-declared class
7 };
```

1.2 Fast PImpl Idiom

1.3 Handle/Object Idiom

1.4 Copy and Swap Idiom

This idiom aims at creating a strong exception-safe implementation of the overloaded assignment operator.

This idiom makes use of the **RAII** idiom in order to acquire the new resource before it forfeits its current resource. If the acquisition is successful

1.5 RAII - Resource Acquisition is Initialization

1.6 DBDII - Dynamic Binding During Initialization Idiom

[calling-virtuals-from-ctor-idiom](#)

1.7 Named Constructor Idiom

[\[139\]](#)

1.8 Construct On First Use Idiom

[\[140\]](#)

1.9 Nifty Counter Idiom

[\[141\]](#)

1.10 Named Parameter Idiom

[\[143\]](#)

1.11 Virtual Friend Function Idiom

[\[144\]](#)

1.12 CRTP - Curiously recurring template pattern

1.13 Type Erasure

Idioms Bibliography

- [128] *More C++ Idioms*. [Wikibooks](#).
- [129] *Pimpl Idiom*. [Wiki Wiki Web](#).
- [130] *CRTP - Curiously Recurring Template Pattern*. [Wiki Wiki Web](#).
- [131] *CRTP - Curiously Recurring Template Pattern*. [Cppreference](#).
- [132] Herb Sutter. *The Fast Pimpl Idiom*. [Guru of the Week - 28](#).
- [133] Herb Sutter. *Compilation Firewalls*. [Guru of the Week - 24](#).
- [134] Herb Sutter. *Compilation Firewalls (C++11-updated version of GotW #24)*. [Guru of the Week - 100](#).
- [135] Herb Sutter. *The Joy of Pimpls (or more about the Compiler-Firewall Idiom)*. [More About Compiler Firewalls](#).
- [136] *What is the copy and swap idiom?* [Stackoverflow](#).
- [137] Herb Sutter. *Exception-Safe Class Design, Part I: Copy Assignment*. [Guru of the Week - 59](#).
- [138] Herb Sutter. *Exception-Safe Class Design, Part II: Inheritance*. [Guru of the Week - 60](#).
- [139] *What is the Named Constructor Idiom?* [ISOC++ Wiki](#).
- [140] *Construct On First Use Idiom*. [ISOC++ Wiki](#).
- [141] *Nifty Counter Idiom*. [ISOC++ Wiki](#).
- [142] *Nifty Counter Idiom*. [Wikibook](#).
- [143] *Named Parameter Idiom*. [ISOC++ Wiki](#).
- [144] *Virtual Friend Function Idiom*. [ISOC++ Wiki](#).
- [145] *C++ Core Guidelines*. [Standard Cpp Foundation Github](#).

3

CHAPTER

Containers

The Containers library is a *generic collection of class templates and algorithms* that allow programmers *to easily implement common data structures* like queues, lists and stacks. The container classes are:

- *sequence containers*
implement data structures which can be accessed sequentially.
- *associative containers*,
can be searched in $O(\log n)$.
- *unordered associative containers*
can be searched in $O(1)$ on average and in $O(n)$ in the worst one.

Decision Charts

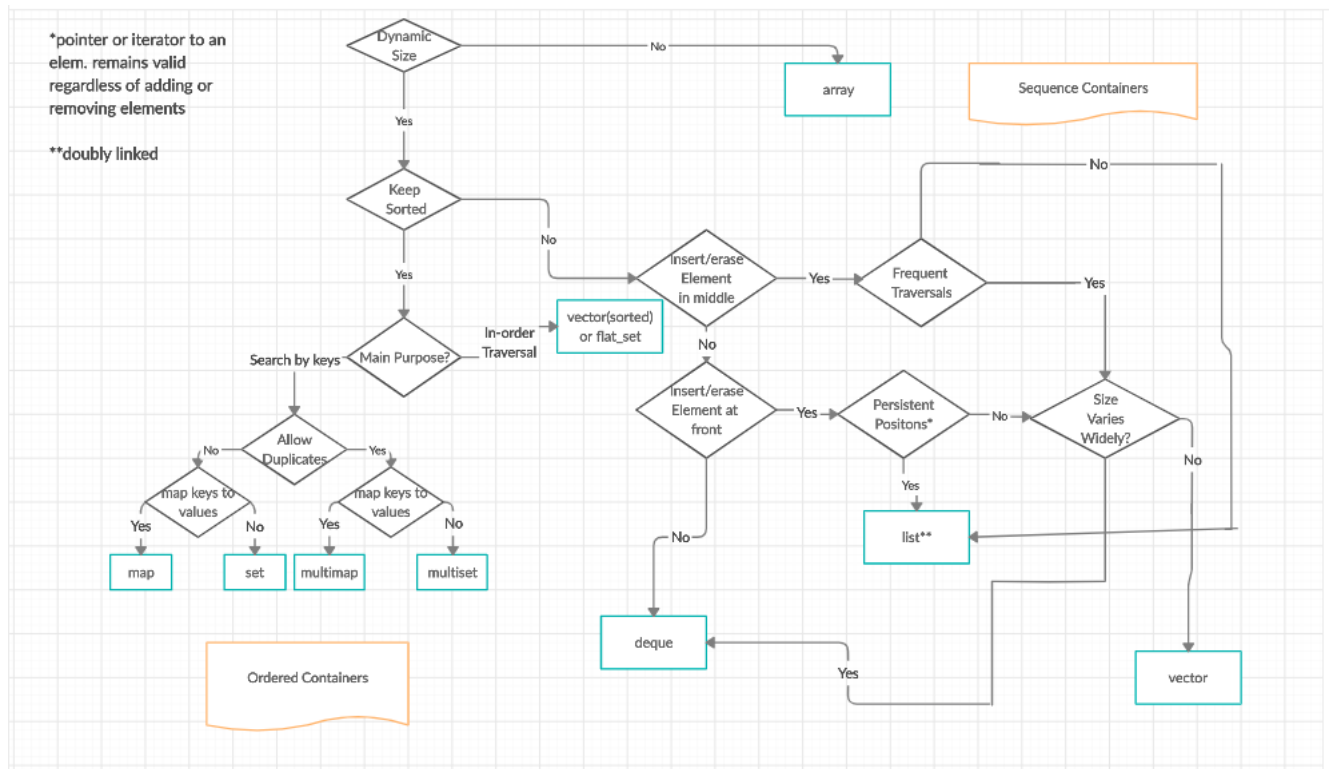


Figure 3.1: Decision Chart: Sequence And Ordered Containers

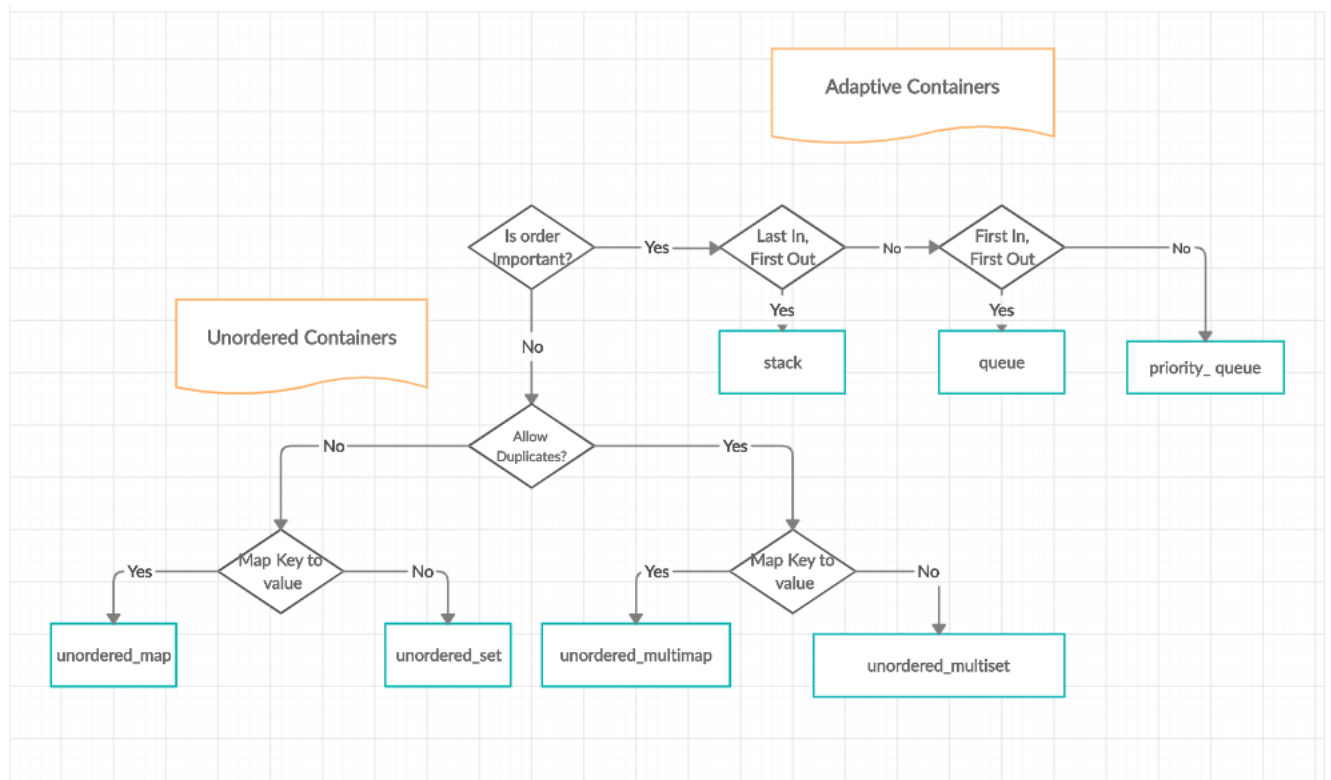


Figure 3.2: Decision Chart: Adaptive And Unordered Containers

1 Links to the CPP Reference documentation

1.1 Sequence Containers

array	<i>static contiguous</i> array
vector	<i>dynamic contiguous</i> array
deque	<i>double-ended</i> queue
forward_list	<i>singly-linked</i> list
list	<i>doubly-linked</i> list

1.2 Associative Containers

set	collection of <i>unique keys, sorted by keys</i>
map	collection of <i>key-value pairs, sorted by keys, keys are unique</i>
multiset	collection of <i>keys, sorted by keys</i>
multimap	collection of <i>key-value pairs, sorted by keys</i>

1.3 Unordered Associative Containers

unordered_set	collection of <i>unique keys, hashed by keys</i>
unordered_map	collection of <i>key-value pairs, hashed by keys, keys are unique</i>
unordered_multiset	collection of <i>keys, hashed by keys</i>
unordered_multimap	collection of <i>key-value pairs, hashed by keys</i>

1.4 Container Adaptors

stack	adapts a container to provide stack (<i>LIFO data structure</i>)
queue	adapts a container to provide queue (<i>FIFO data structure</i>)
priority_queue	adapts a container to provide <i>priority queue</i>

C++23 New Container Adaptors

With C++23, the following four Container Adaptors were introduced:

flat_set	adapts a container to provide a collection of <i>unique keys, sorted by keys</i>
flat_map	adapts two containers to provide a collection of <i>key-value pairs, sorted by unique keys</i>
flat_multiset	adapts a container to provide a collection of <i>keys, sorted by keys</i>
flat_multimap	adapts two containers to provide a collection of <i>key-value pairs, sorted by keys</i>

1.5 Views

2 Invalidation of Iterators and References

Category	Container	Valid after insertion		Valid after erasure		Conditionality
		iterators	reference	iterators	reference	
Sequence Containers	array	N/A		N/A		
	vector	No		N/A		Insertion changed capacity
		Yes		Yes		Before modified element(s) (for insertion only if capacity didn't change)
		No		No		At or after modified element(s)
	deque	No	Yes	Yes, but erased one(s)		Modified first or last element
			No	No		Modified middle only
	list	Yes		Yes, but erased one(s)		
	forward_list	Yes		Yes, but erased one(s)		
Associative Containers	set multiset map multimap	Yes		Yes, but erased one(s)		
Unordered associative Containers	unordered_set unordered_multiset	No	Yes	N/A		Insertion caused rehash
	unordered_map unordered_multimap	Yes		Yes, but erased one(s)		No rehash

Containers Bibliography

- [49] *Containers Library*. [Cplusplusreference](#).
- [50] *array*. [Cplusplusreference](#).
- [51] *vector*. [Cplusplusreference](#).
- [52] *deque*. [Cplusplusreference](#).
- [53] *forward list*. [Cplusplusreference](#).
- [54] *list*. [Cplusplusreference](#).
- [55] *set*. [Cplusplusreference](#).
- [56] *map*. [Cplusplusreference](#).
- [57] *multiset*. [Cplusplusreference](#).
- [58] *multimap*. [Cplusplusreference](#).
- [59] *unordered set*. [Cplusplusreference](#).
- [60] *unordered map*. [Cplusplusreference](#).
- [61] *unordered multiset*. [Cplusplusreference](#).
- [62] *unordered multimap*. [Cplusplusreference](#).
- [63] *stack*. [Cplusplusreference](#).
- [64] *queue*. [Cplusplusreference](#).
- [65] *priority queue*. [Cplusplusreference](#).
- [66] *flat set*. [Cplusplusreference](#).
- [67] *flat map*. [Cplusplusreference](#).
- [68] *flat multiset*. [Cplusplusreference](#).
- [69] *flat multimap*. [Cplusplusreference](#).

4

CHAPTER

Memory Management

1 Memory Management

Allocation means to assign, allot, distribute or "set apart for a particular purpose". Programs manage their memory partitioning or dividing it into different units performing specific tasks.

In C++, there are two ways to create objects and this aspect reflects on the existence of two different units used to allocate the memory, the **stack** and **heap**, which manage the program's used/unused memory in two different ways.

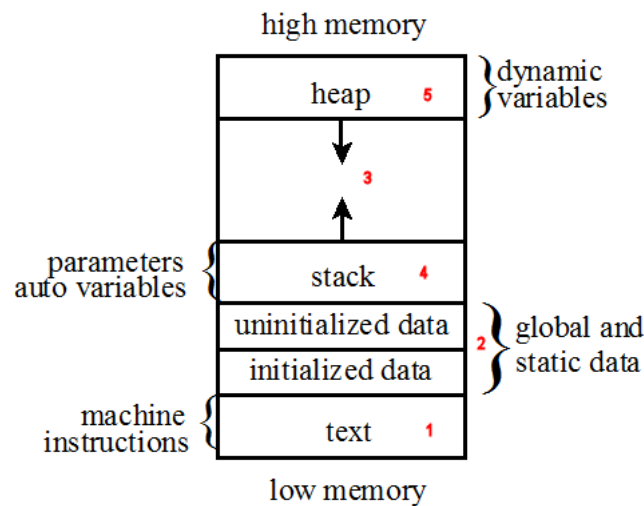


Figure 4.1: *Functional memory organization of a running program*

Text

This area contains machine instructions (i.e. the executable code). **Global** and **static** variables are typically allocated at program startup. **Stack** contains all the automatic variables. **Heap** is meant to allocate the memory for the dynamic variables through the **new** and **delete** operators. The space in between is allocated by the operating system but not used, and is meant to let both the heap and stack grow.

Stack

This is a simple **LIFO** (last in, first out) data structure which must support at least the **push and pop operations**. Even though a stack only allows accesses at the top, it is easy to implement and fast to execute push and pop operations.

Heap

This is a **more flexible data structure**, which allows programs to allocate or deallocate memory anywhere and in any convenient order. This might cause **holes** in the heap, i.e. unallocated memory surrounded by allocated memory.

The only restriction for the heap is that **memory can be allocated in a contiguous block of memory large enough to satisfy the request with a single chunk of memory**. This restriction has **two effects**: it requires the operating system **to scan the heap** until a contiguous block is found, and **to coalesce adjacent blocks of memory** returned by the program.

Side effect is that the operation on the heap are slower than the operations on the stack.

1.1 Memory Allocation and Deallocation in C

<code>void* malloc(std::size_t size)</code>	Allocates size bytes of uninitialized storage. On success, returns the pointer to the beginning of newly allocated memory. To avoid a memory leak, the returned pointer must be deallocated. On failure, returns a null pointer.
<code>void free(void* ptr);</code>	Deallocates the space previously allocated • If ptr is a null pointer, the function does nothing. • The behavior is undefined if the value of ptr does not equal a value returned by an earlier allocation the memory area referred to by ptr has already been deallocated, after std::free returns, an access is made through the pointer ptr.

In the previous table, allocation is meant to be done by one of the following:

`std::malloc`, `std::calloc`, `std::aligned_alloc` (since C++17), or `std::realloc`.

Deallocation is meant to be done by one of the following:

`std::free()` or `std::realloc()`

1.2 Memory Allocation and Deallocation in C++

<code>new expression</code>	Creates and initializes objects with dynamic storage duration, that is, objects whose lifetime is not necessarily limited by the scope in which they were created.
<code>delete expression</code>	Destroys object(s) previously allocated by the new-expression and releases obtained memory area.
<code>operator new</code>	
<code>operator delete</code>	

placement new

Memory Management Bibliography

- [104] *Memory Management: Stack and Heap*. [CppCon YouTube Video](#).
- [105] *Leak-freedom in C++ ... By Default - CppCon 2016*. [CppCon YouTube Video](#).
- [106] *malloc*. [Cppreference](#).
- [107] *calloc*. [Cppreference](#).
- [108] *aligned alloc*. [Cppreference](#).
- [109] *realloc*. [Cppreference](#).
- [110] *free*. [Cppreference](#).
- [111] *new expression*. [Cppreference](#).
- [112] *delete expression*. [Cppreference](#).
- [113] *operator new*. [Cppreference](#).
- [114] *operator delete*. [Cppreference](#).

5

CHAPTER

Smart Pointers

1 Smart Pointers in C++

Cppreference documentation about **dynamic memory management**.

Smart pointers enable *automatic, exception-safe, object lifetime management*.

Generally speaking, a smart pointer is a *data type whose value can be used like a pointer*, with the *additional feature of memory management*, that *frees the memory when it is no longer used*.

A smart pointer should therefore be used any time the ownership of memory needs to be tracked.

1.1 `std::unique_ptr`

cppreference documentation about `std::unique_ptr`.

`std::unique_ptr` is a smart pointer that *owns and manages* another object through a pointer and *disposes of* that object *when* the `std::unique_ptr` goes *out of scope*.

The object is disposed of, using the associated deleter when either of the following happens:

- the managing `std::unique_ptr` object is *destroyed*
- the managing `std::unique_ptr` object is *assigned another pointer* via `operator=` or `reset()`.

Notes

Only **non-const** `std::unique_ptr` can *transfer the ownership* of the managed object to **another** `std::unique_ptr`.

If an object's lifetime is managed by a **const** `std::unique_ptr`, it is *limited to the scope in which the pointer was created*.

`std::unique_ptr` is commonly used to *manage* the *lifetime of objects*, including:

- providing *exception safety* to classes and functions that handle objects with dynamic lifetime, by *guaranteeing deletion* on both normal exit and exit through exception
- *passing ownership* of uniquely-owned objects with dynamic lifetime into functions
- *acquiring ownership* of uniquely-owned objects with dynamic lifetime from functions
- as the element type in move-aware containers, such as `std::vector`, which hold pointers to dynamically-allocated objects (e.g. if *polymorphic behavior* is desired).

`std::unique_ptr` may be constructed for an *incomplete type* `T`, such as to facilitate the use as a handle in the *pImpl idiom*.

Member types

pointer	Must satisfy NullablePtr
element_type	T, the type of the object managed by this <code>std::unique_ptr</code>
deleter_type	Deleter, the function object or lvalue reference to function or to function object, to be called from the destructor

Member functions

constructor	constructs a new <code>unique_ptr</code>
destructor	destructs the managed object if such is present
operator=	assigns the <code>unique_ptr</code>

Modifiers

release	returns a pointer to the managed object
reset	returns the deleter that is used for destruction of the managed object
swap	checks if there is an associated managed object

Observers

get	returns a pointer to the managed object
get_deleter	returns the deleter that is used for destruction of the managed object
operator bool	checks if there is an associated managed object

Single-object version, `unique_ptr`

operator* operator->	dereferences pointer to the managed object
--	--

1.2 `std::shared_ptr`

Cppreference documentation about `std::shared_ptr`.

This is a smart pointer that **retains shared ownership** of an object through a pointer.

Several `shared_ptr` objects may own the same object.

The object is destroyed and its memory deallocated when either of the following happens:

- the **last remaining** `shared_ptr` owning the object **is destroyed**;
- the **last remaining** `shared_ptr` owning the object is **assigned another pointer** via `operator=` or `reset()`.

A `std::shared_ptr` can share ownership of an object while storing a pointer to another object.

A `std::shared_ptr` may also own no objects, in which case it is called empty (an empty `std::shared_ptr` may have a non-null stored pointer if the aliasing constructor was used to create it).

Member functions

<code>constructor</code>	constructs a new <code>unique_ptr</code>
<code>destructor</code>	destructs the managed object if such is present
<code>operator=</code>	assigns the <code>unique_ptr</code>

Modifiers

<code>reset</code>	replaces the managed object
<code>swap</code>	swaps the managed objects

Observers

<code>get</code>	returns the stored pointer
<code>operator*</code> <code>operator-></code>	dereferences the stored pointer
<code>operator[]</code>	provides indexed access to the stored array
<code>use_count</code>	returns the number of <code>shared_ptr</code> objects referring to the same managed object
<code>unique</code>	checks if the managed object is managed only by the current <code>shared_ptr</code> instance
<code>operator bool</code>	checks if the stored pointer is not null
<code>owner_before</code>	provides owner-based ordering of shared pointers

1.3 `std::weak_ptr`

[cppreference documentation about `std::weak\_ptr`](#).

`std::weak_ptr` is a smart pointer that holds a non-owning ("weak") reference to an object that is managed by `std::shared_ptr`. It must be converted to `std::shared_ptr` in order to access the referenced object.

`std::weak_ptr` models temporary ownership

Member functions

constructor	creates a new <code>weak_ptr</code>
destructor	destroys a <code>weak_ptr</code>
operator=	assigns a <code>weak_ptr</code>

Modifiers

reset	releases the ownership of the managed object
swap	swaps the managed objects

Observers

use_count	returns the number of <code>shared_ptr</code> objects that manage the object
expired	checks whether the referenced object was already deleted
lock	creates a <code>shared_ptr</code> that manages the referenced object
owner_before	provides owner-based ordering of weak pointers

1.4 `std::auto_ptr`

[cppreference](#) documentation about `std::auto_ptr`.

It was deprecated in C++11 and removed in C++17.

Smart Pointers Bibliography

- [146] *Dynamic Memory Management Library*. [Cplusplusreference](#).
- [147] *unique pointer*. [Cplusplusreference](#).
- [148] *shared pointer*. [Cplusplusreference](#).
- [149] *weak pointer*. [Cplusplusreference](#).
- [150] *auto pointer*. [Cplusplusreference](#).

6

CHAPTER

C++

1 C++: General Knowledge and Good Practices

1.1 Rule of three/five/zero

Rule of three

If a class requires a *user-defined destructor*, a user-defined *copy constructor*, or a user-defined *copy assignment operator*, it almost certainly requires all three.

Because C++ copies and copy-assigns objects of user-defined types in various situations (passing/returning by value, manipulating a container, etc), these special member functions will be called, if accessible, and if they are not user-defined, they are implicitly-defined by the compiler.

The implicitly-defined special member functions are typically incorrect if the class manages a resource whose handle is an object of non-class type (raw pointer, POSIX file descriptor, etc), whose destructor does nothing and copy constructor/assignment operator performs a "shallow copy" (copy the value of the handle, without duplicating the underlying resource).

Classes that manage non-copyable resources through copyable handles may have to declare copy assignment and copy constructor private and not provide their definitions or define them as deleted. This is another application of the rule of three: deleting one and leaving the other to be implicitly-defined will most likely result in errors.

Rule of five

Because the presence of a user-defined destructor, copy-constructor, or copy-assignment operator prevents the implicit definition of the move constructor and the move assignment operator, *any class for which move semantics are desirable*, has to declare all five special member functions.

Unlike Rule of Three, failing to provide move constructor and move assignment is usually not an error, but a missed optimization opportunity.

Rule of zero

Classes that have custom destructors, copy/move constructors or copy/move assignment operators should deal exclusively with ownership (which follows from the Single Responsibility Principle). Other classes should not have custom destructors, copy/move constructors or copy/move assignment operators. (see [2])

This rule also appears in the C++ Core Guidelines as C.20: If you can avoid defining default operations, do.

When a *base class is intended for polymorphic use*, its destructor may have to be declared public and virtual. This blocks implicit moves (and deprecates implicit copies), and so the special member functions have to be declared as defaulted (see [3])

However, this makes the class prone to slicing, which is why polymorphic classes often define copy as deleted (see [4]), which leads to the following generic wording for the Rule of Five: If you define or =delete any default operation, define or =delete them all (see [5])

1.2 The Most Vexing Parse

Examples

The term "*most vexing parse*" was first used by Scott Meyers in 2001. While unusual in C, the phenomenon was quite common in C++ until the introduction of uniform initialization in C++11.

This issue is about ambiguous code line like

```
1 void f(double my_dbl)
2 {
3     int i(int(my_dbl));
4 }
```

in which the second line can be interpreted both as *a declaration of a variable* `i` whose initial value is obtained by converting `my_dbl` into an `int` and as *a function declaration* equivalent to

```
1 int i(int my_dbl);
```

A similar ambiguity arises for

```
1 struct Foo {};  
2  
3 struct FooKeeper {  
4     explicit FooKeeper(Foo f);  
5     int get_foo();  
6 };  
7  
8 int main() {  
9     FooKeeper foo_keeper(Foo());  
10    return foo_keeper.get_foo();  
11 }
```

in which the line

```
1 TimeKeeper time_keeper(Timer());
```

can be interpreted both as a variable definition and a function declaration. The C++ standard requires the second interpretation, which is not consistent with the previous code.

How to fix it

It is possible to solve this parsing issue in different ways.

In the type conversion example

- C-style cast

```
1     int i((int)my_dbl);  
2
```

- C++ named cast (to prefer to the previous solution)

```
1     int i(static_cast<int>(my_dbl));  
2
```

In the variable declaration example, since C++11 the preferred method is to use uniform brace initialization

```
1 //Any of the following work:  
2 FooKeeper foo_keeper(Foo{});  
3 FooKeeper foo_keeper{Foo()};  
4 FooKeeper foo_keeper{Foo{}};  
5 FooKeeper foo_keeper({});  
6 FooKeeper foo_keeper{{}};
```

Prior to C++11, extra parenthesis or copy-initialization solves the most vexing parse issue

```
1 FooKeeper foo_keeper( /*Avoid MVP*/ (Foo()) );  
2 FooKeeper foo_keeper = FooKeeper(Foo());
```

1.3 General Topics

Built-in / Intrinsic / Primitive Data Types

A **POD type** is a C++ type that has an equivalent in C, and that uses the same rules as C uses for initialization, copying, layout, and addressing.

The actual definition of a POD type is recursive and gets a little gnarly.

Here's a slightly simplified definition of POD:

- a **POD type's non-static data members** must be **public** and can be of any of these types: **bool**, **any numeric type** including the various char variants, **any enumeration type**, **any data-pointer type** (that is, any type convertible to void*), **any pointer-to-function type**, or **any POD type**, including **arrays of any of these**.
- **data-pointers and pointers-to-function** are okay, but **pointers-to-member** are not.
- **references** are not allowed
- a POD type can't have **constructors**, **virtual functions**, **base classes**, or an **overloaded assignment operator**.

When **initializing non-static data members** of built-in / intrinsic / primitive types, it is always better to initialize all non-static data members in the constructor's **initialization list**. In order to initialize your built-in / intrinsic / primitive variable by an expression that the compiler doesn't evaluate solely at compile-time, concern should be put on the initialization order fiasco, explained in 1.4.

A function should return

- **void** if there is no need for a return value;
- **local by value**
- **local by pointer**
- **data member by value**
- **data member by pointer**
- **data member by reference-to-nonconst**
- **data member by referenec-to-const**
- **shared_ptr**
- local **unique_ptr** or **shared_ptr**
- **other**
-

Input/Output via <iostream> and <cstdio>

There are several reasons why <iostream> should be preferred to <cstdio>

- **type-safeness** is better guaranteed since <iostream> lets the compiler know the I/O object statically, while <cstdio>
- **less error prone**
- **extensibility**
- **inheritance**

check for invalid input

```
1 #include <iostream>
2 int main()
3 {
4     std::cout << "Enter a number, or -1 to quit: ";
5     int i = 0;
6     while (std::cin >> i) {      // GOOD FORM
7         if (i == -1) break;
8         std::cout << "You entered " << i << '\n';
9     }
10    // ...
11 }
```

skip invalid input

```
1 // ...
2 while ((std::cout << "How old are you? ")
3         && (!(std::cin >> age) || age < 1 || age > 200)) {
4     std::cout << "That's not a number between 1 and 200; ";
5     std::cin.clear();
6     std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
7 }
8 // ...
```

Functioning of (std::cin » foo)

stop processing past the end of file

Ending output lines with std::endl of '\n'

Providing printing for an entire hierarchy of classes

println() method vs. friend function

providing input for a class

reopen std::cin and std::cout in binary mode

writing/reading objects to/from a data file

sending objects to another computer

printing a char as a number

Const Correctness

ISOCpp FAQ about [const correctness](#).

Const correctness consists in using the keyword `const` to prevent const objects from being mutated.

`const` and pointer combinations

Reading it right-to-left

<code>const X* p</code>	<code>p</code> is a pointer to an <code>X</code> that is constant.
<code>X const* p</code>	equivalent to <code>const X* p</code> .
<code>X* const p</code>	<code>p</code> is a const pointer to an <code>X</code> that is not const.
<code>const X* const p</code>	<code>p</code> is a const pointer to an <code>X</code> that is const.

The second combination is equivalent to the first one: the [const-on-the-right style](#) requires to write the `const` specifier on the right of the object it makes const. This style is also called [consistent const](#) since it is consistent with the way of defining a const member function. Moreover, it is also more consistent with pointer declarations.

`const` and reference combinations

Reading it right-to-left

<code>const X& r</code>	<code>r</code> aliases an <code>X</code> object, which can not be changed via <code>r</code>
<code>X const& r</code>	equivalent to <code>const X& r</code>
<code>X& const r</code>	nonsense functionally equivalent to <code>X& r</code>
<code>const X& const r</code>	nonsense functionally equivalent to <code>const X& r</code>

The equivalence between the second and the first combination is due to the same reason valid for the pointers.

The last two combinations are nonsense and should be avoided. They are redundant as a reference is always constant, in the sense that you can never resear a reference to make it refer to a different object.

Logical and Physical State

When reasoning in terms of const-correctness, it's important to distinguish the logical and the physical state of an object.

On the inside, an object has a [physical state](#), also called [concrete](#) or [bitwise](#) state. This state is the one visible to the programmer who implements the software. From the outside, the client has an access to the object restricted to public member functions and friend functions. What is visible is the [logical](#) or [abstract](#) or [meaningwise state](#).

A lookup functionality for a collection-object is a good example to show how there could be bits in the object's physical state that have no corresponding elements in the object's logical state. The lookup functionality might improve the performance using a cache to

store the already looked-up items. The implementation of the cache is part of the object's physical state but the implementation is internal and the details will probably not be exposed to the users, i.e. it is not part of the object's logical state.

constness of public member functions

The constness of a public method must make sense to the object's users, and those users can see only the object's logical state. Therefore, in this context the constness to take care of is the logical one.

- If a method changes any part of the object's logical state, it logically is a mutator; it should not be const even if (as actually happens!) the method doesn't change any physical bits of the object's concrete state.
- Conversely, a method is logically an inspector and should be const if it never changes any part of the object's logical state, even if (as actually happens!) the method changes physical bits of the object's concrete state.

return-by-reference and const member function

A const member function must not change nor allow its caller to change the logical state of the this object it's been called on.

If an inspector method needs to return a data member of a this object, it should return by const reference or by copy. For example

```
1 class Person
2 {
3     public:
4         const std::string& get_name() const;    // good one
5         std::string& get_lastname() const;      // wrong one
6         int get_age();                          // good one
7 }
```

Compilers won't always catch such a mistake, therefore it is responsibility of the programmer.

const overloading

const overloading is a way to achieve const correctness when there are an inspector and mutator method with the same name. One of the most common case is due to the subscript operators, which usually come in pair as in the following

```
1 class PersonList
2 {
3     public:
4         // Subscript operators often come in pairs
5         const Person& operator[] (unsigned index) const;
6         Person& operator[] (unsigned index);
7         // ...
8 };
```

Of course const-overloading can be used for other things than subscript operator. Following some links to the isocpp FAQs

- [Subscript operator for Matric class](#)
- [Matrix class made simpler](#)
- [Matrix using templates](#)
- [Matrix using templates and vectors](#)
- [Class Template's syntax and semantics](#)

mutable keyword

mutable keyword helps achieving the const correctness, when the code needs to make changes to a const object's physical state. An example could be again the cached previous lookup in 1.3. In this case, the cache would be marked as mutable and the compiler would let the programmer change it even in case of a const object because it would know that a change of that attribute would reflect in a change in the physical state, leaving the logical state as it was before.

Using const is better than removing the constness with a const_cast

Error converting F** → const F**

C++ allows the conversion F** → F const* but gives an error if a program tries to implicitly convert F** → const F**.

The reason why this conversion is not legal in C++ is that it would let silently and accidentally modify a const Foo object without a cast.

```
1 class Foo
2 {
3     public:
4     void modify(); // make some modification to the this object
5 };
6
7 int main()
8 {
9     const Foo x;
10    Foo* p;
11    const Foo** q = &p; // q now points to p;
12                        // this is (fortunately!) an error
13    *q = &x;           // p now points to x
14    p->modify();        // Ouch: modifies a const Foo!!
15    // ...
16 }
```

In case the third line of the main were legal, q would be pointing at p and the next line it would change p to point at x. This would made the const qualifier disappear. The last line would then use p ability to modify its referent and the program would end up

modifying a `const Foo`.

In case of necessity, the solution is to change `const Foo**` to `const Foo* const*`.

References

A [reference](#) is an alias for an object. At the assembly implementation level, a reference `i` to an object `x` is typically the machine address of the object `x`. When the programmer writes an instruction like `i++`, the compiler generates the code to increment the value of the referenced object `x`.

Assigning to a reference changes the state of the referenced object.

In case a function returns a reference, the function call can appear on the left hand of an assignment operator:

```
1 class Array {
2 public:
3     int size() const;
4     float& operator[] (int index);
5     // ...
6 };
7 int main()
8 {
9     Array a;
10    for (int i = 0; i < a.size(); ++i)
11        a[i] = 7;    // This line invokes Array::operator[](int)
12    // ...
13 }
```

Method chaining like `object.method1().method2()`, in which the returned object from `method1()` is the `this` object for `method2()` are possible because of return by reference.

The method chaining is used in the [Named Parameter Idiom](#), described in 1.10 (see also [143]).

Memory Management

The best techniques to prevent memory leaks rely on hiding allocation and deallocation inside more manageable types. Single objects can be managed through `make_unique` and `make_shared`, multiple objects can be managed through the container `vector` or `unordered_map`. Their memory management frees the programmer from explicit memory management, macros, casts, overflow checks, explicit size limits, pointers and similia.

Templates and the standard libraries make this use of containers, resource handles, etc., much easier than it was even a few years ago. The use of exceptions makes it close to essential.

In the following code, a `unique_ptr` is used as a resource handle in order not to cope with allocation/deallocation responsibilities when returning an object allocated on the freestore from a function

```

1 #include<memory>
2 #include<iostream>
3 using namespace std;
4 struct S {
5     S() { cout << "make an S\n"; }
6     ~S() { cout << "destroy an S\n"; }
7     S(const S&) { cout << "copy initialize an S\n"; }
8     S& operator=(const S&) { cout << "copy assign an S\n"; }
9 };
10 S* f()
11 {
12     return new S;    // who is responsible for deleting this S?
13 };
14 unique_ptr<S> g()
15 {
16     return make_unique<S>();    // explicitly transfer responsibility
17                                 // for deleting this S
18 }
19 int main()
20 {
21     cout << "start main\n";
22     S* p = f();
23     cout << "after f() before g()\n";
24     //S* q = g(); // this error would be caught by the compiler
25     unique_ptr<S> q = g();
26     cout << "exit main\n";
27     //leaks *p
28     //implicitly deletes *q
29 }

```

In case containers are not available, it is possible to use a memory leak detector as part of your standard development procedure, or plug in a garbage collector.

In C++11 a literal `nullptr` has been introduced and should be used as the null pointer value in place of the previous solutions (namely `NULL` and `0`).

The function of `delete` is deleting the data pointed by its operand. It must be used not more than once on the same pointer allocated with a `new`, assuming that pointer has not gone through a successive new assignment.

The `malloc()/free()` operations can not be mixed with `new/delete` operations. The reasons are that they do not operate on the same level: they allocate on `different heaps` and `the former work with raw memory` while `the latter work with constructed objects` whose proper construction and distruction are guaranteed.

More specifically, `malloc()` is a function that takes a number (of bytes) as its argument; it returns a `void*` pointing to unitialized storage and this pointer needs to be converted to a proper type. `new` is an operator that takes a type and (optionally) a set of initializers for that type as its arguments; it returns a pointer to an (optionally) initialized object of its type.

`malloc()` reports memory exhaustion by returning `0`. `new` reports allocation and initialization errors by throwing exceptions (`bad_alloc`).

Another important reason to use new and delete is their [overridability](#).

In case there is a need to use `realloc()`, it is better to use vector containers. When `realloc()` has to copy the allocation, it uses a bitwise copy operation, which will tear many C++ objects to shreds. C++ objects should be allowed to copy themselves.

C++ provides the programmer with `set_new_handler`, a function called by allocation functions whenever a memory allocation attempt fails.

In C++ there is not the need to check whether a pointer is null before delete-ing it. The process goes through two steps

```
1 // Original code: delete p;
2 if (p) { // or "if (p != nullptr)"
3     p->~Fred();
4     operator delete(p);
5 }
```

in which `operator delete(p)` calls the memory deallocation primitive, `void operator delete(void* p)`. C++ does not guarantee that `delete` also null out its operand. One reason is that the operand of `delete` is not required to be an lvalue.

In case it is important to null out the pointer, a possible implementation can be

```
1 template<class T> inline void destroy(T*& p) { delete p; p = 0; }
```

Passing a reference has also the effect to prevent the a call of this function template on an rvalue.

For what concerns the call to the destructor at the end of the scope, heap allocation should be used only if an object needs to live beyond the lifetime of the scope you create it in. Even then, `make_unique` or `make_shared`. In rare cases in which heap allocation is really needed and `new` is used to create an object, `delete` must be used to destroy the object, taking into account that code is no longer exception-safe.

In case an object must live in a scope only, heap allocation is not needed at all.

In case an array of objects is created with `new`, the `delete[]` operator must be used to destroy the array. There are several ways a compiler can keep memory of the number of elements to destroy

<https://isocpp.org/wiki/faq/compiler-dependencies#num-elems-in-new-array-overalloc>

<https://isocpp.org/wiki/faq/compiler-dependencies#num-elems-in-new-array-assocarray>

In case of interest, there are several code example to

- [allocating multidimensional arrays using new](#)
- [allocating multidimensional arrays encapsulating pointers in a class](#)
- [reworking the previous example in terms of templates](#)
- [building a multidimensiona array using std::vector](#)

In case a member function needs to delete this following requirements must be satisfied

- this object was allocated via `new` (not by `new[]`, nor by placement new, nor a local object on the stack, nor a namespace-scope / global, nor a member of another object)

- member function will be the last member function invoked on this object.
- the rest of your member function (after the delete this line) doesn't touch any piece of this object (including calling any other member functions or touching any data members). This includes code that will run in destructors for any objects allocated on the stack that are still alive.
- no one even touches the this pointer itself after the delete this line. In other words, you must not examine it, compare it with another pointer, compare it with nullptr, print it, cast it, do anything with it.

It's also possible to use the [Named Constructor Idiom](#) (see 1.7 and [139]) to force a class to be created via new rather than as local, namespace-scope, global or static

```

1 class Fred {
2 public:
3     // The create() methods are the "named constructors":
4     static Fred* create()           { return new Fred();      }
5     static Fred* create(int i)      { return new Fred(i);    }
6     static Fred* create(const Fred& fred) { return new Fred(fred); }
7     // ...
8 private:
9     // The constructors themselves are private or protected:
10    Fred();
11    Fred(int i);
12    Fred(const Fred& fred);
13    // ...
14 };

```

In case Fred class is expected to have derived classes, the constructors must be in protected section of the class.

1.4 Classes and Inheritance

Classes and Objects

A [class](#) is the building block of Object Oriented programming. It defines a data type, that consists of a set of states and a set of operations which transition between those states. A class provides a set of (usually public) operations, and a set of (usually non-public) data bits representing the abstract values that instances of the type can have.

An object is a region of storage with associated semantics defined in the class it belongs to.

An interface is good when

- unnecessary details are intentionally hidden.
This reduces the user's defect-rate.
- users don't need to learn a new set of words and concepts.
This reduces the user's learning curve.

Encapsulation is the mechanism used to prevent unauthorized access to some piece of information or functionality of a class/object.

The key money-saving insight is to separate the volatile part of some chunk of software from the stable part.

The **volatile parts** are the implementation details. In case of a single class this part is encapsulated in the **private/protected** part of the class. Inheritance can also be used as a form of encapsulation.

The **stable parts** are the interfaces. Good ones provide simplified view in the vocabulary of users, and are designed from the outside-in. !!! PUT LINK TO OPERATOR OVERLOADING !!!

If there is a single class, the interface is simply the class's public member functions and friend functions. The private and/or protected parts of a class contain the class's implementation, which is typically where the data lives.

The clean interface separates the interface from its implementation, and encapsulation forces users to use the interface.

Encapsulation does not prevent a user to read the private and/or protected parts of a class, but does prevent the code from being dependent on the insides of the class.

A class method can directly access the non-public members of another instance of its class. This rule of C++ preserves the encapsulation. Otherwise there would be the need to provide a public `get_method` for each private variable used by the assignment operator, the copy constructor, the comparison operators, the binary arithmetic and bitwise operators, and many others.

Methods and friends of a derived class can access the protected base class members of any of its own objects, but not others. Suppose the classes D1 and D2 inherit from class B, that has a private `x` member. In this scenario, the compiler let D1's members and friend directly access the `x` member of any D1. The compiler gives a compile-time error if D1 member or friend tries to directly access the `x` member of anything it does not know is at least a D1, such as via a `B*` pointer, a `B&` reference, a `B` object, a `D2*` pointer, a `D2&` reference, a `D2` object.

The members and base classes of a struct are public by default, while in class, they default to private. For the other aspects are functionally equivalent.

To define an in-class constant useable in a compile time constant expression, there are three choices

```
1 class X {
2     constexpr int c1 = 42; // preferred since C++11
3     static const int c2 = 7;
4     enum { c3 = 19 };
5     array<char, c1> v1;
6     array<char, c2> v2;
7     array<char, c3> v3;
8     // ...
9 };
```

In case the constant isn't needed in a compile time constant expression

```
1 class Z {
```

```

2     static char* p;      // initialize in definition
3     const int i;        // initialize in constructor
4 public:
5     Z(int ii) :i(ii) { }
6 };
7 char* Z::p = "hello, there";

```

It is possible to bind a reference to a static data member (to take its address), if and only if it has an out-of-the-class definition

```

1 class AE {
2     // ...
3 public:
4     static const int c6 = 7;
5     static const int c7 = 31;
6 };
7 const int AE::c7;    // definition
8 void byref(const int&);
9 int f()
10 {
11     byref(AE::c6);    // error: c6 not an
    lvalue
12     byref(AE::c7);    // ok
13     const int* p1 = &AE::c6;    // error: c6 not an lvalue
14     const int* p2 = &AE::c7;    // ok
15     // ...
16 }

```

Like C, C++ doesn't define layouts, just semantic constraints that must be met.

The size of an empty class isn't zero to ensure that two different objects will have different addresses. For the same reason, new always return pointers to different objects.

The Empty Base Optimization (see [42]) is a mandatory requirement on class layout since C++11, and requires that the size of any object or member subobject is required to be at least 1 even if the type is an empty class type'

Constructors

Destructors

Destructors are used to release any resource allocated by the object, like a semaphore, a file and so on. A very common case is delete-ing the object created using new.

Destructors are invoked on a local scope in reverse order of construction in the code.

The objects stored in an array are destructed in reverse order of construction as well.

The sub-objects are destructed in reverse order of construction as well. For example, having this structure

```

1 struct derived: base0, base1
2 {
3     member0 m0_;
4     member1 m1_;
5     ~derived()
6     {

```



```

7     local0 10;
8     local1 11;
9 }
10 }

```

when an derived object goes out of scope, destructors are called in this order: `local1()`, `local0()`, `member1()`, `member0()`, `base1()`, `base0()`

Destructors can not be overloaded.

Destructors need not to be called on local variables. In case it is needed to destroy a local object in a specific point of a local scope, it is possible to use an artificial block `{...}`. In case it's not possible to wrap the object with an artificial block, the solution is to create a function with similar duties.

If the object to destroy was allocated via `new`, a call to the destructor isn't needed unless it was a placement `new`

```

1 void someCode ()
2 {
3     char memory[sizeof(Fred)];
4     void* p = memory;
5     Fred* f = new(p) Fred();
6     // ...
7     f->~Fred();    // Explicitly call the destructor for the placed
                     // object
8 }

```

Is there a placement delete?

From a class destructor, it is not needed to explicitly call destructor of neither member objects nor base classes.

Is there a way to force new to allocate memory from a specific memory area?

Assignment Operator

Operator Overloading

Friends

Cppreference documentation about Friends

A `friend` can be a `function`, a `class`, and both their `template` version.

The friend declaration appears in a class body and grants a function or another class access to `private` and `protected` members of the class where the friend declaration appears.

`friend` declaration helps enforcing encapsulation. Indeed, granting access to private and protected members removes the burden of creating public `getters` and `setters`, since in most of the cases these methods destroy encapsulation, exposing private and protected details which make no sense outside of the class.

The disadvantage of friend functions is that they require extra code when dynamic binding is needed. Indeed, the friend function should call a hidden (usually protected)

virtual member function, as explained in details in 1.11.

Friendship *is not inherited, transitive or reciprocal*:

- a derived class of a friend class is not a friend class;
- a friend of a friend is not necessarily a friend;
- if class A declares class B its friend, B objects have access to A objects' private data, but the opposite is not true.

Usually, it is better to rely on member function if possible and friend ones when it is really needed.

Inheritance - Basics

static initialization

Inheritance - virtual member functions

.

A **virtual member function** is the way of C++ to allow derived classes to replace the implementation provided by the base class. The compiler makes sure the correct derived class' implementation is called, even when the object is accessed through a pointer to the base class.

The reason why the member function are not virtual by default is that many classes are not designed to be used as a base class. Moreover, the virtual function call mechanism needs requires an overhead of memory, typically one word per object.

Dynamic Binding and Static Typing

A pointer to an object may actually point to a class that is derived from the class of the pointer. Thus there are two different types to consider: the (static) type of the pointer and the (dynamic) type of the actual pointed object.

Static typing means that the legality of a member function invocation is checked as early as possible, i.e. at compile time by the compiler itself. In this kind of check, the compiler uses the *static type* of the pointer to determine whether the member function invocation is correct, meaning that the type pointed by the pointer can handle the member function.

Dynamic binding is the virtual function call mechanism, which checks the correctness of the function call at the last possible moment, i.e. at run time and based on the dynamic type of the object pointed to.

Member Function call: virtual vs. non-virtual

The practical difference is that non-virtual member functions are resolved statically, i.e. the compiler checks the *type of the pointer or reference to the object* and selects the correct member function statically at compile time.

On the other side, a virtual member function call is resolved dynamically.

virtual Member Function Call - Overhead Estimation

As explained, a non-virtual member function is resolved statically based on the type of the pointer or reference to the object.

In contrast, **dynamic binding** select the correct virtual member function at run time following a compiler-based variant of the following technique: for each virtual function, the compiler create in each object an hidden **v-pointer** (virtual pointer) pointing to a global table called **v-table** (virtual table). One single **v-table** is created for each class which contains at least a **virtual** function.

To resolve a call to a **virtual** function, the run-time procedure follows the object's **v-pointer** to the class's **v-table**, and then follows the appropriate slot in the **v-table** to the method code.

The **space-cost** overhead consists of an extra pointer per object needing to be dynamically binded and another extra pointer per virtual method. The **time-space** overhead compared to a normal function call consits of the two extra fetches to get the value of the **v-pointer** and a second one to get the address of the method.

Suppose to have a **Base** class with 5 **virtual** functions

```
1 // Original C++ source code
2 class Base
3 {
4     public:
5         virtual return_type virt0( /*...arbitrary params...*/ );
6         virtual return_type virt1( /*...arbitrary params...*/ );
7         virtual return_type virt2( /*...arbitrary params...*/ );
8         virtual return_type virt3( /*...arbitrary params...*/ );
9         virtual return_type virt4( /*...arbitrary params...*/ );
10        // ...
11};
```

There are three steps the compiler goes through

First step: building a static table

Compiler build a static table containing 5 function-pointers, most likely while compiling the .cpp that define the Base's first non-inline virtual function.

Let's call this table **Base::_vtable**. If a function pointer fits into one machine word, the **v-table** adds 5 hidden words of memory overall.

```
1 // FunctionPtr is a generic pointer to a generic member function
2 FunctionPtr Base::_vtable[5] = {
3     &Base::virt0, &Base::virt1, &Base::virt2,
4     &Base::virt3, &Base::virt4 };
```

Second step: adding an hidden pointer

An hidden pointer called **v-pointer** is added by the compiler to **each object of Base** class as it was an hidden data member.

```
1 // Original C++ source code
2 class Base
```

```

3 {
4     public:
5     // ...
6     FunctionPtr* __vptr; // Supplied by the compiler,
7                           // hidden from the programmer
8     // ...
9 };

```

Third step: initializing this->__vptr

Compiler initializes `this->__vptr` within each constructor in order to let each object's `v-pointer` to point at its class `v-table`. This step could be thought of as if the following instruction is added in each constructor's *init list*.

```

1 Base::Base( /*...arbitrary params...*/ )
2     : __vptr(&Base::__vtable[0]) // Supplied by the compiler,
3                                   // hidden from the programmer
4     // ...
5 {
6     // ...
7 }

```

When the code defines a `Der` class which inherits from the `Base` class, the compiler repeats the first and third steps, but *skips the second step*.

In the **first step**, compiler creates another hidden `v-table`, in which the compiler replaces the virtual methods overridden in the derived class. For example, in case `Der` overrides `virt0()...virt2()` the `Der`'s `v-table` looks like

```

1 // Pseudo-code (not C++, not C)
2 // for a static table defined within file Der.cpp
3 FunctionPtr Der::__vtable[5] = {
4     &Der::virt0, &Der::virt1, &Der::virt2,
5     &Base::virt3, &Base::virt4 };

```

In the **third step**, the compilers for each `Der` object changes the `v-pointer` in order to make it point to its class `v-table`.

At this point, it is possible to call a virtual function with code like

```

1 void mycode(Base* p)
2 {
3     p->virt3();
4 }

```

The compiler rewrites the code like

```

1 void mycode(Base* p)
2 {
3     p->__vptr[3](p);
4 }

```

At this point

- a *first load* gets the `v-pointer` and stores it in a register which we call `r1`

- a *second load* gets the work at $r1 + 3 \times 4$ (if function-pointers are 4-bytes long) and stores it in a second register r2
- *compiler calls* the code at location r2

Therefore, it is possible to state that:

- a *minimal space-overhead* is needed to manage virtual functions
- virtual function calls are almost *as fast as* non-virtual function calls
- there is *no chaining effect*, i.e. the fetch-fetch-call mechanism is not dependent on how deep the inheritance gets.

Calling a Base Class virtual member function

When a virtual function is called by its *fully-qualified name*, the compiler skip the virtual call mechanism and uses the same mechanism used for non-virtual functions. Therefore, the fully-qualified virtual function is called by name rather than by slot-number.

To call the virtual function defined in the Base within the derived class, it is possible to write

```
1 void Der::f()
2 {
3     Base::f();
4 }
```

virtual destructor

A virtual destructor is required any time someone deletes a derived-class object through a base-class pointer. In this case, indeed, the destructor invoked matches the type of the pointed object ignoring the type of the pointer itself.

In general, a virtual destructor is needed

- if the class is meant to be *derived from*
- *and* if code uses `new Derived`
- *and* if code uses `delete p` in which p is a pointer to Base pointing to an actual object of Derived class.

To be shorter, destructor needs to be virtual any time there is a virtual method in the class. Indeed

- it's a general safety measure in case the class will be used as a base class;
- it costs nothing since the

virtual constructor

An idiom that allows to achieve a behavior that C++ does not really support.

[virtual-ctors](#)

[copy-of-abc-via-clone](#)

Inheritance - Proper Inheritance and Substitutability

Converting `Derived**` → `Base**`

Differently from the conversion `Derived*` → `Base*`, the conversion mentioned in the title does not compile, because it would be dangerous.

Even though a `Derived` object is a kind of `Base` object, in case the conversion in the title was possible the `Base**` could be dereferenced and it could be possible to make `Base*` point to an object of a [different derived class](#).

This prohibition has similarities with 1.3.

container of `Thing` == kind-of container of `Everything`?

As a consequence of 1.4, a container of a certain `class Thing` objects is not a container of `class Anything` objects, even if `Thing` is a kind of `Anything`.

Is an array of `Derived` a kind-of array of `Base`?

In the same perspective of the previous two paragraphs, the answer is negative and easy to explain: a `Derived` object is larger than a `Base` object and it would mess up the pointer arithmetics in case it was possible to store both the kinds of objects in the same array.

Inheritance - Abstract Base Classes

At the design level, an `ABC` corresponds to an abstract concept. At the programming language level, an `ABC` is a class containing one or more pure `virtual` functions, as in the following example

```
1 class Shape
2 {
3 public:
4     virtual void draw() const = 0;    // = 0 means "pure virtual"
5     // ...
6 };
```

Even though it is possible to provide a definition of a pure `virtual` function, it could be confusing.

Copy Constructor and Assignment Operator for a class containing a pointer to a (abstract) base class

If a class owns 1.4

Inheritance - What your mother never told you

final classes and virtual methods

Calling virtual function from non-virtual functions of base class

Inheritance - private and protected inheritance

Private and protected inheritance are expressed using respectively the keyword `private` and `protected` in place of `public` in the definition of the derived class.

`private` inheritance is a syntactic variant of the concepts of composition or aggregation. For example, the "`Car has-a Engine`" relationship can be implemented both using single composition and `private` inheritance. For the latter, an example is

```
1 class Car : private Engine
2 {
3     public:
4     Car() : Engine(8) {}    // in case Engine has a ctor
5                             // which takes an int
6     using Engine::start;
7 }
```

`private` inheritance has both similarities and differences when compared to composition. Some of the similarities are

- a single `Engine` is present in both cases;
- in neither case a conversion `Car* → Engine*` can be done by users;
- `Car` has a `start()` method that calls `Engine`'s `start` method.

The differences are

- simple-composition is needed in case multiple `Engine` are contained in a `Car`;
- `private` inheritance can lead to unnecessary highlightbfmultiple inheritance
- `private` inheritance allows member of `Car` to convert `Car* → Engine*`
- `private` inheritance allows access to `protected` members of the base class
- `private` inheritance allows `Car` to override `Engine`'s `virtual` functions
- `private` inheritance make it simpler to implement a `Car`'s `start()` method that calls the `Engine`'s `start()` method.

Which should I prefer: composition or private inheritance?

Composition should be generally preferred to `private` inheritance. The latter gives access to the internals and there could be many class to expose. This aspect could make the program harder to maintain and easy to break when something in the code changes. A scenario in which `private` inheritance is *preferred* is when a `class Foo` uses code in `Class Bar` and the code in the latter class invokes member functions in the former class. The solution is to let `class Foo` call non-virtual in `class Bar`, and `class Bar` calls its own (pure) `virtual`s, which are overridden by `class Foo`.

Should I pointer-cast from a private derived class to its base class?

It is generally not a good practice.

The conversion `PrivatelyDer* → Base*` (and likewise for the references) is safe and no cast is needed or recommended. However, users of `PrivatelyDer` should avoid this conversion, since a private decision is subject to change without notice.

How is protected inheritance related to private inheritance?

Similarly, `protected` inheritance allows to override `virtual` functions in the base class and does not claim the derived is a kind-of-base class.

Dissimilarly, `protected` inheritance allows derived classes of derived classes to know about the inheritance relationship. The benefit is the capability of the *protected derived class* to exploit the relationship to the base class. The *cost* is the fact that changes the relationship in the protected derived class can potentially break further derived classes.

Access rules with private and protected inheritance

Considering the following code

```
1 class B { /*...*/ };
2 class D_priv : private B { /*...*/ };
3 class D_prot : protected B { /*...*/ };
4 class D_publ : public B { /*...*/ };
5 class UserClass { B b; /*...*/ };
```

- none of the derived classes can access anything private in B;
- in `D_priv`, the `public` and `protected` of B parts are `private`
- in `D_prot` the public and protected parts of B are protected;
- in `D_publ` the `public` parts of B are public and the `protected` parts are protected (`D_publ` is-a-kind-of-a B);
- in `class UserClass` can access only the `public` parts of B.

The correct way to make a `public` member `f(int, float)` of B public in `D_prot` or in `D_priv` makes use of the `scope operator` as the in the following code


```

1 class D_prot : protected B
2 {
3 public:
4     using B::f;
5 }

```

Inheritance - Multiple and Virtual Inheritance

Do we really need multiple inheritance?

Every modern language with static type checking and inheritance provides multiple inheritance. In C++, an *ABC* serves usually as an interface and class can need many. Other language simply have a separate name for pure abstract classes interfaces. Using multiple inheritance is not really common but it is better than using workarounds.

Disciplines for using multiple inheritance

- Inheritance should be used if it removes `if/switch` statements. Indeed, while composition is meant for code reuse, inheritance's primary purpose is dynamic binding and the flexibility which comes with it.
- *pure ABCs* play a crucial role in the context of *multiple inheritance*, since classes with as little data as possible and almost all pure virtual methods help avoiding inheritance for code reuse and using it for interface sustainability'
- The **bridge pattern** or **nested generalization** are possible alternatives and a wise designer checks out all the ways before choosing the best.

Example for the guidelines

Imagine a context in which there are multiple variants of a similar concepts, for example land, water, air and space vehicles, and different power sources, for example gas, wind, nuclear power and so on.

Before tying up everything using multiple inheritance, there are two aspects to take into account choosing a good strategy:

- users need to call methods on a *Vehicle*-reference and expect the implementation of those methods to be specific to a concrete vehicle, like *LandVehicle*;
- users need a *Vehicle*-reference that refers to a concrete vehicle, for example *GasPoweredVehicle*, and want to call methods on that *Vehicle*-reference expecting the implementation to get overridden by *GasPoweredVehicle*.

In case both the conditions stand, multiple inheritance could be the correct choice. Better alternatives could still be available depending on a few more decision criteria.

In case there are **N** geographies (land, water, air, space, e.c.) and **M** power sources, there are at least 3 options.

- **Bridge Pattern**

`Vehicle` and `Engine` are two pure ABCs, concrete vehicles and engines are derived from. `Vehicle` interface has an `Engine*` and everything is matched at run-time.

This way the code grows *gracefully*, because there are only $N+M$ derived classes to implement. There are several disadvantages as well. A first one is due to having at most $N+M$ overrides and therefore the same number of concrete algorithms and adding more could be very difficult.

Bridge Pattern is not able to solve nonsensical choices, as a pedal powered space vehicles and to solve this issue extra checks are needed as well.

Another issue is the absence of an interface for all the gas powered vehicles, which are considered just a kind of `Vehicle`. On the other side, all gas powered vehicles share their code in derived class `GasPoweredEngine`.

- **Nested Generalization**

One of the hierarchies is chosen as primary, and suppose to choose the geography as primary in the current example.

Concrete classes as `LandVehicle` and `WaterVehicle` are derived from `Vehicle` and each would have further derived classes, one per power source. For example, `LandVehicle` would be the base class for `GasPoweredLandVehicle`, `NuclearPoweredVehicle` and so on.

This way, the codebase does not grow gracefully because $N \times M$ different derived classes need to be implemented, the advantage is the possibility to have the same number of different algorithms.

This approach provides with a fine granular control removing the nonsensical combination because the combinations are decided in the design and should be reasonable.

Analougusly to *Bridge Pattern*, there is not a common base class for the specific vehicles, and finally it is not obvious how to share code between different vehicles with same power source.

- **Multiple Inheritance**

There are two different hierarchies as in the *Bridge Pattern*, the `Engine*` is no longer a data member of the vehicle classes though, and in its place roughly $N \times M$ derived classes are implemented below both the hierarchy of the geographies and the hierarchy of engines.

In this context, the concept of Engine changes and vehicle classes should be renamed, e.g. `GasPoweredEngine` would be renamed to `GasPoweredVehicle`. Moreover, `GasPoweredLandVehicle` multiply inherits from `GasPoweredVehicle` and `LandVehicle`. At the cost of not having a graceful growth, the gain is a fine granular control simply because nonsensical combinations like `PedalPoweredSpaceVehicle` are not created for the user.

In this model, it is also possible for any gas powered vehicle to share a common interface. Unlike the *Bridge Pattern*, the derived class can override the shared code

to replace parts that are not ideal for the new class.

Visualization of tradeoffs

Given N and M , the tradeoffs of the approaches described in 1.4 are summarized in table 6.1

	Grow Gracefully	Low Code Bulk	Fine-Grained Control	Static Detect Bad Combos	Polimorphic on both sides	Share Code
Bridge						
Nested Gen.						
Multiple In.						

Table 6.1: *Bridge Pattern, Nested Generalization and Multiple Inheritance's tradeoffs.*

The meaning of the columns are the following

- **Grow Gracefully**

For the example of the previous paragraph, the growth is due to adding more geographies or power sources and in this case it is linear.

- **Low Code Bulk**

- **Fine Grained Control**

It takes into account the option of having a different algorithm for any of the different $N \times M$ possibilities or being stuck with a single solution for all the concrete classes of vehicles.

- **Static Detect Bad Combos**

- **Polymorphic on Both Sides**

- **Share Common Code**

Second Example for the guidelines

This example is more symmetric than the previous one, resulting in being better managed in using a multiple-inheritance solution.

Suppose to start with two categories of vehicles, for example `LandVehicle` and `WaterVehicle`, and later an `AmphibiousVehicle` is needed as well.

Given this conditions

- it is really needed to have an amphibious vehicle, instead of using the other classes with a bit of changes;

- users need a `LandVehicle` reference that refers to an `AmphibiousVehicle` and need to call methods on the reference expecting the actual implementation is specific to the referenced object;
- same as the previous point, with `WaterVehicle`.

in this case the *multiple inheritance* would be the best choice. Answering the question seen in 1.4 could help being more confident with this choice.

The Dreaded Diamond

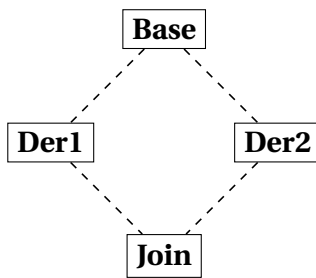


Figure 6.1: *A Dreaded Diamond.*

A *dreaded diamond* describes a class structure in which a particular class appears more than once in an inheritance hierarchy and looks like the diagram in figure 6.1.

Actually, C++ provides techniques to deal with the dreads of this triangle and this situation is not really dreaded but just something to be aware of.

The issue is due to `Base` being inherited twice with the consequence that any data member declared in `Base` appears twice within a `Join` object.

This duality creates ambiguities: when `Join` change a `Base`'s data member, it is not clear which derived class to go through for the change; likewise, a conversion `Join* → Base*` is ambiguous.

C++ lets resolve the ambiguity with the *scope operator*, there is a better solution though.

virtual Inheritance

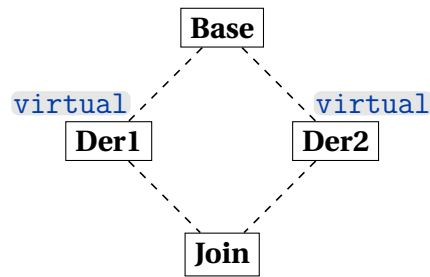


Figure 6.2: *A Nicer Diamond.*

To avoid duplicates in the `Join` class, every class that derives from `Base` needs to `virtual`-inherit. This way, an instance of `Join` will contain only one `Base` subobject and this solution is better than using fully qualified names.

delegate to a sister class

`virtual` inheritance helps achieving delegation to a sister class.

```
1 class Base
2 {
3 public:
4     virtual void foo() = 0;
5     virtual void bar() = 0;
6 };
7
8 class Der1 : public virtual Base
9 {
10 public:
11     virtual void foo();
12 };
13 void Der1::foo()
14 { bar(); }
15
16 class Der2 : public virtual Base
17 {
18 public:
19     virtual void bar();
20 };
21
22 class Join : public Der1, public Der2
23 {
24 public:
25     // ...
26 };
```

`Join` objects contain the `Der1`'s overridden `foo()` and the `Der2`'s overridden `bar()`. For this reason, when `Der1::foo()` calls `this->bar()` it ends up calling `Der2::bar()`.

Considerations needed when

- **using virtual inheritance**

Usually, `virtual` base classes and the ones that directly inherits from them are pure abstract classes, and contain very little if any data.

- **inheriting from a class that uses virtual inheritance**

The initialization list of the most-derived-class's constructor directly invokes the virtual base class's constructor.

C++ has a set of rules to make sure the `virtual` base class's constructor and destructor get called once per instance

- `virtual` base classes are constructed before all non-virtual base classes.
- constructors for `virtual` base classes anywhere in your class's inheritance hierarchy are called by the most derived class's constructor.

A consequence is the need to pass whatever parameters are required to call the `virtual` base class's constructor. In case of multiple `virtual` base classes, the most-derived class's constructor needs more parameters than other contexts require.

The practice of leaving the `virtual` base class without any data to initialize implies that the constructor probably takes no parameters.

- **using a class that uses virtual inheritance**

`dynamic_cast` is preferable to C-style downcasts.

Order of constructors in a multiple and/or virtual inheritance

First constructors to be executed are the virtual base classes' ones, in the order of appearance of base class names in the declaration of the derived class.

After the previous step, the construction order is from base class to derived class. Indeed, the compiler at first makes an hidden call to the non-virtual base classes' constructor in the derived class's constructor.

The previous rule is applied recursively: if `B1` inherits from `B1a` and `B1b`, and `B2` inherits from `B2a` and `B2b`, the final order of initialization is:

`B1a → B1b → B1 → B2a → B2b → B2 → D`.

The last order is determined by the order that the base classes appear in the declaration of the class.

Order of destructors in a multiple and/or virtual inheritance

For what concerns the non-virtual inheritance, the destructor order is the opposite of the constructor order:

`D → B2 → B2b → B2a → B1 → B1b → B1a`

Afterwards, the destructor of the virtual base classes that appear in the hierarchy are called, taking into account a class only when it uniquely appear for the first time.

1.5 Beyond Classes

Templates

A class template is a description of how to build a family of classes that all look basically the same, and a function template describes how to build a family of similar looking functions.

A class template is sometimes called **parameterized type** or **genericity**.

Class templates are often used to build type safe containers (although this only scratches the surface for how they can be used).

```
1 // This would go into a header file such as "Array.h"
2 template<typename T>
3 class Array {
4 public:
5     Array(int len=10)
6         : len_(len), data_(new T[len]) { }
7     ~Array() { delete[] data_; }
8     int len() const { return len_; }
9     const T& operator[](int i) const { return data_[check(i)]; }
10    T& operator[](int i) { return data_[check(i)]; }
11    Array(const Array<T>&);
12    Array(Array<T>&&);
13    Array<T>& operator= (const Array<T>&);
14    Array<T>& operator= (Array<T>&&);
15 private:
16     int len_;
17     T* data_;
18     int check(int i) const {
19         assert(i >= 0 && i < len_);
20         return i;
21     }
22 };
```

The template in the previous example removes the need to rewrite the same class for different data-types, using a general identifier called T in the example.

Like for a normal class, templates' methods can be defined outside the class.

Instantiations of class templates need to be explicit about the parameters over which they are instantiating, for example

```
1 int main()
2 {
3     Array<int> ai;
4     Array<Array<int>> aai; // before C++11,
5                           // compiler would see a >> (right-shift)
6 }
```

The syntax for a function template is the following

```
1 template<typename T>
2 void swap(T& x, T& y)
3 {
4     T tmp = x;
```

```

5   x = y;
6   y = tmp;
7 }

```

Template functions usually do not need to be explicit about the parameters over which they are instantiating. The compiler can usually determine them automatically. Every time we used `swap()` with a given pair of types, the compiler will go to the above definition and will create yet another “template function” as an instantiation of the above. Every once in a while it can be needed to explicitly select which version of a function template to use, because the compiler cannot deduce the type at all, or it would deduce the wrong type.

For example, the function template might have not any parameter of its template argument types and calls to this function need to explicitly select the template version.

It could be useful to force the compiler to select a function template that requires certain promotions on arguments. With a template like the following

```

1  template<typename T>
2  void g(T x)
3  {
4      // ...
5  }

```

`g("xyz")` would select `g<char*>(char*)` but to call the template for strings it is possible to use `g<std::string>("xyz")`.

It might also happen that the function template has two parameters of the same type but the user calls it with two different types

```

1  template<typename T>
2  void g(T x, T y);
3  int m = 0;
4  long n = 1;
5  g<int>(m, n);

```

In this case, the compiler can not deduce itself, and an explicit selection must be done. It is possible to use [template specialization](#), in case the function template’s observable behavior is consistent for all the `T` types with the differences limited to implementation details. For example, a function template that represent an object as a string could make use of `std::boolalpha` to represent a `bool` as `false` and `true`, or `std::precision` to represent a `double`, and so on.

In case the function template has common code along with a relatively small amount of `T`-specific code,

Function templates participate in [name resolution for overloaded functions](#), the rules are different though. For a template to be considered in overload resolution, the type has to match exactly. If the types do not match exactly, the conversions are not considered and the template is simply dropped from the set of viable functions. That’s what is known as [SFINAE](#) — Substitution Failure Is Not An Error.

The reasons why ... are

- A template is not a class or a function. A template is a “pattern” that the compiler

uses to generate a family of classes or functions.

- In order for the compiler to generate the code, it must see both the template definition (not just declaration) and the specific types/whatever used to “fill in” the template. For example, if you’re trying to use a `Foo<int>`, the compiler must see both the `Foo` template and the fact that you’re trying to make a specific `Foo<int>`.
- Your compiler probably doesn’t remember the details of one `.cpp` file while it is compiling another `.cpp` file. It could, but most do not.
This is called the [separate compilation model](#).

Container Classes

A contiguous array is the optimal construct for accessing a sequence of objects in memory, in terms of space and time.

In C++ there are better alternatives to plain C arrays, easier to write to read and less prone to errors but with same speed. Two of the main issues with C-style arrays are

- C array doesn’t know its own size
- the name of a C array converts to its first element almost always

The first issue implies that there can not be array assignment. Moreover, an array size must be determined at compile-time.

The second issue makes array interacts badly with inheritance. In the following code

```
1 class Base { void fct(); /* ... */ };
2 class Derived : Base { /* ... */ };
3 void f(Base* p, int sz)
4 {
5     for (int i=0; i<sz; ++i) p[i].fct();
6 }
7 Base ab[20];
8 Derived ad[20];
9 void g()
10 {
11     f(ab, 20);
12     f(ad, 20);    // disaster!
13 }
```

the array of `Derived` is treated as an array of `Base` and the different sizes of the two break the subscripting without the compiler clearly notifying it, as it would do with vectors.

Some of the benefits that come with the container classes are the following

- it is possible to insert an element in almost any place without the need
-

Some of the things that can be done using arrays are

- `std::map` can be used as a lookup table;

- `std::map` can be used as a sparse array or matrix;
- `std::array` can be used in place of plain arrays, since it provides the user with boundary checks, memory management in case of exceptions
-
-

The storage for a `std::vector<T>` or `std::array<T, N>` is guaranteed to be **contiguous**. This means that for any non-empty `std::vector<T>` or `std::array<T, N>` named `v` and an integer `n` in the range `0 ... v.size()-1`, it is always sure that `&v[0] + n == &v[n]`.

On the other side, `v.begin()` is not guaranteed to be a `T*` therefore it is not guaranteed to have the same value as `&v[0]`. Using `v.data()` is always safe though.

Containers provided by C++ are homogeneous, in the sense that they contain elements of the same type. In order to hold elements of several different types, it is possible to use a container of pointers to a polymorphic type. Homogeneous containers are the easiest to use, give the best compile-time error messages and have no unnecessary run-time overhead.

To have a heterogeneous container, it is possible to define a common interface for all the elements and make a container of pointers:

```
1 class Io_obj { /* ... */ };    // the interface needed to take part
    in object I/O
2 vector<Io_obj*> v1;           // if you want to manage the pointers
    directly
3 vector<shared_ptr<Io_obj>> v2; // if you want a "smart pointer" to
    handle the objects
```

An alternative is to use libraries like `boost::any`.

If a pointer is used, objects in the container can be accessed through the pointers themselves, leveraging the polymorphic behavior as well. To know the exact type of the object, it is possible to use `dynamic_cast<>` or `typeid()`. To copy the container of different objects, the **virtual constructor idiom** is probably needed (see 1.4).

In order to make things easier, it is possible to rely on a `shared_ptr`.

It could be necessary to have a container of disjoint classes, that do not share a common base class. In this case, it is possible to create a handle class and make a container of handle objects. This approach comes with the benefit of encapsulating most of the memory management and object lifetime, but also with the downside of maintenance every time there are changes to the set of elements that can be contained.

[check and include](#)

Class Libraries

The **STL** is a library that consists mainly of container classes, along with iterators and algorithms to work with the content of the containers. Technically speaking it is no longer meaningful since the classes it provides have been integrated into the standard library.

Sometimes, it is described as a separate library though.

A good resource about STL is

[Apache C++ Standard Library](#)

The National-Institute-of-Health's-class-library, [NIHCL](#), is a C++ translation of the Smalltalk class library. It can be retrieved [here](#).

For what concerns Numerical Recipes, following some resources

[LAPACK: Linear Algebra Subroutine Library](#).

[Netlib](#) is a collection of mathematical software, papers, and databases.

Other information about C++ class libraries can be found in the following resources

[Available C++ Libraries FAQ](#) maintained by Nikki Locke.

[Mathtools.net](#) A Resource for the Technical Computing Community.

[boost C++ libraries](#)

Exceptions and Error Handlings

Reference and Value Semantics

[virtual](#) data allows a derived class to change the exact class of a base class's member object. It is not directly supported by C++, it can be simulated though.

To [simulate a virtual data](#),

- the base class must have a [pointer to the member object](#);
- the derived class must provide a [new object to be pointed to by the base class's pointer](#)

The base class would also have one or more constructors that provide their own referent

Inline Functions

Conceptually similar to what happens with a `#define` macro, the compiler inline-expands a function call, inserting the function's code directly into the caller's code stream.

Because of the [procedurally integration](#) the optimizer can make on the called code, this expansion might improve the performances. Considering the following code

```
1 void f()
2 {
3     int x = /*...*/;
4     int y = /*...*/;
5     int z = /*...*/;
6     // ...code that uses x, y and z...
7     g(x, y, z);
8     // ...more code that uses x, y and z...
9 }
```

in a typical C++ implementation with registers and a stack, registers and parameters get written on the stack just before the call to `g(x)`, then the parameters get read from the stack into `g(x)` and read again to restore the register after `g(x)` returns to `f(x)`.

If the compiler inline-expands the call to `g(x)`, there would not be a function call and

the not needed writes and reads that come with it.

It is not automatic that an inline function can improve the performance, since it might depend on multiple factors.

-
-
-

inline functions should be used instead of macros

<https://isocpp.org/wiki/faq/misc-technical-issues#macros-with-if>

<https://isocpp.org/wiki/faq/misc-technical-issues#macros-with-multi-stmts>

<https://isocpp.org/wiki/faq/misc-technical-issues#macros-with-token-pasting>

<https://isocpp.org/wiki/faq/big-picture#use-evil-things-sometimes>

In order to make both a non-member and function inline, the declaration stays the same but the definition must be prepended with the keyword `inline` and written in a [header file](#). In case the definition is placed in a `.cpp` file, unless the function is used only in that single file there will be an unresolved external error from the linker.

Another way to tell the compiler that a function is inline is to define the member function in the class body without the `inline` keyword in the definition. This way might be confusing for readers though, since it mixes the what (behavior) with the how (implementation). Therefore, in case of a highly reused class the definition outside the class should be preferred.

In case of an inline member function defined outside the class, the keyword must be put on the definition only, as this separation makes it easier for the user to read the header file to determine the [observable semantics](#) or [external behavior](#).

Pointers to Member Functions

Pointers to member functions are different than pointers to simple functions.

Considering the function

```
1 int f(char a, float b);
```

its type changes depending on whether it is an ordinary function or a non-static member of an imaginary Fred class.

```
1 int (*)(char, float);           // in case of an ordinary function
2                               // or a static member function
3 int (Fred::*)(char, float)     // in case of a member function
4                               // of class Fred
```

Pointers to member functions require elaborate techniques to be passed to signal handler, X event callback, system call that starts a thread/task and so on, since the interrupts are unable to provide the member function with a `this` pointer.

One possible solution is to use a static member as interrupt.

When creating pointers to member a function for a class, for example

```

1 class Fred
2 {
3 public:
4     int f(char x, float y);
5     // ...
6 };

```

it useful to rely on typedef to create a pointer FredMemFn to any member of Fred class that takes a (char, float)

```

1 typedef int (Fred::*FredMemFn)(char x, float y);

```

Consequently, it is trivial to declare a member-function pointer

```

1 int main()
2 {
3     FredMemFn p = &Fred::f;
4     // ...
5 }

```

or a function that receive a member-function pointer

```

1 void userCode(FredMemFn p)
2 { /*...*/ }

```

or a function that returns a member-function pointer

```

1 void userCode(FredMemFn p)
2 { /*...*/ }

```

C++17 provides the programmer with an `std::invoke`

```

1 void userCode(Fred& fred, FredMemFn p)
2 {
3     int ans = std::invoke(p, fred, 'x', 3.14);
4     // Would normally be: int ans = (fred.*p)('x', 3.14);
5     // ...
6 }

```

In case `std::invoke` is not accessible, a macro like the following

```

1 #define CALL_MEMBER_FN(object, ptrToMember) ((object).*(ptrToMember))
2

```

could help reducing code complexity

```

1 void userCode(Fred& fred, FredMemFn p)
2 {
3     int ans = CALL_MEMBER_FN(fred, p)('x', 3.14);
4     // Would normally be: int ans = (fred.*p)('x', 3.14);
5     // ...
6 }

```

In case the member function is constant, the typedef should look like

```

1 typedef int (Fred::*FredMemFn)(int) const;

```

[pointer-to-member access operators](#) are the last resource, in case neither the `std::invoke` nor a macro are possible options

```

1 class Fred { /*...*/ };
2 typedef int (Fred::*FredMemFn)(int i, double d);
3 void sample(Fred x, Fred& y, Fred* z, FredMemFn func)
4 {
5     x.*func(42, 3.14);
6     y.*func(42, 3.14);
7     z->*func(42, 3.14);
8 }

```

Neither a pointer to a member function nor a pointer to a function can be converted to a `void*`

A [functionoid](#) is similar to a function pointer but with more flexibility and thread safety.

Serialization and Unserialization

1.6 Value Categories

[Cppreference documentation about value categories.](#)

Due to the expressiveness of the C++ language, the terms [lvalue](#) and [rvalue](#) references evolved in a more complex set.

glvalues

[Generalized](#) lvalue is used to refer to something that could be either a [lvalue](#) or [xvalue](#).

rvalues

This term is inherited from C, in which [rvalues](#) can be on the [right](#) side of an assignment. In C++ this term can refer to things that are either [xvalues](#) and [prvalues](#).

lvalues

This term is also inherited from C, in which [lvalues](#) can be on the [left](#) side of an assignment. Therefore, the simplest one is the name of a variable in a declaration.

Any expression resulting in an [lvalue reference](#) is also an [lvalue](#).

```

1 int v;           // v is an lvalue of type int
2 A* p1;           // *p1 is an lvalue
3 A& r1 = v1;      // r1 is an lvalue
4 A& f();           // f() is an lvalue

```

Accessing members of objects through pointers or references yields [lvalues](#). For example, with the following code

```

1 struct Foo
2 {
3     Bar a;
4     Bar b;
5 };
6
7 Foo& f2();
8 Foo* p2;
9 Foo v2;

```


`f2().a`, `p2->b` and `v2.a` are all *lvalues* if type `A`.

String literals are *lvalues* of type array.

To conclude, a **named object** declared with an *rvalue reference* is an *lvalue*. The name *rvalue reference* indicates that it can bind to an *rvalue*, and once you've declared a variable and given it a name it is an *lvalue*.

```
1 void Foo(A&& bar)
2 {
3     // within Foo, bar is an lvalue
4     // which will only bind to rvalues
5 }
```

xvalues

This term indicates **eXpiring** value: an unnamed object that will be destroyed soon.

xvalues can be treated either as *glvalues* or *rvalues*, based on the context.

Usually, *xvalues* only arise through *explicit casts* and *function calls*. If an expression is **cast to an *rvalue* reference to some type `T`** then the result is an *xvalue of type `T`*

```
1 static_cast<A&&>(v1) // yields an xvalue of type A
```

If the return type of a function is an *rvalue reference* to some type `T`, the result is an *xvalue* of type `T`. For example, the `std::move` is declared as

```
1 template <typename T>
2 constexpr remove_reference_t<T>&& move(T&& t) noexcept;
```

`std::move(v1)` is an **xvalue** of type `A`. The type deduction rules deduce `T` to be `A&` because `v1` is an *lvalue*.

prvalues

Pure *rvalue* is an *rvalue* which is not an *xvalue*

Literals other than string literals are *prvalues*. For example, `42` is a *prvalue* of type `int` and `3.141f` is a *prvalue* of type `float`

Temporaries are *prvalues*. For example any temporaries created as a result of an implicit conversion is also a *prvalue*. In the following code

```
1 int consume_string(std::string&& s);
2 int i = consume_string("hello");
```

the string literal in the function call will implicitly convert to a temporary of type `std::string`, which can **bind to the *rvalue* reference** used for the function parameter since the temporary is a *prvalue*

1.7 Reference Binding

[cppreference documentation](#) about reference declaration.

The **main difference** between *lvalues* and *rvalues* is the way they bind to references. The differences in type deduction rules can have a big impact too.

1.8 lvalue references

These references are declared with a single ampersand &.

A **non-const lvalue reference will only bind to non-const lvalues** of the same type, or a class derived from the referenced type

```
1 class C : public A{};
2
3 int i = 42;
4 A a;
5 B b;
6 C c;
7 const A ca{};
8
9 A& r1 = a;
10 A& r2 = c;
11 A& r3 = b;      // error, wrong type
12 int& r4 = i;
13 int& r5 = 42;    // error, cannot bind rvalue
14 A& r6 = ca;      // error, cannot bind const object
15                  // to non const ref
16 A& r7 = r1;
17 A& r8 = A();     // error, cannot bind rvalue
18 A& r9 = B().a;   // error, cannot bind rvalue
19 A& r10 = C();    // error, cannot bind rvalue
```

A **const lvalue reference can also bind to rvalues**. The object bound to the reference must have the same type as the referenced type or a class derived from the referenced type. It is possible to bind both const and non-const values to a const lvalue reference.

```
1 const A& cr1 = a;
2 const A& cr2 = c;
3 const A& cr3 = b;      // error, wrong type
4 const int& cr4 = i;
5 const int& cr5 = 42;    // rvalue can bind OK
6 const A& cr6 = ca;      // OK, can bind const object to const ref
7 const A& cr7 = cr1;
8 const A& cr8 = A();     // OK, can bind rvalue
9 const A& cr9 = B().a;   // OK, can bind rvalue
10 const A& cr10 = C();    // OK, can bind rvalue
11 const A& cr11 = r1;
```

Binding a temporary object (i.e. a **prvalue**) to a const lvalue reference **extends the lifetime** of that temporary to the lifetime of the reference. In the previous example, it is possible to use r8, r9 and r10 later in the code.

Lifetime extension does not extend to references initialized from the first reference. If a function parameter is a const lvalue reference and gets bound to a temporary passed to the function call, the temporary is destroyed when the function returns. This happens also if the reference was stored in a longer-lived variable, such as a member of a newly constructed object.

volatile and **const volatile** lvalue references are much less used but behave as ex-

pected: volatile T& bind to a volatile or non-volatile, non-const lvalue of type T or a class derived from T. However, volatile const lvalue references do not bind to rvalues.

2 C++11 New Features - legacy section to remove

2.1 Reference Collapsing

In C++ it is possible to form reference to reference in `templates` and `typedef` expressions. In this context, the reference collapsing rule apply: rvalue reference to rvalue reference collapses to an rvalue reference, all the others collapse to lvalue references.

```
1 typedef int& lref;
2 typedef int&& rref;
3 int n;
4
5 lref& r1 = n; // type of r1 is int&
6 lref&& r2 = n; // type of r2 is int&&
7 rref& r3 = n; // type of r3 is int&
8 rref&& r4 = 1; // type of r4 is int&&
```

Implicit conversion

A reference-to-A can be initialized with any D object which has an implicit conversion operator that returns A&. In this case the reference is bound to the result of the conversion operator.

A const lvalue-reference-to-E will bind only to objects implicitly convertible to E. The reference is bound to the temporary resulting from the conversion.

General properties

In general, `rvalues` can't be modified and their address can't be taken.

However, as long as class X has a member function named `do_something`, the expression `X().do_something` is valid.

`ref qualifiers` along with `const` can be used to tag different overloads of class member functions, to indicate they can be applied to only *lvalues* or to only *rvalues*.

When the type of a variable or function template parameter is deduced from its initializer, whether the initializer is an lvalue or rvalue can affect the deduced type.

3 Contemporary C++: new language and library features

3.1 C++11 new language features

Following a summary of the new language features introduced with C++11.

move semantics

Move semantics is a new way of moving resources around in an optimal way, since it helps *avoiding unnecessary copies* of temporary objects. It is based on rvalue references and plays an *important role in exception safety* for C++.

Moving an object means to transfer ownership of some resource it manages to another object.

The first benefit of move semantics is performance optimization. When an object is about to reach the end of its lifetime, either because it's a temporary or by explicitly calling `std::move`, a move is often a cheaper way to transfer resources. For example, moving a `std::vector` is just copying some pointers and internal state over to the new vector – copying would involve having to copy every single contained element in the vector, which is expensive and unnecessary if the old vector will soon be destroyed.

Moves also make it possible for non-copyable types such as `std::unique_ptr`s (smart pointers) to guarantee at the language level that there is only ever one instance of a resource being managed at a time, while being able to transfer an instance between scopes.

variadic templates

The `...` syntax creates a parameter pack or expands one.

A template parameter pack is a template parameter that accepts zero or more template arguments (non-types, types, or templates).

A template with at least one parameter pack is called a variadic template.

```
1 template <typename... T>
2 struct arity {
3     constexpr static int value = sizeof...(T);
4 };
5 static_assert(arity<>::value == 0);
6 static_assert(arity<char, short, int>::value == 3);
```

An interesting use for this is creating an initializer list from a parameter pack in order to iterate over variadic function arguments.

```
1 template <typename First, typename... Args>
2 auto sum(const First first, const Args... args) -> decltype(first) {
3     const auto values = {first, args...};
4     return std::accumulate(values.begin(), values.end(), First{0});
5 }
6
7 sum(1, 2, 3, 4, 5); // 15
8 sum(1, 2, 3);      // 6
```

```
9 sum(1.5, 2.0, 3.7); // 7.2
```

rvalue references

An rvalue reference to T, which is a non-template type parameter (such as int, or a user-defined type), is created with the syntax T&&. Rvalue references only bind to rvalues. An [rvalue reference](#) can be used to extend the lifetime of temporary objects, making the referenced object modifiable through them. The reference must be const in order to bind to a const object, though const rvalue reference are much rarer.

```
1 const A make_const_A();
2 A&& rr1 = a; // error, cannot bind lvalue
3 // to rvalue reference
4 A&& rr2 = A();
5 A&& rr3 = B(); // error, wrong type
6 int&& rr4 = i; // error, cannot bind lvalue
7 int&& rr5 = 42;
8 A&& rr6 = make_const_A(); // error, cannot bind
9 // const object
10 // to non const ref
11 const A&& rr7 = A();
12 const A&& rr8 = make_const_A();
13 A&& rr9 = B().a;
14 A&& rr10 = C();
15 A&& rr11 = std::move(a); // std::move returns an rvalue
16 A&& rr12 = rr11; // error rvalue references
17 // are lvalues
```

rvalue references extend the lifetime of temporary objects in the same way const lvalue references do, therefore temporaries associated with rr2, rr5, rr7, rr8, rr9 and rr10 remain valid until the corresponding references are destroyed.

forwarding references

Cppreference documentation about [forward utility](#).

Cppreference about [forwarding references](#).

Perfect forwarding is an important C++0x feature built on top of rvalue references. It provides the programmer with the capability automatically apply the move semantics, even when there are intervening function calls to separate the source and the destination of a move.

Examples include [constructors](#) and [setter functions](#), that can forward arguments they receive directly to the data member they are initializing or setting. Standard Library functions like [make_shared](#) "perfect-forwards" its arguments to the class constructor of whatever object the to-be-created shared_ptr is to point to.

The idiomatic expression of the perfect forwarding make use of the std::forward

```
1 template <typename T>
2 void wrapper(T&& arg)
3 {
```

```

4     wrapped(std::forward<T1>(t1), std::forward<T2>(t2));
5 }

```

initializer lists

Cppreference documentation about Initializer list

An object of type `std::initializer_list<T>` is a lightweight proxy object that provides access to an array of objects of type `const T`.

Uniform initialization syntax and semantics

C++98 provides the users with several ways of initializing an object. These ways depend on the type and the initialization context and have different ways of working.

Some examples can be the following code lines

```

1 string a[] = { "foo", " bar" };           // ok: initialize array
2 vector<string> v = { "foo", " bar" };     // error: initializer list
3                                           // for non-aggregate vector
4 void f(string a[]);
5 f( { "foo", " bar" } );                  // syntax error:
6                                           // block as argument

```

and

```

1 int a = 2;                               // "assignment style"
2 int aa[] = { 2, 3 };                     // assignment style with list
3 complex z(1,2);                           // "functional style" initialization
4 x = Ptr(y);                               // "functional style" for
5                                           // conversion/cast/construction
6
7 int a(1);                                 // variable definition
8 int b();                                  // function declaration
9 int b(foo);                               // variable definition or function declaration

```

C++11 introduces a `{}-initializer`, which provides a much simpler and consistent way to perform initialization, for all different initialization context as in the following example

```

1 X x1 = X{1,2};
2 X x2 = {1,2};    // the = is optional
3 X x3{1,2};
4 X* p = new X{1,2};
5 struct D : X
6 {
7     D(int x, int y) :X{x,y} { /* ... */ };
8 };
9 struct S
10 {
11     int a[3];
12     S(int x, int y, int z) :a{x,y,z} { /* ... */ }; // solution to
13                                                         // old problem
14 };

```

`{}-initializer` produces the same result in any context

```

1 X x{a};
2 X* p = new X{a};
3 z = X{a};           // use as cast
4 f({a});             // function argument (of type X)
5 return {a};         // function return value (function returning X)

```

`{}-initializer` is also useful to solve the `most vexing parse` problem, explained in 1.2.

static assertions

Assertions that are evaluated at compile-time.

```

1 constexpr int x = 0;
2 constexpr int y = 1;
3 static_assert(x == y, "x != y");

```

auto

Cppreference documentation about `auto`.

References in bibliography: [17], [18], [19].

This placeholder type specifier has the following meanings:

- for **variables**:
the variable that is being declared will be automatically deduced from its initializer;
- for **functions**:
the return type will be deduced from its return statements;
- for **non-type template parameters**:
the type will be deduced from the argument.

Following a summary of `contextes in which auto may appear`.

In the summary, the `rules for template argument deduction` are referenced.

- if used **in the type specifier of a variable** the variable type is deduced from the initializer list using the template argument deduction:

```

1 auto x = expr;
2

```

In case `no pointer or reference` is attached to `auto`, both `const` and references are ignored:

```

1 int y = 10;
2 int& r = y;
3 auto x = r;           // type is int (reference ignored)
4
5 const int y = 10;
6 auto x = y;           // type is int (const ignored)
7
8 int y = 10;

```

```

9  const int& r = y;
10 auto x = r;                // type is int
11                               // (const and reference ignored)
12
13 const int a[10] = {};
14 auto x = a;                // type is int*
15                               // (array to pointer conversion)
16

```

Modifiers which may accompany auto will participate in the type deduction. For example, in the code

```

1  const auto& i = expr;
2  template<class U> void f(constU& u);
3

```

the type deduced for the first code line is the same deduced for the argument u of the second line if the function call f(expr) was compiled.

Therefore **auto&&** may be deduced both as an *lvalue* reference and an *rvalue* reference.

In case **auto** is used to declare multiple variables, the deduced type must match.

- If used **in a function defined with the trailing return type syntax**, it is just part of the syntax and automatic type deduction is not performed

```

1  auto function_name(parameter_list) -> return_type
2  {
3      // Function implementation
4  }
5

```

- If used **as return type of a function defined without the trailing return type**, or **as a return type of a lambda expression**, the keyword auto indicates that the return type will be deduced from the return statement, using the rules for template argument deduction.
- If used **in combination with decltype** and the declared type of the variable is decltype(auto), the keyword auto is replaced with the expression (or expression list) of its initializer and the actual type is deduced using the rules for decltype.
- If used **along with decltype in the return type of a function** and the return type of the function is decltype(auto), the keyword auto is replaced with the operand of its return statement, and the actual return type is deduced using the rules of decltype.
- If used in the **type-id of a new expression**, the type is deduced from the initializer.
- If used **in the parameter declaration of a non-type template parameter**, for example `template<auto I> struct A;`, its type is deduced from the corresponding argument.

- if used **as a nested scope like in `auto::`** ???
- if used **in the parameter declaration of a lambda-expression**, such lambda expression is a *generic lambda*. This means that the following generic lambda will be written by the compiler as an instance of the templates which follows

```

1 auto g_lambda = [] (auto a) { return a; } ;
2 class /* unnamed */
3 {
4 public:
5     template<typename T>
6     T operator () (T a) const { return a; }
7 };
8

```

- if used **in a function parameter declaration**, like `void f(auto);`, the function declaration introduces an abbreviated function template [29]

auto-typed variables are deduced by the compiler according to the type of their initializer.

```

1 auto a = 3.14; // double
2 auto b = 1; // int
3 auto& c = b; // int&
4 auto d = { 0 }; // std::initializer_list<int>
5 auto&& e = 1; // int&&
6 auto&& f = b; // int&
7 auto g = new auto(123); // int*
8 const auto h = 1; // const int
9 auto i = 1, j = 2, k = 3; // int, int, int
10 auto l = 1, m = true, n = 1.61; // error -- 'l' deduced to be int, '
    m' is bool
11 auto o; // error -- 'o' requires initializer

```

Extremely useful for readability, especially for complicated types:

```

1 std::vector<int> v = ...;
2 std::vector<int>::const_iterator cit = v.cbegin();
3 // vs.
4 auto cit = v.cbegin();

```

Functions can also deduce the return type using auto. In C++11, a return type must be specified either explicitly, or using `decltype` like so:

```

1 template <typename X, typename Y>
2 auto add(X x, Y y) -> decltype(x + y) {
3     return x + y;
4 }
5 add(1, 2); // == 3
6 add(1, 2.0); // == 3.0
7 add(1.5, 1.5); // == 3.0

```

The trailing return type in the above example is the declared type (see section on `decltype`) of the expression `x + y`. For example, if `x` is an integer and `y` is a double, `decltype(x +`

y) is a double. Therefore, the above function will deduce the type depending on what type the expression $x + y$ yields. Notice that the trailing return type has access to its parameters, and this when appropriate.

lambda expressions

Cppreference about lambda expressions

A [lambda expression](#) is a closure, i.e. an unnamed

captures	a comma-separated list of zero or more captures, optionally beginning with a capture-default
	A lambda expression can use a variable without capturing it if the variable • is a non-local variable or has static or thread local storage duration (in which case the variable cannot be captured), or • is a reference that has been initialized with a constant expression. A lambda expression can read the value of a variable without capturing it if the variable • has const non-volatile integral or enumeration type and has been initialized with a constant expression, or • is constexpr and has no mutable members.
tparams	a non-empty comma-separated list of template parameters used to provide names to the template parameters of a generic lambda
t-requires	adds constraints to tparams
front-attr	since C++23 study
params	the parameter list of operator() of the closure type
specs	A list of the following specifiers, each specifier is allowed at most once in each sequence. If not provided, the objects captured by copy are const • mutable allows body to modify the objects captured by copy • constexpr since C++17 • constexpr since C++20 • static since C++23
exception	provides the dynamic exception specification or (until C++20) the noexcept specifier for operator() of the closure type
back-attr	an attribute specifier sequence applies to the type of operator() of the closure type (and thus the <code>[[noreturn]]</code> attribute cannot be used)
trailing-type	$\rightarrow$ ret where ret specifies the return type
requires	adds constraints to operator() of the closure type
body	function body

decltype

`decltype` inspects the declared type of an entity or the type and value category of an expression.

Explain difference between parenthesized

See [30]

type aliases

Semantically similar to using a typedef however, type aliases with using are easier to read and are compatible with templates.

```
1 template <typename T>
2 using Vec = std::vector<T>;
3 Vec<int> v; // std::vector<int>
4
5 using String = std::string;
6 String s {"foo"};
```

nullptr

Cppreference documentation about nullptr

The keyword `nullptr` denotes the pointer literal. It is a `prvalue` of type `std::nullptr_t`. There exist implicit conversions from `nullptr` to null pointer value of any pointer type and any pointer to member type. Similar conversions exist for any null pointer constant, which includes values of type `std::nullptr_t` as well as the macro `NULL`.

strongly-typed enums

Type-safe enums that solve a variety of problems with C-style enums including: implicit conversions, inability to specify the underlying type, scope pollution.

```
1 // Specifying underlying type as 'unsigned int'
2 enum class Color : unsigned int { Red = 0xff0000, Green = 0xff00,
   Blue = 0xff };
3 // 'Red'/'Green' in 'Alert' don't conflict with 'Color'
4 enum class Alert : bool { Red, Green };
5 Color c = Color::Red;
```

attributes

Attributes provide a universal syntax over `__attribute__((...))`, `__declspec`, etc.

```
1 // 'noreturn' attribute indicates 'f' doesn't return.
2 [[ noreturn ]] void f() {
3     throw "error";
4 }
```

constexpr

Cppreference about constexpr

The constexpr specifier declares that it is possible to evaluate the value of the function or variable at compile time. Such variables and functions can then be used where only compile time constant expressions are allowed (provided that appropriate function arguments are given).

delegating constructors

Constructors can now call other constructors in the same class using an initializer list.

```
1 struct Foo {
2     int foo;
3     Foo(int foo) : foo{foo} {}
4     Foo() : Foo(0) {}
5 };
6
7 Foo foo;
8 foo.foo; // == 0
```

user-defined literals

User-defined literals allow you to extend the language and add your own syntax. To create a literal, define a `T operator "" X(...)` function that returns a type `T`, with a name `X`. Note that the name of this function defines the name of the literal. Any literal names not starting with an underscore are reserved and won't be invoked. There are rules on what parameters a user-defined literal function should accept, according to what type the literal is called on.

Converting Celsius to Fahrenheit

```
1 // 'unsigned long long' parameter required for integer literal.
2 long long operator "" _celsius(unsigned long long tempCelsius) {
3     return std::llround(tempCelsius * 1.8 + 32);
4 }
5 24_celsius; // == 75
```

String to integer conversion

```
1 // 'const char*' and 'std::size_t' required as parameters.
2 int operator "" _int(const char* str, std::size_t) {
3     return std::stoi(str);
4 }
5
6 "123"_int; // == 123, with type 'int'
```

explicit virtual overrides

Specifies that a virtual function overrides another virtual function. If the virtual function does not override a parent's virtual function, throws a compiler error.

```

1 struct A {
2     virtual void foo();
3     void bar();
4 };
5
6 struct B : A {
7     void foo() override; // correct -- B::foo overrides A::foo
8     void bar() override; // error -- A::bar is not virtual
9     void baz() override; // error -- B::baz does not override A::baz
10 };

```

final specifier

Specifies that a virtual function cannot be overridden in a derived class or that a class cannot be inherited from.

```

1 struct A {
2     virtual void foo();
3 };
4
5 struct B : A {
6     virtual void foo() final;
7 };
8
9 struct C : B {
10     virtual void foo(); // error -- declaration of 'foo' overrides a '
    final' function
11 };

```

Class cannot be inherited from.

```

1 struct A final {};
2 struct B : A {}; // error -- base 'A' is marked 'final'

```

default functions

A more elegant, efficient way to provide a default implementation of a function, such as a constructor.

```

1 struct A {
2     A() = default;
3     A(int x) : x{x} {}
4     int x {1};
5 };
6 A a; // a.x == 1
7 A a2 {123}; // a.x == 123

```

With inheritance:

```

1 struct B {
2     B() : x{1} {}
3     int x;
4 };

```

```

5
6 struct C : B {
7     // Calls B::B
8     C() = default;
9 };
10
11 C c; // c.x == 1

```

deleted functions

A more elegant, efficient way to provide a deleted implementation of a function. Useful for preventing copies on objects.

```

1 class A {
2     int x;
3
4 public:
5     A(int x) : x{x} {};
6     A(const A&) = delete;
7     A& operator=(const A&) = delete;
8 };
9
10 A x {123};
11 A y = x; // error -- call to deleted copy constructor
12 y = x; // error -- operator= deleted

```

range-based for loops

A [range-for statement](#) applies to all standard containers and string as well, providing a less-typing way to for-loop over a *range* of elements.

An example of usage in the following code

```

1 std::array<int, 5> a {1, 2, 3, 4, 5};
2 for (int& x : a) x *= 2;
3 // a == { 2, 4, 6, 8, 10 }
4
5 std::array<int, 5> a {1, 2, 3, 4, 5};
6 for (int x : a) x *= 2;
7 // a == { 1, 2, 3, 4, 5 }

```

In the previous loops, note the difference when using `int` and `int&`.

special member functions for move semantics

The copy constructor and copy assignment operator are called when copies are made, and with C++11's introduction of move semantics, there is now a move constructor and move assignment operator for moves.

```

1 struct A {
2     std::string s;
3     A() : s{"test"} {}
4     A(const A& o) : s{o.s} {}

```

```

5   A(A&& o) : s{std::move(o.s)} {}
6   A& operator=(A&& o) {
7       s = std::move(o.s);
8       return *this;
9   }
10 };
11
12 A f(A a) {
13     return a;
14 }
15
16 A a1 = f(A{}); // move-constructed from rvalue temporary
17 A a2 = std::move(a1); // move-constructed using std::move
18 A a3 = A{};
19 a2 = std::move(a3); // move-assignment using std::move
20 a1 = f(A{}); // move-assignment from rvalue temporary

```

converting constructors

Converting constructors will convert values of braced list syntax into constructor arguments.

```

1 struct A {
2     A(int) {}
3     A(int, int) {}
4     A(int, int, int) {}
5 };
6
7 A a {0, 0}; // calls A::A(int, int)
8 A b(0, 0); // calls A::A(int, int)
9 A c = {0, 0}; // calls A::A(int, int)
10 A d {0, 0, 0}; // calls A::A(int, int, int)

```

Note that the braced list syntax does not allow narrowing:

```

1 struct A {
2     A(int) {}
3 };
4
5 A a(1.1); // OK
6 A b {1.1}; // Error narrowing conversion from double to int

```

Note that if a constructor accepts a `std::initializer_list`, it will be called instead:

```

1 struct A {
2     A(int) {}
3     A(int, int) {}
4     A(int, int, int) {}
5     A(std::initializer_list<int>) {}
6 };
7
8 A a {0, 0}; // calls A::A(std::initializer_list<int>)
9 A b(0, 0); // calls A::A(int, int)
10 A c = {0, 0}; // calls A::A(std::initializer_list<int>)

```

```
11 A d {0, 0, 0}; // calls A::A(std::initializer_list<int>)
```

explicit conversion functions

Conversion functions can now be made explicit using the explicit specifier.

```
1 struct A {
2     operator bool() const { return true; }
3 };
4
5 struct B {
6     explicit operator bool() const { return true; }
7 };
8
9 A a;
10 if (a); // OK calls A::operator bool()
11 bool ba = a; // OK copy-initialization selects A::operator bool()
12
13 B b;
14 if (b); // OK calls B::operator bool()
15 bool bb = b; // error copy-initialization does not consider B::
               operator bool()
```

inline-namespaces

All members of an inline namespace are treated as if they were part of its parent namespace, allowing specialization of functions and easing the process of versioning. This is a transitive property, if A contains B, which in turn contains C and both B and C are inline namespaces, C's members can be used as if they were on A.

```
1 namespace Program {
2     namespace Version1 {
3         int getVersion() { return 1; }
4         bool isFirstVersion() { return true; }
5     }
6     inline namespace Version2 {
7         int getVersion() { return 2; }
8     }
9 }
10
11 int version {Program::getVersion()}; // Uses getVersion
    () from Version2
12 int oldVersion {Program::Version1::getVersion()}; // Uses getVersion
    () from Version1
13 bool firstVersion {Program::isFirstVersion()}; // Does not
    compile when Version2 is added
```

non-static data member initializers

Allows non-static data members to be initialized where they are declared, potentially cleaning up constructors of default initializations.


```

1 // Default initialization prior to C++11
2 class Human {
3     Human() : age{0} {}
4     private:
5         unsigned age;
6 };
7 // Default initialization on C++11
8 class Human {
9     private:
10         unsigned age {0};
11 };

```

right angle brackets

C++11 is now able to infer when a series of right angle brackets is used as an operator or as a closing statement of typedef, without having to add whitespace.

```

1 typedef std::map<int, std::map <int, std::map <int, int> > >
   cpp98LongTypedef;
2 typedef std::map<int, std::map <int, std::map <int, int>>>
   cpp11LongTypedef;

```

ref-qualified member functions

Member functions can now be qualified depending on whether *this is an lvalue or rvalue reference.

```

1 struct Bar {
2     // ...
3 };
4
5 struct Foo {
6     Bar getBar() & { return bar; }
7     Bar getBar() const& { return bar; }
8     Bar getBar() && { return std::move(bar); }
9 private:
10     Bar bar;
11 };
12
13 Foo foo{};
14 Bar bar = foo.getBar(); // calls 'Bar getBar() &'
15
16 const Foo foo2{};
17 Bar bar2 = foo2.getBar(); // calls 'Bar Foo::getBar() const&'
18
19 Foo{}.getBar(); // calls 'Bar Foo::getBar() &&'
20 std::move(foo).getBar(); // calls 'Bar Foo::getBar() &&'
21
22 std::move(foo2).getBar(); // calls 'Bar Foo::getBar() const&&'

```

trailing return types

C++11 allows functions and lambdas an alternative syntax for specifying their return types.

```
1 int f() {
2     return 123;
3 }
4 // vs.
5 auto f() -> int {
6     return 123;
7 }
8
9 auto g = []() -> int {
10     return 123;
11 };
```

This feature is especially useful when certain return types cannot be resolved:

```
1 // NOTE: This does not compile!
2 template <typename T, typename U>
3 decltype(a + b) add(T a, U b) {
4     return a + b;
5 }
6
7 // Trailing return types allows this:
8 template <typename T, typename U>
9 auto add(T a, U b) -> decltype(a + b) {
10     return a + b;
11 }
```

In C++14, `decltype(auto)` (C++14) can be used instead.

noexcept specifier

[C++ preference about noexcept](#) The `noexcept` specifier specifies whether a function could throw exceptions. It is an improved version of `throw()`.

```
1 void func1() noexcept;           // does not throw
2 void func2() noexcept(true);    // does not throw
3 void func3() throw();           // does not throw
4
5 void func4() noexcept(false);   // may throw
```

Non-throwing functions are permitted to call potentially-throwing functions. Whenever an exception is thrown and the search for a handler encounters the outermost block of a non-throwing function, the function `std::terminate` is called.

```
1 extern void f(); // potentially-throwing
2 void g() noexcept {
3     f();          // valid, even if f throws
4     throw 42;     // valid, effectively a call to std::terminate
5 }
```

char32_t and char16_t

Provides standard types for representing UTF-8 strings.

```
1 char32_t utf8_str[] = U"\u0123";  
2 char16_t utf8_str[] = u"\u0123";
```

raw string literals

C++11 introduces a new way to declare string literals as "raw string literals". Characters issued from an escape sequence (tabs, line feeds, single backslashes, etc.) can be inputted raw while preserving formatting. This is useful, for example, to write literary text, which might contain a lot of quotes or special formatting. This can make your string literals easier to read and maintain.

A raw string literal is declared using the following syntax:

```
1 R"delimiter(raw_characters)delimiter"
```

where:

- delimiter is an optional sequence of characters made of any source character except parentheses, backslashes and spaces.
- raw_characters is any raw character sequence; must not contain the closing sequence ")delimiter".

Example:

```
1 // msg1 and msg2 are equivalent.  
2 const char* msg1 = "\nHello,\n\tworld!\n";  
3 const char* msg2 = R"  
4 Hello,  
5     world!  
6>";
```

3.2 C++11 new library features

std::move

std::move indicates that the object passed to it may have its resources transferred. Using objects that have been moved from should be used with care, as they can be left in an unspecified state (see: <http://stackoverflow.com/questions/7027523/what-can-i-do-with-a-moved-from-object>).

A definition of std::move (performing a move is nothing more than casting to an rvalue reference):

```
1 template <typename T>
2 typename remove_reference<T>::type&& move(T&& arg) {
3     return static_cast<typename remove_reference<T>::type&&>(arg);
4 }
```

Transferring std::unique_ptrs:

```
1 std::unique_ptr<int> p1 {new int{0}}; // in practice, use std::
    make_unique
2 std::unique_ptr<int> p2 = p1; // error -- cannot copy unique
    pointers
3 std::unique_ptr<int> p3 = std::move(p1); // move 'p1' into 'p3'
    // now unsafe to
4     dereference object held by 'p1'
```

std::forward

Returns the arguments passed to it while maintaining their value category and cv-qualifiers.

Useful for generic code and factories. Used in conjunction with forwarding references.

A definition of std::forward:

```
1 template <typename T>
2 T&& forward(typename remove_reference<T>::type& arg) {
3     return static_cast<T&&>(arg);
4 }
```

An example of a function wrapper which just forwards other A objects to a new A object's copy or move constructor:

```
1 struct A {
2     A() = default;
3     A(const A& o) { std::cout << "copied" << std::endl; }
4     A(A&& o) { std::cout << "moved" << std::endl; }
5 };
6
7 template <typename T>
8 A wrapper(T&& arg) {
9     return A{std::forward<T>(arg)};
10 }
11
12 wrapper(A{}); // moved
13 A a;
```

```

14 wrapper(a); // copied
15 wrapper(std::move(a)); // moved

```

See also: forwarding references, rvalue references.

std::thread

The `std::thread` library provides a standard way to control threads, such as spawning and killing them. In the example below, multiple threads are spawned to do different calculations and then the program waits for all of them to finish.

```

1 void foo(bool clause) { /* do something... */ }
2
3 std::vector<std::thread> threadsVector;
4 threadsVector.emplace_back([]() {
5     // Lambda function that will be invoked
6 });
7 threadsVector.emplace_back(foo, true); // thread will run foo(true)
8 for (auto& thread : threadsVector) {
9     thread.join(); // Wait for threads to finish
10 }

```

std::to_string

Converts a numeric argument to a `std::string`.

```

1 std::to_string(1.2); // == "1.2"
2 std::to_string(123); // == "123"

```

type traits

Type traits defines a compile-time template-based interface to query or modify the properties of types.

```

1 static_assert(std::is_integral<int>::value);
2 static_assert(std::is_same<int, int>::value);
3 static_assert(std::is_same<std::conditional<true, int, double>::type,
4     int>::value);

```

smart pointers

C++11 introduces new smart pointers: `std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`. `std::auto_ptr` now becomes deprecated and then eventually removed in C++17.

`std::unique_ptr` is a non-copyable, movable pointer that manages its own heap-allocated memory. Note: Prefer using the `std::make_X` helper functions as opposed to using constructors. See the sections for `std::make_unique` and `std::make_shared`.

```

1 std::unique_ptr<Foo> p1 { new Foo{} }; // 'p1' owns 'Foo'
2 if (p1) {
3     p1->bar();
4 }

```

```

5
6 {
7     std::unique_ptr<Foo> p2 {std::move(p1)};    // Now 'p2' owns 'Foo'
8     f(*p2);
9
10    p1 = std::move(p2);    // Ownership returns to 'p1' -- 'p2' gets
                            destroyed
11 }
12
13 if (p1) {
14     p1->bar();
15 }
16 // 'Foo' instance is destroyed when 'p1' goes out of scope

```

A `std::shared_ptr` is a smart pointer that manages a resource that is shared across multiple owners. A shared pointer holds a control block which has a few components such as the managed object and a reference counter. All control block access is thread-safe, however, manipulating the managed object itself is not thread-safe.

```

1 void foo(std::shared_ptr<T> t) {
2     // Do something with 't'...
3 }
4
5 void bar(std::shared_ptr<T> t) {
6     // Do something with 't'...
7 }
8
9 void baz(std::shared_ptr<T> t) {
10    // Do something with 't'...
11 }
12
13 std::shared_ptr<T> p1 {new T{}};
14 // Perhaps these take place in another threads?
15 foo(p1);
16 bar(p1);
17 baz(p1);

```

std::chrono

The `chrono` library contains a set of utility functions and types that deal with durations, clocks, and time points. One use case of this library is benchmarking code:

```

1 std::chrono::time_point<std::chrono::steady_clock> start, end;
2 start = std::chrono::steady_clock::now();
3 // Some computations...
4 end = std::chrono::steady_clock::now();
5
6 std::chrono::duration<double> elapsed_seconds = end - start;
7 double t = elapsed_seconds.count(); // t number of seconds,
    represented as a 'double'

```

tuples

Tuples are a fixed-size collection of heterogeneous values. Access the elements of a `std::tuple` by unpacking using `std::tie`, or using `std::get`.

```
1 // 'playerProfile' has type 'std::tuple<int, const char*, const char
  *>'.
2 auto playerProfile = std::make_tuple(51, "Frans Nielsen", "NYI");
3 std::get<0>(playerProfile); // 51
4 std::get<1>(playerProfile); // "Frans Nielsen"
5 std::get<2>(playerProfile); // "NYI"
```

std::tie

Creates a tuple of lvalue references. Useful for unpacking `std::pair` and `std::tuple` objects. Use `std::ignore` as a placeholder for ignored values. In C++17, structured bindings should be used instead.

```
1 // With tuples...
2 std::string playerName;
3 std::tie(std::ignore, playerName, std::ignore) = std::make_tuple(91,
  "John Tavares", "NYI");
4
5 // With pairs...
6 std::string yes, no;
7 std::tie(yes, no) = std::make_pair("yes", "no");
```

std::array

`std::array` is a container built on top of a C-style array. Supports common container operations such as sorting.

```
1 std::array<int, 3> a = {2, 1, 3};
2 std::sort(a.begin(), a.end()); // a == { 1, 2, 3 }
3 for (int& x : a) x *= 2; // a == { 2, 4, 6 }
```

unordered containers

These containers maintain average constant-time complexity for search, insert, and remove operations. In order to achieve constant-time complexity, sacrifices order for speed by hashing elements into buckets. There are four unordered containers:

- `unordered_set`
- `unordered_multiset`
- `unordered_map`
- `unordered_multimap`

std::make_shared

std::make_shared is the recommended way to create instances of std::shared_ptr due to the following reasons:

- Avoid having to use the new operator.
- Prevents code repetition when specifying the underlying type the pointer shall hold.
- It provides exception-safety. Suppose we were calling a function foo like so:

```
1 foo(std::shared_ptr<T>{new T{}}, function_that_throws(), std::  
    shared_ptr<T>{new T{}});
```

The compiler is free to call new T{}, then function_that_throws(), and so on... Since we have allocated data on the heap in the first construction of a T, we have introduced a leak here. With std::make_shared, we are given exception-safety:

```
1 foo(std::make_shared<T>(), function_that_throws(), std::  
    make_shared<T>());
```

- Prevents having to do two allocations. When calling std::shared_ptr{ new T{} }, we have to allocate memory for T, then in the shared pointer we have to allocate memory for the control block within the pointer.

See the section on smart pointers for more information on std::unique_ptr and std::shared_ptr.

std::ref

std::ref(val) is used to create object of type std::reference_wrapper that holds reference of val. Used in cases when usual reference passing using & does not compile or & is dropped due to type deduction. std::cref is similar but created reference wrapper holds a const reference to val.

```
1 // create a container to store reference of objects.  
2 auto val = 99;  
3 auto _ref = std::ref(val);  
4 _ref++;  
5 auto _cref = std::cref(val);  
6 //_cref++; does not compile  
7 std::vector<std::reference_wrapper<int>>vec; // vector<int>&vec does  
    not compile  
8 vec.push_back(_ref); // vec.push_back(&i) does not compile  
9 cout << val << endl; // prints 100  
10 cout << vec[0] << endl; // prints 100  
11 cout << _cref; // prints 100
```


memory model

C++11 introduces a memory model for C++, which means library support for threading and atomic operations. Some of these operations include (but aren't limited to) atomic loads/stores, compare-and-swap, atomic flags, promises, futures, locks, and condition variables.

std::async

`std::async` runs the given function either asynchronously or lazily-evaluated, then returns a `std::future` which holds the result of that function call.

The first parameter is the policy which can be:

- `std::launch::async` | `std::launch::deferred` It is up to the implementation whether to perform asynchronous execution or lazy evaluation.
- `std::launch::async` Run the callable object on a new thread.
- `std::launch::deferred` Perform lazy evaluation on the current thread.

```
1 int foo() {
2     /* Do something here, then return the result. */
3     return 1000;
4 }
5
6 auto handle = std::async(std::launch::async, foo); // create an
    async task
7 auto result = handle.get(); // wait for the result
```

std::begin/end

`std::begin` and `std::end` free functions were added to return begin and end iterators of a container generically. These functions also work with raw arrays which do not have begin and end member functions.

```
1 template <typename T>
2 int CountTwos(const T& container) {
3     return std::count_if(std::begin(container), std::end(container),
4         [](int item) {
5             return item == 2;
6         });
7 }
8
9 std::vector<int> vec = {2, 2, 43, 435, 4543, 534};
10 int arr[8] = {2, 43, 45, 435, 32, 32, 32, 32};
11 auto a = CountTwos(vec); // 2
12 auto b = CountTwos(arr); // 1
```

3.3 C++14 new language features

Following a summary of the new language features introduced with C++14.

binary literals

Binary literals provide a convenient way to represent a base-2 number. It is possible to separate digits with '.

```
1 0b110 // == 6
2 0b1111'1111 // == 255
```

generic lambda expressions

C++14 now allows the auto type-specifier in the parameter list, enabling polymorphic lambdas.

```
1 auto identity = [](auto x) { return x; };
2 int three = identity(3); // == 3
3 std::string foo = identity("foo"); // == "foo"
```

lambda capture initializers

This allows creating lambda captures initialized with arbitrary expressions. The name given to the captured value does not need to be related to any variables in the enclosing scopes and introduces a new name inside the lambda body. The initializing expression is evaluated when the lambda is created (not when it is invoked).

```
1 int factory(int i) { return i * 10; }
2 auto f = [x = factory(2)] { return x; }; // returns 20
3
4 auto generator = [x = 0] () mutable {
5     // this would not compile without 'mutable' as we are modifying x
6     // on each call
7     return x++;
8 };
9 auto a = generator(); // == 0
10 auto b = generator(); // == 1
11 auto c = generator(); // == 2
```

Because it is now possible to move (or forward) values into a lambda that could previously be only captured by copy or reference we can now capture move-only types in a lambda by value. Note that in the below example the p in the capture-list of task2 on the left-hand-side of = is a new variable private to the lambda body and does not refer to the original p.

```
1 auto p = std::make_unique<int>(1);
2
3 auto task1 = [=] { *p = 5; }; // ERROR: std::unique_ptr cannot be
4 // copied
5 // vs.
```

```

5 auto task2 = [p = std::move(p)] { *p = 5; }; // OK: p is move-
    constructed into the closure object
6 // the original p is empty after task2 is created

```

Using this reference-captures can have different names than the referenced variable.

```

1 auto x = 1;
2 auto f = [&r = x, x = x * 10] {
3     ++r;
4     return r + x;
5 };
6 f(); // sets x to 2 and returns 12

```

return type deduction

Using an auto return type in C++14, the compiler will attempt to deduce the type for you. With lambdas, you can now deduce its return type using auto, which makes returning a deduced reference or rvalue reference possible.

```

1 // Deduce return type as 'int'.
2 auto f(int i) {
3     return i;
4 }
5
6 template <typename T>
7 auto& f(T& t) {
8     return t;
9 }
10
11 // Returns a reference to a deduced type.
12 auto g = [] (auto& x) -> auto& { return f(x); };
13 int y = 123;
14 int& z = g(y); // reference to 'y'

```

decltype(auto)

The decltype(auto) type-specifier also deduces a type like auto does. However, it deduces return types while keeping their references and cv-qualifiers, while auto will not.

```

1 const int x = 0;
2 auto x1 = x; // int
3 decltype(auto) x2 = x; // const int
4 int y = 0;
5 int& y1 = y;
6 auto y2 = y1; // int
7 decltype(auto) y3 = y1; // int&
8 int&& z = 0;
9 auto z1 = std::move(z); // int
10 decltype(auto) z2 = std::move(z); // int&&
11
12 // Note: Especially useful for generic code!
13

```

```

14 // Return type is 'int'.
15 auto f(const int& i) {
16     return i;
17 }
18
19 // Return type is 'const int&'.
20 decltype(auto) g(const int& i) {
21     return i;
22 }
23
24 int x = 123;
25 static_assert(std::is_same<const int&, decltype(f(x))>::value == 0);
26 static_assert(std::is_same<int, decltype(f(x))>::value == 1);
27 static_assert(std::is_same<const int&, decltype(g(x))>::value == 1);

```

relaxing constraints on constexpr functions

In C++11, constexpr function bodies could only contain a very limited set of syntaxes, including (but not limited to): typedefs, usings, and a single return statement. In C++14, the set of allowable syntaxes expands greatly to include the most common syntax such as if statements, multiple returns, loops, etc.

```

1 constexpr int factorial(int n) {
2     if (n <= 1) {
3         return 1;
4     } else {
5         return n * factorial(n - 1);
6     }
7 }
8 factorial(5); // == 120

```

variable templates

C++14 allows variables to be templated:

```

1 template<class T>
2 constexpr T pi = T(3.1415926535897932385);
3 template<class T>
4 constexpr T e = T(2.7182818284590452353);

```

[[deprecated]] attribute

C++14 introduces the [[deprecated]] attribute to indicate that a unit (function, class, etc.) is discouraged and likely yield compilation warnings. If a reason is provided, it will be included in the warnings.

```

1 [[deprecated]]
2 void old_method();
3 [[deprecated("Use new_method instead")]]
4 void legacy_method();

```

3.4 C++14 new library features

Following a summary of the new library features introduced with C++14.

user-defined literals for standard library types

New user-defined literals for standard library types, including new built-in literals for chrono and basic_string. These can be constexpr meaning they can be used at compile-time. Some uses for these literals include compile-time integer parsing, binary literals, and imaginary number literals.

```
1 using namespace std::chrono_literals;
2 auto day = 24h;
3 day.count(); // == 24
4 std::chrono::duration_cast<std::chrono::minutes>(day).count(); // ==
    1440
```

compile-time integer sequences

The class template `std::integer_sequence` represents a compile-time sequence of integers. There are a few helpers built on top:

- `std::make_integer_sequence<T, N>` - creates a sequence of 0, ..., N - 1 with type T.
- `std::index_sequence_for<T...>` - converts a template parameter pack into an integer sequence.

Convert an array into a tuple:

```
1 template<typename Array, std::size_t... I>
2 decltype(auto) a2t_impl(const Array& a, std::integer_sequence<std::
    size_t, I...>) {
3     return std::make_tuple(a[I]...);
4 }
5
6 template<typename T, std::size_t N, typename Indices = std::
    make_index_sequence<N>>
7 decltype(auto) a2t(const std::array<T, N>& a) {
8     return a2t_impl(a, Indices());
9 }
```

std::make_unique

`std::make_unique` is the recommended way to create instances of `std::unique_ptr`s due to the following reasons:

- Avoid having to use the new operator.
- Prevents code repetition when specifying the underlying type the pointer shall hold.

- Most importantly, it provides exception-safety. Suppose we were calling a function `foo` like so:

```
1 foo(std::unique_ptr<T>{new T{}}, function_that_throws(),
2     std::unique_ptr<T>{new T{}});
```

The compiler is free to call `new T{}`, then `function_that_throws()`, and so on...

Since we have allocated data on the heap in the first construction of a `T`, we have introduced a leak here. With `std::make_unique`, we are given exception-safety:

```
1 foo(std::make_unique<T>(), function_that_throws(),
2     std::make_unique<T>());
```

See the section on smart pointers (C++11) for more information on `std::unique_ptr` and `std::shared_ptr`.

3.5 C++17 new language features

Following a summary of the new language features introduced with C++17.

template argument deduction for class templates

Automatic template argument deduction much like how it's done for functions, but now including class constructors.

```
1 template <typename T = float>
2 struct MyContainer {
3     T val;
4     MyContainer() : val{} {}
5     MyContainer(T val) : val{val} {}
6     // ...
7 };
8 MyContainer c1 {1}; // OK MyContainer<int>
9 MyContainer c2; // OK MyContainer<float>
```

declaring non-type template parameters with auto

Following the deduction rules of `auto`, while respecting the non-type template parameter list of allowable types^[*], template arguments can be deduced from the types of its arguments:

```
1 template <auto... seq>
2 struct my_integer_sequence {
3     // Implementation here ...
4 };
5
6 // Explicitly pass type 'int' as template argument.
7 auto seq = std::integer_sequence<int, 0, 1, 2>();
8 // Type is deduced to be 'int'.
9 auto seq2 = my_integer_sequence<0, 1, 2>();
```

* - For example, you cannot use a `double` as a template parameter type, which also makes this an invalid deduction using `auto`.

folding expressions

A fold expression performs a fold of a template parameter pack over a binary operator.

- An expression of the form `(... op e)` or `(e op ...)`, where `op` is a fold-operator and `e` is an unexpanded parameter pack, are called unary folds.
- An expression of the form `(e1 op ... op e2)`, where `op` are fold-operators, is called a binary fold. Either `e1` or `e2` is an unexpanded parameter pack, but not both.

```
1 template <typename... Args>
2 bool logicalAnd(Args... args) {
3     // Binary folding.
4     return (true && ... && args);
5 }
6
7 bool b = true;
8 bool& b2 = b;
9 logicalAnd(b, b2, true); // == true
10
11 template <typename... Args>
12 auto sum(Args... args) {
13     // Unary folding.
14     return (... + args);
15 }
16
17 sum(1.0, 2.0f, 3); // == 6.0
```

new rules for auto deduction from braced-init-list

Changes to auto deduction when used with the uniform initialization syntax. Previously, `auto x {3};` deduces a `std::initializer_list<int>`, which now deduces to `int`.

```
1
2 auto x1 {1, 2, 3}; // error: not a single element
3 auto x2 = {1, 2, 3}; // x2 is std::initializer_list<int>
4 auto x3 {3}; // x3 is int
5 auto x4 {3.0}; // x4 is double
```

constexpr lambda

Compile-time lambdas using `constexpr`.

```
1 auto identity = [](int n) constexpr { return n; };
2 static_assert(identity(123) == 123);
3
4 constexpr auto add = [](int x, int y) {
5     auto L = [=] { return x; };
6     auto R = [=] { return y; };
7     return [=] { return L() + R(); };
8 };
9
10 static_assert(add(1, 2)() == 3);
```

```

8 \begin{lstlisting}[language=C++]
9
10 constexpr int addOne(int n) {
11     return [n] { return n + 1; }();
12 }
13
14 static_assert(addOne(1) == 2);

```

lambda capture this by value

Capturing this in a lambda's environment was previously reference-only. An example of where this is problematic is asynchronous code using callbacks that require an object to be available, potentially past its lifetime. `*this` (C++17) will now make a copy of the current object, while `this` (C++11) continues to capture by reference.

```

1 struct MyObj {
2     int value {123};
3     auto getValueCopy() {
4         return [*this] { return value; };
5     }
6     auto getValueRef() {
7         return [this] { return value; };
8     }
9 };
10 MyObj mo;
11 auto valueCopy = mo.getValueCopy();
12 auto valueRef = mo.getValueRef();
13 mo.value = 321;
14 valueCopy(); // 123
15 valueRef(); // 321

```

inline variables

The inline specifier can be applied to variables as well as to functions. A variable declared inline has the same semantics as a function declared inline.

```

1 // Disassembly example using compiler explorer.
2 struct S { int x; };
3 inline S x1 = S{321}; // mov esi, dword ptr [x1]
4                       // x1: .long 321
5
6 S x2 = S{123};        // mov eax, dword ptr [.L_ZZ4mainE2x2]
7                       // mov dword ptr [rbp - 8], eax
8                       // .L_ZZ4mainE2x2: .long 123

```

It can also be used to declare and define a static member variable, such that it does not need to be initialized in the source file.

```

1 struct S {
2     S() : id{count++} {}
3     ~S() { count--; }
4     int id;

```



```

5   static inline int count{0}; // declare and initialize count to 0
   within the class
6 };

```

nested namespaces

Using the namespace resolution operator to create nested namespace definitions.

```

1 namespace A {
2     namespace B {
3         namespace C {
4             int i;
5         }
6     }
7 }

```

The code above can be written like this:

```

1 namespace A::B::C {
2     int i;
3 }

```

structured bindings

A proposal for de-structuring initialization, that would allow writing `auto [ x, y, z ] = expr;` where the type of `expr` was a tuple-like object, whose elements would be bound to the variables `x`, `y`, and `z` (which this construct declares). Tuple-like objects include `std::tuple`, `std::pair`, `std::array`, and aggregate structures.

```

1 using Coordinate = std::pair<int, int>;
2 Coordinate origin() {
3     return Coordinate{0, 0};
4 }
5
6 const auto [ x, y ] = origin();
7 x; // == 0
8 y; // == 0

```

```

1 std::unordered_map<std::string, int> mapping {
2     {"a", 1},
3     {"b", 2},
4     {"c", 3}
5 };
6
7 // Destructure by reference.
8 for (const auto& [key, value] : mapping) {
9     // Do something with key and value
10 }

```

selection statements with initializer

New versions of the `if` and `switch` statements which simplify common code patterns and help users keep scopes tight.

```

1 {
2     std::lock_guard<std::mutex> lk(mx);
3     if (v.empty()) v.push_back(val);
4 }
5 // vs.
6 if (std::lock_guard<std::mutex> lk(mx); v.empty()) {
7     v.push_back(val);
8 }

```

```

1 Foo gadget(args);
2 switch (auto s = gadget.status()) {
3     case OK: gadget.zip(); break;
4     case Bad: throw BadFoo(s.message());
5 }
6 // vs.
7 switch (Foo gadget(args); auto s = gadget.status()) {
8     case OK: gadget.zip(); break;
9     case Bad: throw BadFoo(s.message());
10 }

```

constexpr if

Write code that is instantiated depending on a compile-time condition.

```

1 template <typename T>
2 constexpr bool isIntegral() {
3     if constexpr (std::is_integral<T>::value) {
4         return true;
5     } else {
6         return false;
7     }
8 }
9 static_assert(isIntegral<int>() == true);
10 static_assert(isIntegral<char>() == true);
11 static_assert(isIntegral<double>() == false);
12 struct S {};
13 static_assert(isIntegral<S>() == false);

```

utf-8 character literals

A character literal that begins with u8 is a character literal of type char. The value of a UTF-8 character literal is equal to its ISO 10646 code point value.

```

1 char x = u8'x';

```

direct-list-initialization of enums

Enums can now be initialized using braced syntax.

```

1
2 enum byte : unsigned char {};
3 byte b {0}; // OK

```

```

4 byte c {-1}; // ERROR
5 byte d = byte{1}; // OK
6 byte e = byte{256}; // ERROR

```

[[fallthrough]], [[nodiscard]], [[maybe_unused]] attributes

C++17 introduces three new attributes:

[[fallthrough]], [[nodiscard]] and [[maybe_unused]].

- [[fallthrough]] indicates to the compiler that falling through in a switch statement is intended behavior. This attribute may only be used in a switch statement, and must be placed before the next case/default label.

```

1 switch (n) {
2     case 1:
3         // ...
4         [[fallthrough]];
5     case 2:
6         // ...
7         break;
8     case 3:
9         // ...
10        [[fallthrough]];
11    default:
12        // ...
13 }

```

- [[nodiscard]] issues a warning when either a function or class has this attribute and its return value is discarded.

```

1 [[nodiscard]] bool do_something() {
2     return is_success; // true for success, false for failure
3 }
4
5 do_something(); // warning: ignoring return value of 'bool
6                 do_something()',
7                 // declared with attribute 'nodiscard'
8
9 // Only issues a warning when 'error_info' is returned by value
10
11 struct [[nodiscard]] error_info {
12     // ...
13 };
14
15 error_info do_something() {
16     error_info ei;
17     // ...
18     return ei;
19 }
20
21 do_something(); // warning: ignoring returned value of type '
22                 error_info',

```

```
13 // declared with attribute 'nodiscard'
```

- `[[maybe_unused]]` indicates to the compiler that a variable or parameter might be unused and is intended.

```
1 void my_callback(std::string msg, [[maybe_unused]] bool error)
2 {
3     // Don't care if 'msg' is an error message, just log it.
4     log(msg);
5 }
```

`__has_include`

`__has_include` (operand) operator may be used in `#if` and `#elif` expressions to check whether a header or source file (operand) is available for inclusion or not.

One use case of this would be using two libraries that work the same way, using the backup/experimental one if the preferred one is not found on the system.

```
1 #ifdef __has_include
2 #   if __has_include(<optional>)
3 #       include <optional>
4 #       define have_optional 1
5 #   elif __has_include(<experimental/optional>)
6 #       include <experimental/optional>
7 #       define have_optional 1
8 #       define experimental_optional
9 #   else
10 #       define have_optional 0
11 #   endif
12 #endif
```

It can also be used to include headers existing under different names or locations on various platforms, without knowing which platform the program is running on, OpenGL headers are a good example for this which are located in OpenGL directory on macOS and GL on other platforms.

```
1 #ifdef __has_include
2 #   if __has_include(<OpenGL/gl.h>)
3 #       include <OpenGL/gl.h>
4 #       include <OpenGL/glu.h>
5 #   elif __has_include(<GL/gl.h>)
6 #       include <GL/gl.h>
7 #       include <GL/glu.h>
8 #   else
9 #       error No suitable OpenGL headers found.
10 #   endif
11 #endif
```

class template argument deduction

Class template argument deduction (CTAD) allows the compiler to deduce template arguments from constructor arguments.

```

1 std::vector v{ 1, 2, 3 }; // deduces std::vector<int>
2
3 std::mutex mtx;
4 auto lck = std::lock_guard{ mtx }; // deduces to std::lock_guard<std
   ::mutex>
5
6 auto p = new std::pair{ 1.0, 2.0 }; // deduces to std::pair<double,
   double>

```

For user-defined types, deduction guides can be used to guide the compiler how to deduce template arguments if applicable:

```

1 template <typename T>
2 struct container {
3     container(T t) {}
4
5     template <typename Iter>
6     container(Iter beg, Iter end);
7 };
8
9 // deduction guide
10 template <typename Iter>
11 container(Iter b, Iter e) -> container<typename std::iterator_traits
   <Iter>::value_type>;
12
13 container a{ 7 }; // OK: deduces container<int>
14
15 std::vector<double> v{ 1.0, 2.0, 3.0 };
16 auto b = container{ v.begin(), v.end() }; // OK: deduces container<
   double>
17
18 container c{ 5, 6 }; // ERROR: std::iterator_traits<int>::value_type
   is not a type

```

3.6 C++17 new library features:

Following a summary of the new library features introduced with C++17.

std::variant

The class template `std::variant` represents a type-safe union. An instance of `std::variant` at any given time holds a value of one of its alternative types (it's also possible for it to be valueless).

```

1 std::variant<int, double> v{ 12 };
2 std::get<int>(v); // == 12
3 std::get<0>(v); // == 12
4 v = 12.0;
5 std::get<double>(v); // == 12.0
6 std::get<1>(v); // == 12.0

```

std::optional

The class template `std::optional` manages an optional contained value, i.e. a value that may or may not be present. A common use case for optional is the return value of a function that may fail.

```
1 std::optional<std::string> create(bool b) {
2     if (b) {
3         return "Godzilla";
4     } else {
5         return {};
6     }
7 }
8
9 create(false).value_or("empty"); // == "empty"
10 create(true).value(); // == "Godzilla"
11 // optional-returning factory functions are usable as conditions of
12 // while and if
13 if (auto str = create(true)) {
14     // ...
15 }
```

std::any

A type-safe container for single values of any type.

```
1 std::any x {5};
2 x.has_value() // == true
3 std::any_cast<int>(x) // == 5
4 std::any_cast<int&>(x) = 10;
5 std::any_cast<int>(x) // == 10
```

std::string_view

A non-owning reference to a string. Useful for providing an abstraction on top of strings (e.g. for parsing).

```
1 // Regular strings.
2 std::string_view cppstr {"foo"};
3 // Wide strings.
4 std::wstring_view wctr_v {L"baz"};
5 // Character arrays.
6 char array[3] = {'b', 'a', 'r'};
7 std::string_view array_v(array, std::size(array));
8
9 std::string str {"    trim me"};
10 std::string_view v {str};
11 v.remove_prefix(std::min(v.find_first_not_of(" "), v.size()));
12 str; // == "    trim me"
13 v; // == "trim me"
```

std::invoke

Invoke a Callable object with parameters. Examples of callable objects are std::function or lambdas; objects that can be called similarly to a regular function.

```
1 template <typename Callable>
2 class Proxy {
3     Callable c_;
4
5 public:
6     Proxy(Callable c) : c_{ std::move(c) } {}
7
8     template <typename... Args>
9     decltype(auto) operator()(Args&&... args) {
10         // ...
11         return std::invoke(c_, std::forward<Args>(args)...);
12     }
13 };
14
15 const auto add = [](int x, int y) { return x + y; };
16 Proxy p{ add };
17 p(1, 2); // == 3
```

std::apply

Invoke a Callable object with a tuple of arguments.

```
1     auto add = [](int x, int y) {
2         return x + y;
3     };
4 std::apply(add, std::make_tuple(1, 2)); // == 3
```

std::filesystem

The new std::filesystem library provides a standard way to manipulate files, directories, and paths in a filesystem.

Here, a big file is copied to a temporary path if there is available space:

```
1 const auto bigFilePath {"bigFileToCopy"};
2 if (std::filesystem::exists(bigFilePath)) {
3     const auto bigFileSize {std::filesystem::file_size(bigFilePath)};
4     std::filesystem::path tmpPath {"/tmp"};
5     if (std::filesystem::space(tmpPath).available > bigFileSize) {
6         std::filesystem::create_directory(tmpPath.append("example"));
7         std::filesystem::copy_file(bigFilePath, tmpPath.append("newFile"
8         ));
9     }
10 }
```

std::byte

The new std::byte type provides a standard way of representing data as a byte. Benefits of using std::byte over char or unsigned char is that it is not a character type, and is also not an arithmetic type; while the only operator overloads available are bitwise operations.

S

```
1 std::byte a {0};
2 std::byte b {0xFF};
3 int i = std::to_integer<int>(b); // 0xFF
4 std::byte c = a & b;
5 int j = std::to_integer<int>(c); // 0
```

Note that std::byte is simply an enum, and braced initialization of enums become possible thanks to direct-list-initialization of enums.

splicing for maps and sets

Moving nodes and merging containers without the overhead of expensive copies, moves, or heap allocations/deallocations.

Moving elements from one map to another:

```
1 std::map<int, string> src {{1, "one"}, {2, "two"}, {3, "buckle my
   shoe"}};
2 std::map<int, string> dst {{3, "three"}};
3 dst.insert(src.extract(src.find(1))); // Cheap remove and insert of
   { 1, "one" } from 'src' to 'dst'.
4 dst.insert(src.extract(2)); // Cheap remove and insert of { 2, "two"
   } from 'src' to 'dst'.
5 // dst == { { 1, "one" }, { 2, "two" }, { 3, "three" } };
```

Inserting an entire set:

```
1 std::set<int> src {1, 3, 5};
2 std::set<int> dst {2, 4, 5};
3 dst.merge(src);
4 // src == { 5 }
5 // dst == { 1, 2, 3, 4, 5 }
```

Inserting elements which outlive the container:

```
1 auto elementFactory() {
2     std::set<...> s;
3     s.emplace(...);
4     return s.extract(s.begin());
5 }
6 s2.insert(elementFactory());
```

Changing the key of a map element:

```
1 std::map<int, string> m {{1, "one"}, {2, "two"}, {3, "three"}};
2 auto e = m.extract(2);
3 e.key() = 4;
4 m.insert(std::move(e));
5 // m == { { 1, "one" }, { 3, "three" }, { 4, "two" } }
```


parallel algorithms

Many of the STL algorithms, such as the copy, find and sort methods, started to support the parallel execution policies: seq, par and par_unseq which translate to "sequentially", "parallel" and "parallel unsequenced".

```
1 std::vector<int> longVector;
2 // Find element using parallel execution policy
3 auto result1 = std::find(std::execution::par, std::begin(longVector)
4     , std::end(longVector), 2);
5 // Sort elements using sequential execution policy
6 auto result2 = std::sort(std::execution::seq, std::begin(longVector)
7     , std::end(longVector));
```

std::sample

Samples n elements in the given sequence (without replacement) where every element has an equal chance of being selected.

```
1 const std::string ALLOWED_CHARS = "
2     abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
3 std::string guid;
4 // Sample 5 characters from ALLOWED_CHARS.
5 std::sample(ALLOWED_CHARS.begin(), ALLOWED_CHARS.end(), std::
6     back_inserter(guid),
7     5, std::mt19937{ std::random_device{}() }));
8
9 std::cout << guid; // e.g. G1fW2
```

std::clamp

Clamp given value between a lower and upper bound.

```
1 std::clamp(42, -1, 1); // == 1
2 std::clamp(-42, -1, 1); // == -1
3 std::clamp(0, -1, 1); // == 0
4
5 // 'std::clamp' also accepts a custom comparator:
6 std::clamp(0, -1, 1, std::less<>{}); // == 0
```

std::reduce

Fold over a given range of elements. Conceptually similar to std::accumulate, but std::reduce will perform the fold in parallel. Due to the fold being done in parallel, if you specify a binary operation, it is required to be associative and commutative. A given binary operation also should not change any element or invalidate any iterators within the given range.

The default binary operation is std::plus with an initial value of 0.

```
1 const std::array<int, 3> a{ 1, 2, 3 };
2 std::reduce(std::cbegin(a), std::cend(a)); // == 6
```

```

3 // Using a custom binary op:
4 std::reduce(std::cbegin(a), std::cend(a), 1, std::multiplies<>{});
    // == 6

```

Additionally you can specify transformations for reducers:

```

1 std::transform_reduce(std::cbegin(a), std::cend(a), 0, std::plus
    <>{}, times_ten); // == 60
2
3 const std::array<int, 3> b{ 1, 2, 3 };
4 const auto product_times_ten = [](const auto a, const auto b) {
    return a * b * 10; };
5
6 std::transform_reduce(std::cbegin(a), std::cend(a), std::cbegin(b),
    0, std::plus<>{}, product_times_ten); // == 140

```

prefix sum algorithms

Support for prefix sums (both inclusive and exclusive scans) along with transformations.

```
1 const std::array<int, 3> a{ 1, 2, 3 };
2
3 std::inclusive_scan(std::cbegin(a), std::cend(a),
4     std::ostream_iterator<int>{ std::cout, " " }, std::plus<>{}); //
    1 3 6
5
6 std::exclusive_scan(std::cbegin(a), std::cend(a),
7     std::ostream_iterator<int>{ std::cout, " " }, 0, std::plus<>{});
    // 0 1 3
8
9 const auto times_ten = [](const auto n) { return n * 10; };
10
11 std::transform_inclusive_scan(std::cbegin(a), std::cend(a),
12     std::ostream_iterator<int>{ std::cout, " " }, std::plus<>{},
13     times_ten); // 10 30 60
14
15 std::transform_exclusive_scan(std::cbegin(a), std::cend(a),
16     std::ostream_iterator<int>{ std::cout, " " }, 0, std::plus<>{},
17     times_ten); // 0 10 30
```

gcd and lcm

Greatest common divisor (GCD) and least common multiple (LCM).

```
1 const int p = 9;
2 const int q = 3;
3 std::gcd(p, q); // == 3
4 std::lcm(p, q); // == 9
```

std::not_fn

Utility function that returns the negation of the result of the given function.

```
1 const std::ostream_iterator<int> ostream_it{ std::cout, " " };
2 const auto is_even = [](const auto n) { return n % 2 == 0; };
3 std::vector<int> v{ 0, 1, 2, 3, 4 };
4
5 // Print all even numbers.
6 std::copy_if(std::cbegin(v), std::cend(v), ostream_it, is_even); //
    0 2 4
7 // Print all odd (not even) numbers.
8 std::copy_if(std::cbegin(v), std::cend(v), ostream_it, std::not_fn(
    is_even)); // 1 3
```

string conversion to/from numbers

Convert integrals and floats to a string or vice-versa. Conversions are non-throwing, do not allocate, and are more secure than the equivalents from the C standard library.

Users are responsible for allocating enough storage required for `std::to_chars`, or the function will fail by setting the error code object in its return value.

These functions allow you to optionally pass a base (defaults to base-10) or a format specifier for floating type input.

`std::to_chars` returns a (non-const) char pointer which is one-past-the-end of the string that the function wrote to inside the given buffer, and an error code object. `std::from_chars` returns a const char pointer which on success is equal to the end pointer passed to the function, and an error code object.

Both error code objects returned from these functions are equal to the default-initialized error code object on success.

Convert the number 123 to a `std::string`:

```
1 const int n = 123;
2
3 // Can use any container, string, array, etc.
4 std::string str;
5 str.resize(3); // hold enough storage for each digit of 'n'
6
7 const auto [ ptr, ec ] = std::to_chars(str.data(), str.data() + str.
    size(), n);
8
9 if (ec == std::errc{}) { std::cout << str << std::endl; } // 123
10 else { /* handle failure */ }
```

Convert from a `std::string` with value "123" to an integer:

```
1 const std::string str{ "123" };
2 int n;
3
4 const auto [ ptr, ec ] = std::from_chars(str.data(), str.data() +
    str.size(), n);
5
6 if (ec == std::errc{}) { std::cout << n << std::endl; } // 123
7 else { /* handle failure */ }
```

3.7 C++20 new language features

Following a summary of the new language features introduced with C++20.

coroutines

Coroutines are special functions that can have their execution suspended and resumed. To define a coroutine, the `co_return`, `co_await`, or `co_yield` keywords must be present in the function's body. C++20's coroutines are stackless; unless optimized out by the compiler, their state is allocated on the heap.

An example of a coroutine is a generator function, which yields (i.e. generates) a value at each invocation:

```
1 generator<int> range(int start, int end) {
2     while (start < end) {
3         co_yield start;
4         start++;
5     }
6
7     // Implicit co_return at the end of this function:
8     // co_return;
9 }
10
11 for (int n : range(0, 10)) {
12     std::cout << n << std::endl;
13 }
```

The above range generator function generates values starting at start until end (exclusive), with each iteration step yielding the current value stored in start. The generator maintains its state across each invocation of range (in this case, the invocation is for each iteration in the for loop). `co_yield` takes the given expression, yields (i.e. returns) its value, and suspends the coroutine at that point. Upon resuming, execution continues after the `co_yield`.

Another example of a coroutine is a task, which is an asynchronous computation that is executed when the task is awaited:

```
1 task<void> echo(socket s) {
2     for (;;) {
3         auto data = co_await s.async_read();
4         co_await async_write(s, data);
5     }
6
7     // Implicit co_return at the end of this function:
8     // co_return;
9 }
```

In this example, the `co_await` keyword is introduced. This keyword takes an expression and suspends execution if the thing you're awaiting on (in this case, the read or write) is not ready, otherwise you continue execution. (Note that under the hood, `co_yield` uses `co_await`.)

Using a task to lazily evaluate a value:

```
1 task<int> calculate_meaning_of_life() {
2     co_return 42;
3 }
4
5 auto meaning_of_life = calculate_meaning_of_life();
6 // ...
7 co_await meaning_of_life; // == 42
```

Note: While these examples illustrate how to use coroutines at a basic level, there is lots more going on when the code is compiled. These examples are not meant to be complete coverage of C++20's coroutines. Since the generator and task classes are not provided by the standard library yet, I used the cppcoro library to compile these examples.

concepts

Concepts are named compile-time predicates which constrain types. They take the following form:

```
1 template < template-parameter-list >
2 concept concept-name = constraint-expression;
```

where constraint-expression evaluates to a constexpr Boolean. Constraints should model semantic requirements, such as whether a type is a numeric or hashable. A compiler error results if a given type does not satisfy the concept it's bound by (i.e. constraint-expression returns false). Because constraints are evaluated at compile-time, they can provide more meaningful error messages and runtime safety.

```
1 // 'T' is not limited by any constraints.
2 template <typename T>
3 concept always_satisfied = true;
4 // Limit 'T' to integrals.
5 template <typename T>
6 concept integral = std::is_integral_v<T>;
7 // Limit 'T' to both the 'integral' constraint and signedness.
8 template <typename T>
9 concept signed_integral = integral<T> && std::is_signed_v<T>;
10 // Limit 'T' to both the 'integral' constraint and the negation of
    the 'signed_integral' constraint.
11 template <typename T>
12 concept unsigned_integral = integral<T> && !signed_integral<T>;
```

There are a variety of syntactic forms for enforcing concepts:

```
1 // Forms for function parameters:
2 // 'T' is a constrained type template parameter.
3 template <my_concept T>
4 void f(T v);
5
6 // 'T' is a constrained type template parameter.
7 template <typename T>
```

```

8     requires my_concept<T>
9 void f(T v);
10
11 // 'T' is a constrained type template parameter.
12 template <typename T>
13 void f(T v) requires my_concept<T>;
14
15 // 'v' is a constrained deduced parameter.
16 void f(my_concept auto v);
17
18 // 'v' is a constrained non-type template parameter.
19 template <my_concept auto v>
20 void g();
21
22 // Forms for auto-deduced variables:
23 // 'foo' is a constrained auto-deduced value.
24 my_concept auto foo = ...;
25
26 // Forms for lambdas:
27 // 'T' is a constrained type template parameter.
28 auto f = []<my_concept T> (T v) {
29     // ...
30 };
31 // 'T' is a constrained type template parameter.
32 auto f = []<typename T> requires my_concept<T> (T v) {
33     // ...
34 };
35 // 'T' is a constrained type template parameter.
36 auto f = []<typename T> (T v) requires my_concept<T> {
37     // ...
38 };
39 // 'v' is a constrained deduced parameter.
40 auto f = [](my_concept auto v) {
41     // ...
42 };
43 // 'v' is a constrained non-type template parameter.
44 auto g = []<my_concept auto v> () {
45     // ...
46 };

```

The `requires` keyword is used either to start a `requires` clause or a `requires` expression:

```

1 template <typename T>
2     requires my_concept<T> // 'requires' clause.
3 void f(T);
4
5 template <typename T>
6 concept callable = requires (T f) { f(); }; // 'requires' expression
7
8 template <typename T>
9     requires requires (T x) { x + x; } // 'requires' clause and
    expression on same line.

```

```

10 T add(T a, T b) {
11     return a + b;
12 }
13
14 Note that the parameter list in a requires expression is optional.
    Each requirement in a requires expression are one of the
    following:
15
16     Simple requirements - asserts that the given expression is valid
    .
17
18 \begin{lstlisting}[language=C++]
19 template <typename T>
20 concept callable = requires (T f) { f(); };

```

Type requirements - denoted by the typename keyword followed by a type name, asserts that the given type name is valid.

```

1 struct foo {
2     int foo;
3 };
4
5 struct bar {
6     using value = int;
7     value data;
8 };
9
10 struct baz {
11     using value = int;
12     value data;
13 };
14
15 // Using SFINAE, enable if 'T' is a 'baz'.
16 template <typename T, typename = std::enable_if_t<std::is_same_v<T,
    baz>>>
17 struct S {};
18
19 template <typename T>
20 using Ref = T&;
21
22 template <typename T>
23 concept C = requires {
24     // Requirements on type 'T':
25     typename T::value; // A) has an inner member named 'value'
26     typename S<T>;     // B) must have a valid class template
    specialization for 'S'
27     typename Ref<T>;   // C) must be a valid alias template
    substitution
28 };
29
30 template <C T>
31 void g(T a);

```



```

32
33 g(foo{}); // ERROR: Fails requirement A.
34 g(bar{}); // ERROR: Fails requirement B.
35 g(baz{}); // PASS.

```

Compound requirements - an expression in braces followed by a trailing return type or type constraint.

```

1  template <typename T>
2  concept C = requires(T x) {
3      {x} -> std::convertible_to<typename T::inner>; // the type of the
4      expression 'x' is convertible to 'T::inner'
5      {x + 1} -> std::same_as<int>; // the expression 'x + 1' satisfies
6      'std::same_as<decltype((x + 1))>'
7      {x * 1} -> std::convertible_to<T>; // the type of the expression '
8      x * 1' is convertible to 'T'
9  };
10
11      Nested requirements - denoted by the requires keyword, specify
12      additional constraints (such as those on local parameter
13      arguments).
14
15 \begin{lstlisting}[language=C++]
16 template <typename T>
17 concept C = requires(T x) {
18     requires std::same_as<sizeof(x), size_t>;
19 };

```

See also: concepts library.

designated initializers

C-style designated initializer syntax. Any member fields that are not explicitly listed in the designated initializer list are default-initialized.

```

1  struct A {
2      int x;
3      int y;
4      int z = 123;
5  };
6
7  A a {.x = 1, .z = 2}; // a.x == 1, a.y == 0, a.z == 2

```

template syntax for lambdas

Use familiar template syntax in lambda expressions.

```

1  auto f = []<typename T>(std::vector<T> v) {
2      // ...
3  };

```

range-based for loop with initializer

This feature simplifies common code patterns, helps keep scopes tight, and offers an elegant solution to a common lifetime problem.

```
1 for (auto v = std::vector{1, 2, 3}; auto& e : v) {
2     std::cout << e;
3 }
4 // prints "123"
```

[[likely]] and [[unlikely]] attributes

Provides a hint to the optimizer that the labelled statement has a high probability of being executed.

```
1 switch (n) {
2 case 1:
3     // ...
4     break;
5
6 [[likely]] case 2: // n == 2 is considered to be arbitrarily more
7     // ...         // likely than any other value of n
8     break;
9 }
```

If one of the likely/unlikely attributes appears after the right parenthesis of an if-statement, it indicates that the branch is likely/unlikely to have its substatement (body) executed.

```
1 int random = get_random_number_between_x_and_y(0, 3);
2 if (random > 0) [[likely]] {
3     // body of if statement
4     // ...
5 }
```

It can also be applied to the substatement (body) of an iteration statement.

```
1 while (unlikely_truthy_condition) [[unlikely]] {
2     // body of while statement
3     // ...
4 }
```

deprecate implicit capture of this

Implicitly capturing this in a lambda capture using [=] is now deprecated; prefer capturing explicitly using [=, this] or [=, *this].

```
1 struct int_value {
2     int n = 0;
3     auto getter_fn() {
4         // BAD:
5         // return [=]() { return n; };
6
7         // GOOD:
8         return [=, *this]() { return n; };
9 }
```

```
9     }  
10 };
```

class types in non-type template parameters

Classes can now be used in non-type template parameters. Objects passed in as template arguments have the type `const T`, where `T` is the type of the object, and has static storage duration.

```
1 struct foo {  
2     foo() = default;  
3     constexpr foo(int) {}  
4 };  
5  
6 template <foo f>  
7 auto get_foo() {  
8     return f;  
9 }  
10  
11 get_foo(); // uses implicit constructor  
12 get_foo<foo{123}>();
```

constexpr virtual functions

Virtual functions can now be `constexpr` and evaluated at compile-time. `constexpr` virtual functions can override non-`constexpr` virtual functions and vice-versa.

```
1 struct X1 {  
2     virtual int f() const = 0;  
3 };  
4  
5 struct X2: public X1 {  
6     constexpr virtual int f() const { return 2; }  
7 };  
8  
9 struct X3: public X2 {  
10     virtual int f() const { return 3; }  
11 };  
12  
13 struct X4: public X3 {  
14     constexpr virtual int f() const { return 4; }  
15 };  
16  
17 constexpr X4 x4;  
18 x4.f(); // == 4
```

explicit(bool)

Conditionally select at compile-time whether a constructor is made explicit or not. `explicit(true)` is the same as specifying `explicit`.

```

1 struct foo {
2     // Specify non-integral types (strings, floats, etc.) require
      explicit construction.
3     template <typename T>
4     explicit(!std::is_integral_v<T>) foo(T) {}
5 };
6
7 foo a = 123; // OK
8 foo b = "123"; // ERROR: explicit constructor is not a candidate (
      explicit specifier evaluates to true)
9 foo c {"123"}; // OK

```

immediate functions

Similar to constexpr functions, but functions with a consteval specifier must produce a constant. These are called immediate functions.

```

1 consteval int sqr(int n) {
2     return n * n;
3 }
4
5 constexpr int r = sqr(100); // OK
6 int x = 100;
7 int r2 = sqr(x); // ERROR: the value of 'x' is not usable in a
      constant expression
8             // OK if 'sqr' were a 'constexpr' function

```

using enum

Bring an enum's members into scope to improve readability. Before:

```

1 enum class rgba_color_channel { red, green, blue, alpha };
2
3 std::string_view to_string(rgba_color_channel channel) {
4     switch (channel) {
5         case rgba_color_channel::red:    return "red";
6         case rgba_color_channel::green:  return "green";
7         case rgba_color_channel::blue:   return "blue";
8         case rgba_color_channel::alpha:  return "alpha";
9     }
10 }

```

After:

```

1 enum class rgba_color_channel { red, green, blue, alpha };
2
3 std::string_view to_string(rgba_color_channel my_channel) {
4     switch (my_channel) {
5         using enum rgba_color_channel;
6         case red:    return "red";
7         case green:  return "green";
8         case blue:   return "blue";
9         case alpha:  return "alpha";

```

```

10 }
11 }

```

lambda capture of parameter pack

Capture parameter packs by value:

```

1 template <typename... Args>
2 auto f(Args&&... args){
3     // BY VALUE:
4     return [...args = std::forward<Args>(args)] {
5         // ...
6     };
7 }

```

Capture parameter packs by reference:

```

1 template <typename... Args>
2 auto f(Args&&... args){
3     // BY REFERENCE:
4     return [&...args = std::forward<Args>(args)] {
5         // ...
6     };
7 }

```

char8_t

Provides a standard type for representing UTF-8 strings.

```

1 char8_t utf8_str[] = u8"\u0123";

```

constexpr

The constexpr specifier requires that a variable must be initialized at compile-time.

```

1 const char* g() { return "dynamic initialization"; }
2 constexpr const char* f(bool p) { return p ? "constant initializer"
3     : g(); }
4 constexpr const char* c = f(true); // OK
5 constexpr const char* d = f(false); // ERROR: 'g' is not constexpr,
6     so 'd' cannot be evaluated at compile-time.

```

3.8 C++20 new library features

Following a summary of the new library features introduced with C++20.

concepts library

Concepts are also provided by the standard library for building more complicated concepts. Some of these include:

Core language concepts:

same_as - specifies two types are the same. derived_from - specifies that a type is derived from another type. convertible_to - specifies that a type is implicitly convertible to another type. common_with_ - specifies that two types share a common type. integral - specifies that a type is an integral type. default_constructible - specifies that an object of a type can be default-constructed.

Comparison concepts:

boolean - specifies that a type can be used in Boolean contexts. equality_comparable - specifies that operator== is an equivalence relation.

Object concepts:

movable - specifies that an object of a type can be moved and swapped. copyable - specifies that an object of a type can be copied, moved, and swapped. semiregular - specifies that an object of a type can be copied, moved, swapped, and default constructed. regular - specifies that a type is regular, that is, it is both semiregular and equality_comparable.

Callable concepts:

invocable - specifies that a callable type can be invoked with a given set of argument types. predicate - specifies that a callable type is a Boolean predicate.

See also: concepts.

synchronized buffered outputstream

Buffers output operations for the wrapped output stream ensuring synchronization (i.e. no interleaving of output).

```
1 std::ostream{std::cout} << "The value of x is:" << x << std::endl;
```

std::span

A span is a view (i.e. non-owning) of a container providing bounds-checked access to a contiguous group of elements. Since views do not own their elements they are cheap to construct and copy – a simplified way to think about views is they are holding references to their data. As opposed to maintaining a pointer/iterator and length field, a span wraps both of those up in a single object.

Spans can be dynamically-sized or fixed-sized (known as their extent). Fixed-sized spans benefit from bounds-checking.

Span doesn't propagate const so to construct a read-only span use std::span<const T>.

Example: using a dynamically-sized span to print integers from various containers.

```
1 void print_ints(std::span<const int> ints) {
2     for (const auto n : ints) {
3         std::cout << n << std::endl;
4     }
5 }
6
7 print_ints(std::vector{ 1, 2, 3 });
8 print_ints(std::array<int, 5>{ 1, 2, 3, 4, 5 });
```

```

9
10 int a[10] = { 0 };
11 print_ints(a);
12 // etc.

```

Example: a statically-sized span will fail to compile for containers that don't match the extent of the span.

```

1 void print_three_ints(std::span<const int, 3> ints) {
2     for (const auto n : ints) {
3         std::cout << n << std::endl;
4     }
5 }
6
7 print_three_ints(std::vector{ 1, 2, 3 }); // ERROR
8 print_three_ints(std::array<int, 5>{ 1, 2, 3, 4, 5 }); // ERROR
9 int a[10] = { 0 };
10 print_three_ints(a); // ERROR
11
12 std::array<int, 3> b = { 1, 2, 3 };
13 print_three_ints(b); // OK
14
15 // You can construct a span manually if required:
16 std::vector c{ 1, 2, 3 };
17 print_three_ints(std::span<const int, 3>{ c.data(), 3 }); // OK: set
    pointer and length field.
18 print_three_ints(std::span<const int, 3>{ c.cbegin(), c.cend() });
    // OK: use iterator pairs.

```

bit operations

C++20 provides a new <bit> header which provides some bit operations including popcount.

```

1 std::popcount(0u); // 0
2 std::popcount(1u); // 1
3 std::popcount(0b1111'0000u); // 4

```

math constants

Mathematical constants including PI, Euler's number, etc. defined in the <numbers> header.

```

1 std::numbers::pi; // 3.14159...
2 std::numbers::e; // 2.71828...
3
4 \subsubsection{\href{https://github.com/AnthonyCalandra/modern-cpp-
    features/tree/master/\stdis_constant_evaluated}
5                 {std::is\char' _constant\char' _evaluated}}
6 Predicate function which is truthy when it is called in a compile-
    time context.
7

```

```

8 \begin{lstlisting}[language=C++]
9 constexpr bool is_compile_time() {
10     return std::is_constant_evaluated();
11 }
12
13 constexpr bool a = is_compile_time(); // true
14 bool b = is_compile_time(); // false

```

std::make_shared supports arrays

```

1 auto p = std::make_shared<int[]>(5); // pointer to 'int[5]'
2 // OR
3 auto p = std::make_shared<int[5]>(); // pointer to 'int[5]'

```

starts_with and ends_with on strings

Strings (and string views) now have the starts_with and ends_with member functions to check if a string starts or ends with the given string.

```

1 std::string str = "foobar";
2 str.starts_with("foo"); // true
3 str.ends_with("baz"); // false

```

check if associative container has element

Associative containers such as sets and maps have a contains member function, which can be used instead of the "find and check end of iterator" idiom.

```

1 std::map<int, char> map {{1, 'a'}, {2, 'b'}};
2 map.contains(2); // true
3 map.contains(123); // false
4
5 std::set<int> set {1, 2, 3};
6 set.contains(2); // true

```

std::bit_cast

A safer way to reinterpret an object from one type to another.

```

1 float f = 123.0;
2 int i = std::bit_cast<int>(f);

```

std::midpoint

Calculate the midpoint of two integers safely (without overflow).

```

1 std::midpoint(1, 3); // == 2

```


std::to_array

Converts the given array/"array-like" object to a std::array.

```
1 std::to_array("foo"); // returns 'std::array<char, 4>'
2 std::to_array<int>({1, 2, 3}); // returns 'std::array<int, 3>'
3
4 int a[] = {1, 2, 3};
5 std::to_array(a); // returns 'std::array<int, 3>'
```

C++ Bibliography

- [1] *The Most Vexing Parse: How to Spot It and Fix It Quickly*. [Fluent C++](#).
- [2] R. Martinho Fernandes. *Rule of Zero*. [ISOC++ Blog](#). 2012.
- [3] Scott Meyers. *A Concern about the Rule of Zero*. [The View from Aristeia - Scoot Meyers' Activities and Interests](#). 2014.
- [4] *A polymorphic class should suppress public copy/move*. [Standard Cpp Foundation Github](#).
- [5] *If you define or =delete any copy, move, or destructor function, define or =delete them all*. [Standard Cpp Foundation Github](#).
- [6] *References*. [ISOC++ Wiki](#).
- [7] *Most vexing parse*. [WikiPedia](#).
- [8] *Core C++ - lvalues and rvalues*. [Just Software Solutions](#). 2016.
- [9] *Rvalue References and Perfect Forwarding*. [Just Software Solutions](#). 2008.
- [10] Peter Dimov, Howard E. Hinnant, and Dave Abraham. *The Forwarding Problem: Arguments*. [open-std.org - N1385=02-0043](#).
- [11] Meyers, Sutter, and Alexandrescu. *Session Announcement: Adventures in Perfect Forwarding*. [C++ and Beyond](#). 2011.
- [12] *What are the main purposes of std::forward and which problems does it solve?* [StackOverflow Thread](#).
- [13] *What is the move semantic?* [StackOverflow Thread](#).
- [14] Howard Hinnant. *Everything you wanted to know about Move Semantics*. [Slideshare](#). 2014.
- [15] *If your function must not throw declare it noexcept*. [Cpp Core Guidelines](#).
- [16] *What is the meaning of auto keyword*. [StackOverflow Thread](#).
- [17] Herb Sutter. *auto variables, Part 1*. [Guru of the week - 92](#).
- [18] Herb Sutter. *auto variables, Part 2*. [Guru of the week - 93](#).
- [19] Herb Sutter. *AAA style (Almost Always auto)*. [Guru of the week - 94](#).
- [20] *What's In a Class? - The Interface Principle*. [Guru of the week - C++ Report, 10\(3\), March 1998](#).
- [21] *Declaring non-type template arguments with auto*. [Programming Language C++, Evolution Working Group - P0127R1](#). 2016.
- [22] *Declaring non-type template parameters with auto*. [Programming Language C++, Evolution Working Group - P0127R2](#). 2016.
- [23] *Pros and Cons of Alternative Function Syntax in C++*. [Petr Zemek's Blog of a Software Engineer](#).

- [24] *ADL - Argument-dependent lookup*. [Cplusplusreference](#).
- [25] *Lambda Expressions*. [Cplusplusreference](#).
- [26] Crowl Larvi Freeman. *Lambda Expressions and Closures: Wording for Monomorphic Lambdas (Revision 4)*. [open-std.org](#). 2008.
- [27] Vandevorde. *New wording for C++0x Lambdas*. [open-std.org](#). 2009.
- [28] *Closure (Computer Programming)*. [Wikipedia](#).
- [29] *Abbreviated function template*. [Cplusplusreference](#).
- [30] *decltype* specifier. [Cplusplusreference](#).
- [31] *Member Function Pointers and the Fastest Possible C++ Delegates*. [codeproject.com](#).
- [32] Stroustrup and Dos Reis. *Initializer list (Rev. 3)*. [open-std.org](#). 2007.
- [33] Merrill and Vandevorde. *Initializer Lists - Alternative Mechanism and Rationale (v.2)*. [Initializer Lists — Alternative Mechanism and Rationale \(v. 2\)](#). 2008.
- [34] Thorsten Ottosen. *Wording for range-based for-loop (revision 2)*. [open-std.org](#). 2007.
- [35] *Range-based for statements and ADL*. [open-std.org](#).
- [36] Svoboda. *Delete operators default to noexcept*. [open-std.org](#). 2010.
- [37] Mauer. *Deducing noexcept for destructors*. [open-std.org](#). 2010.
- [38] Abrahams, Sharoni, and Gregor. *Allowing Move Constructors to Throw (Rev. 1)*. [open-std.org](#). 2010.
- [39] Bjarne Stroustrup. *Bjarne Stroustrup's C++ Style and Technique FAQ*. [stroustrup.com](#).
- [40] Bjarne Stroustrup. *Exception Safety: Concepts and Techniques*. [stroustrup.com](#).
- [41] *Static Initialization Order Fiasco*. [cplusplusreference.com](#).
- [42] *Empty base optimization*. [cplusplusreference.com](#).
- [43] *Apache C++ Standard Template Library*. [apache.org](#).
- [44] *Lapack: Linear Algebra Subroutine Library*. [siam.org](#).
- [45] *Netlib*. [netlib.org](#).
- [46] *Available C++ Libraries FAQ maintained by Nikki Locke*. [trumphurst.com](#).
- [47] *Mathtools - A Resource for Technical Computing Community*. [mathtools.net](#).
- [48] *boost Library*. [boost.org](#).

Concurrency

C++ includes built-in support for threads, atomic operations, mutual exclusion, condition variables, and futures.

1 Threads

<code>thread</code>	manages a separate thread
<code>jthread</code>	thread with support for auto-joining and cancellation

```
1 #include <string>
2 #include <iostream>
3 #include <thread>
4
5 using namespace std;
6
7 // The function we want to execute on the new thread.
8 void task1(string msg)
9 {
```

```

10     cout << "task1 says: " << msg;
11 }
12
13 int main()
14 {
15     // Constructs the new thread and runs it. Does not block execution
16     .
17     thread t1(task1, "Hello");
18
19     // Do other things...
20
21     // Makes the main thread wait for the new thread to finish
22     // execution, therefore blocks its own execution.
23     t1.join();
24 }

```

- A program is an executable file which consists of a set of instructions to perform some task and is usually stored on the disk of your computer.
- A process is what we call a program that has been loaded into memory along with all the resources it needs to operate. It has its own memory space.
- A thread is the unit of execution within a process. A process can have multiple threads running as a part of it, where each thread uses the process's memory space and shares it with other threads.
- Multithreading is a technique where multiple threads are spawned by a process to do different tasks, at about the same time, just one after the other. This gives you the illusion that the threads are running in parallel, but they are actually run in a concurrent manner. In Python, the Global Interpreter Lock (GIL) prevents the threads from running simultaneously.
- Multiprocessing is a technique where parallelism in its truest form is achieved. Multiple processes are run across multiple CPU cores, which do not share the resources among them. Each process can have many threads running in its own memory space. In Python, each process has its own instance of Python interpreter doing the job of executing the instructions.

1.1 Functions managing the current thread

<code>yield</code>	suggests that the implementation reschedule execution of threads
<code>get_id</code>	returns the thread id of the current thread
<code>sleep_for</code>	stops the execution of the current thread for a specified time duration
<code>sleep_until</code>	stops the execution of the current thread until a specified time point

2 Atomicity

2.1 Atomic operations

These components are provided for fine-grained atomic operations allowing for lockless concurrent programming. Each atomic operation is indivisible with regards to any other atomic operation that involves the same object. Atomic objects are [free of data races](#).

2.2 Atomic types

atomic	atomic class template and specializations for bool, integral and pointer types
atomic_ref	provides atomic operations on non-atomic objects

2.3 Operations on atomic types

atomic_is_lock_free	checks if the atomic type's operations are lock-free
atomic_store atomic_store_explicit	atomically replaces the value of the atomic object with a non-atomic argument
atomic_load atomic_load_explicit	atomically obtains the value stored in an atomic object
atomic_exchange atomic_exchange_explicit	atomically replaces the value of the atomic object with non-atomic argument and returns the old value of the atomic
atomic_compare_exchange_weak atomic_compare_exchange_weak_explicit atomic_compare_exchange_strong atomic_compare_exchange_strong_explicit	atomically compares the value of the atomic object with non-atomic argument and performs atomic exchange if equal or atomic load if not
atomic_fetch_add atomic_fetch_add_explicit	adds a non-atomic value to an atomic object and obtains the previous value of the atomic

<code>atomic_fetch_sub</code> <code>atomic_fetch_sub_explicit</code>	subtracts a non-atomic value from an atomic object and obtains the previous value of the atomic
<code>atomic_fetch_and</code> <code>atomic_fetch_and_explicit</code> <code>atomic_fetch_or</code> <code>atomic_fetch_or_explicit</code>	replaces the atomic object with the result of bitwise AND with a non-atomic argument and obtains the previous value of the atomic replaces the atomic object with the result of bitwise OR with a non-atomic argument and obtains the previous value of the atomic
<code>atomic_fetch_xor</code> <code>atomic_fetch_xor_explicit</code>	blocks the thread until notified and the atomic value changes
<code>atomic_wait</code> <code>atomic_wait_explicit</code>	replaces the atomic object with the result of bitwise XOR with a non-atomic argument and obtains the previous value of the atomic
<code>atomic_notify_one</code>	notifies a thread blocked in <code>atomic_wait</code>
<code>atomic_notify_all</code>	notifies all threads blocked in <code>atomic_wait</code>

2.4 Flag type and operations

<code>atomic_flag</code>	the lock-free boolean atomic type
<code>atomic_flag_test_and_set</code> <code>atomic_flag_test_and_set_explicit</code>	atomically sets the flag to true and returns its previous value
<code>atomic_flag_clear</code> <code>atomic_flag_clear_explicit</code>	atomically sets the value of the flag to false
<code>atomic_flag_test</code> <code>atomic_flag_test_explicit</code>	atomically returns the value of the flag
<code>atomic_flag_wait</code> <code>atomic_flag_wait_explicit</code>	blocks the thread until notified and the flag changes
<code>atomic_flag_notify_one</code>	notifies a thread blocked in <code>atomic_flag_wait</code>
<code>atomic_flag_notify_all</code>	notifies all threads blocked in <code>atomic_flag_wait</code>

2.5 Initialization

<code>atomic_init</code>	non-atomic initialization of a default-constructed atomic object
<code>ATOMIC_VAR_INIT</code>	constant initialization of an atomic variable of static storage duration
<code>ATOMIC_FLAG_INIT</code>	initializes an <code>std::atomic_flag</code> to false

2.6 Memory synchronization ordering

<code>memory_order</code>	defines memory ordering constraints for the given atomic operation
<code>kill_dependency</code>	removes the specified object from the <code>memory_order_consume</code> dependency tree
<code>atomic_thread_fence</code>	generic memory order-dependent fence synchronization primitive
<code>atomic_signal_fence</code>	fence between a thread and a signal handler executed in the same thread

2.7 Mutual exclusion

Mutual exclusion algorithms prevent multiple threads from simultaneously accessing shared resources. This prevents data races and provides support for synchronization between threads.

<code>mutex</code>	provides basic mutual exclusion facility
<code>timed_mutex</code>	provides mutual exclusion facility which implement locking with a timeout
<code>recursive_mutex</code>	provides mutual exclusion facility which can be locked recursively by the same thread
<code>recursive_timed_mutex</code>	provides mutual exclusion facility which can be locked recursively by the same thread and implements locking with a timeout
<code>shared_mutex</code>	provides shared mutual exclusion facility
<code>shared_timed_mutex</code>	provides shared mutual exclusion facility and implements locking with a timeout

2.8 Generic mutex management

<code>lock_guard</code>	implements a strictly scope-based mutex ownership wrapper
<code>scoped_lock</code>	deadlock-avoiding RAII wrapper for multiple mutexes
<code>unique_lock</code>	implements movable mutex ownership wrapper
<code>shared_lock</code>	implements movable shared mutex ownership wrapper
<code>defer_lock_t</code> <code>try_to_lock_t</code> <code>adopt_lock_t</code>	tag type used to specify locking strategy
<code>defer_lock</code> <code>try_to_lock</code> <code>adopt_lock</code>	tag constants used to specify locking strategy

2.9 Generic locking management

<code>try_lock</code>	attempts to obtain ownership of mutexes via repeated calls to <code>try_lock</code>
<code>lock</code>	locks specified mutexes, blocks if any are unavailable

A lock allows only one thread to enter the part that's locked and the lock is not shared with any other processes.

A mutex is the same as a lock but it can be system wide (shared by multiple processes).

A semaphore does the same as a mutex but allows x number of threads to enter, this can be used for example to limit the number of cpu, io or ram intensive tasks running at the same time.

For a more detailed post about the differences between mutex and semaphore read [here](#). You also have read/write locks that allows either unlimited number of readers or 1 writer at any given time.

2.10 Call once

<code>once_flag</code>	helper object to ensure that <code>call_once</code> invokes the function only once
<code>call_once</code>	invokes a function only once even if called from multiple threads

2.11 Condition variables

<code>condition_variable</code>	provides a condition variable associated with a <code>std::unique_lock</code>
<code>condition_variable_any</code>	provides a condition variable associated with any lock type
<code>notify_all_at_thread_exit</code>	schedules a call to <code>notify_all</code> to be invoked when this thread is completely finished
<code>cv_status</code>	lists the possible results of timed waits on condition variables

2.12 Futures

<code>promise</code>	stores a value for asynchronous retrieval
<code>packaged_task</code>	packages a function to store its return value for asynchronous retrieval
<code>future</code>	waits for a value that is set asynchronously
<code>shared_future</code>	waits for a value (possibly referenced by other futures) that is set asynchronously
<code>async</code>	runs a function asynchronously (potentially in a new thread) and returns a <code>std::future</code> that will hold the result
<code>launch</code>	specifies the launch policy for <code>std::async</code>
<code>future_status</code>	specifies the results of timed waits performed on <code>std::future</code> and <code>std::shared_future</code>

2.13 Future errors

<code>future_error</code>	reports an error related to futures or promises
<code>future_category</code>	identifies the future error category
<code>future_errc</code>	identifies the future error codes

2.14 Memory Order

[C++ reference documentation about `std::memory\_order`](#).

C++ defines several memory orders to specify how memory accesses are to be ordered around an atomic operation. Without any constraint, the apparent order of changes can even differ among multiple reader threads.

The default behavior provides for sequentially consistent ordering. This default behavior can reduce performance, therefore the possibility to give an additional `std::memory_order` argument to define the constraints that the compiler and processor must enforce beyond atomicity.

- **`memory_order_relaxed`**

Relaxed operation: there are *no synchronization or ordering constraints* imposed on other reads or writes, only this operation's atomicity is guaranteed

- **`memory_order_consume`**

A load operation with this memory order performs a consume operation on the affected memory location: no reads or writes in the current thread dependent on the value currently loaded can be reordered before this load. Writes to data-dependent variables in other threads that release the same atomic variable are visible in the current thread. On most platforms, this affects compiler optimizations only

- **`memory_order_acquire`**

A load operation with this memory order performs the acquire operation on the affected memory location: no reads or writes in the current thread can be reordered before this load. All writes in other threads that release the same atomic variable are visible in the current thread

- **`memory_order_release`**

A store operation with this memory order performs the release operation: no reads or writes in the current thread can be reordered after this store. All writes in the current thread are visible in other threads that acquire the same atomic variable and writes that carry a dependency into the atomic variable become visible in other threads that consume the same atomic

- **`memory_order_acq_rel`**

A read-modify-write operation with this memory order is both an acquire operation and a release operation. No memory reads or writes in the current thread can be reordered before the load, nor after the store. All writes in other threads that release the same atomic variable are visible before the modification and the modification is visible in other threads that acquire the same atomic variable.

- **`memory_order_seq_cst`**

A load operation with this memory order performs an acquire operation, a store performs a release operation, and read-modify-write performs both an acquire

operation and a release operation, plus a single total order exists in which all threads observe all modifications in the same order.

Concurrency Bibliography

- [87] Crowl McKenney Bohem. *C++ Data-Dependency Ordering: Atomics and Memory Model*. [ISO/IEC JTC1 SC22 WG21 N2556](#). 2008.
- [88] *Memory model synchronization modes*. [GNU gcc Wiki](#).
- [89] *What do each memory_order mean?* [StackOverflow Thread](#).

8

CHAPTER

Python

1 PEP

2 Built-in Data Types

Python has several [types built in the interpreter](#); the principal ones are numerics, sequences, mappings, classes, instances and exceptions.

2.1 dict

2.2 list

2.3 set

2.4 frozenset

2.5 tuple

2.6 string

2.7 bytes

2.8 bytearray

3 unittest Module

3.1 Classes and functions

4 Decorator

A function decorator is a callable object that takes a function as its input parameter
[decorator](#)

[Decorators with parameters](#)

[PythonDecoratorLibrary](#)

5 Special Instance Method

Special Instance Method

6 Providing Multiple Ctors

multiple `__init__`

Python Bibliography

- [121] *Python Glossary*. docs.python.org.
- [122] *Providing Multiple Constructors in Your Python Classes*. [Real Python](#).
- [123] *Sorting Mini How To*. wiki.python.org.
- [124] *Thread-based parallelism*. docs.python.org.
- [125] *Process-based parallelism*. docs.python.org.
- [126] *Asynchronous I/O*. docs.python.org.
- [127] *Asyncio in Python - Full Tutorial*. [YouTube](#).

9

CHAPTER

Software Testing

1 Seven Testing Principles

2 SDLC vs. STLC

3 Python Unit Testing

3.1 unittest

`unittest` — Unit testing framework

It supports test automation, sharing of setup and shutdown code for tests, aggregation of tests into collections, and independence of the tests from the reporting framework.

To achieve this, unittest supports some important concepts in an object-oriented way:

- **test fixture** A test fixture represents the *preparation needed* to perform one or more tests, and any associated *cleanup actions*. This may involve, for example, creating temporary or proxy databases, directories, or starting a server process.

- **test case** A test case is the *individual unit of testing*. It checks for a specific response to a particular set of inputs. `unittest` provides a base class, `TestCase`, which may be used to create new test cases.
- **test suite** A test suite is a *collection of test cases, test suites, or both*. It is used to aggregate tests that should be executed together.
- **test runner** A test runner is a *component which orchestrates* the execution of tests and provides the outcome to the user. The runner may use a graphical interface, a textual interface, or return a special value to indicate the results of executing the tests.

A testcase is created by subclassing `unittest.TestCase` or using `FunctionTestCase` for legacy compatibility

Child class methods' names start by `test_` as a naming convention to inform the test runner.

`unittest.main` provides a command line interface to execute tests.

In the following tables 9.1, 9.2, 9.3 and 9.4 a *list of assert methods provided by `TestCase`*

Method	Checks that	New in
<code>assertEqual(a, b)</code>	<code>a == b</code>	
<code>assertNotEqual(a, b)</code>	<code>a != b</code>	
<code>assertTrue(x) bool(x)</code>	<code>is True</code>	
<code>assertFalse(x) bool(x)</code>	<code>is False</code>	
<code>assertIs(a, b)</code>	<code>a is b</code>	3.1
<code>assertIsNot(a, b)</code>	<code>a is not b</code>	3.1
<code>assertIsNone(x)</code>	<code>x is None</code>	3.1
<code>assertIsNotNone(x)</code>	<code>x is not None</code>	3.1
<code>assertIn(a, b)</code>	<code>a in b</code>	3.1
<code>assertNotIn(a, b)</code>	<code>a not in b</code>	3.1
<code>assertIsInstance(a, b)</code>	<code>isinstance(a, b)</code>	3.2
<code>assertNotIsInstance(a, b)</code>	<code>not isinstance(a, b)</code>	3.2

Table 9.1: *TestCase assert methods.*

Method	Checks that	New in
<code>assertRaises(exc, fun, *args, **kwds)</code>	fun raises exc	
<code>assertRaisesRegex(exc, r, fun, *args, **kwds)</code>	fun raises exc and the message matches regex r	3.1
<code>assertWarns(warn, fun, *args, **kwds)</code>	fun raises warn	3.2
<code>assertWarnsRegex(warn, r, fun, *args, **kwds)</code>	fun raises warn and the message matches regex r	3.2
<code>assertLogs(logger, level)</code>	The with block logs on logger with minimum level	3.4
<code>assertNoLogs(logger, level)</code>	The with block does not log on logger with minimum level	3.10

Table 9.2: *TestCase methods to check the production of exceptions.*

Method	Checks that	New in
<code>assertAlmostEqual(a, b)</code>	<code>round(a-b, 7) == 0</code>	
<code>assertNotAlmostEqual(a, b)</code>	<code>round(a-b, 7) != 0</code>	
<code>assertGreater(a, b)</code>	<code>a > b</code>	3.1
<code>assertGreaterEqual(a, b)</code>	<code>a >= b</code>	3.1
<code>assertLess(a, b)</code>	<code>a < b</code>	3.1
<code>assertLessEqual(a, b)</code>	<code>a <= b</code>	3.1
<code>assertRegex(s, r)</code>	<code>r.search(s)</code>	3.1
<code>assertNotRegex(s, r)</code>	<code>not r.search(s)</code>	3.2
<code>assertCountEqual(a, b)</code>	a and b have the same elements in the same number, regardless of their order.	3.2

Table 9.3: *TestCase methods to perform more specific checks*

Method	Used to compare	New in
<code>assertMultiLineEqual(a, b)</code>	strings	3.1
<code>assertSequenceEqual(a, b)</code>	sequences	3.1
<code>assertListEqual(a, b)</code>	lists	3.1
<code>assertTupleEqual(a, b)</code>	tuples	3.1
<code>assertSetEqual(a, b)</code>	sets or frozensets	3.1
<code>assertDictEqual(a, b)</code>	dicts	3.1

Table 9.4: *TestCase* type-specific methods automatically used by *assertEqual*

Organizing test code

`unittest.TestCase.setUp()`

In case of a considerable number of tests to run, it is possible to factor out the set-up using .

The `TestCase.setUp` method does nothing on default, but can be overridden by child classes in order to prepare the `test fixture`.

Any exception different than `AssertionError` or `SkipTest` raised by this method is interpreted as an error rather than a test failure.

`unittest.TestCase.tearDown()`

The `TestCase.tearDown()` method is called immediately after the test results are recorded, even if the test method raised an exception. The only requirement for this method to be called is the success of `setUp()` call.

Grouping Tests

3.2 doctest

`doctest`

4 C++ Unit Testing

4.1 gMock

`gMock for Dummies`

`gMock Cookbook`

5 Integration Test

Integration testing *checks the communication and data flow between two modules of the same system after integrating them.*

A software consists of *several small modules* that carry certain features and come together to build the system. Sometimes even when these modules pass the quality test

independently, they *might encounter data flow issues upon integration*.

Integration testing verifies the *compatibility between these individual parts* and ensures they work in unison *without any issues after integration*. It can be a black-box testing or white-box testing technique.

An example of integration testing would be checking the logical connection between the 'Proceed to Payment' page and the payment gateway for an e-commerce application.

6 Regression Test

Regression testing *re-checks software or an application after new changes are introduced*.

This testing process runs *functional and non-functional checks on the existing features of the software to ensure that no issues arise due to new code changes*.

The goal of regression testing is *to identify any defects in the present system that may occur due to updates/changes*. Regression testing can either be white box testing or black box testing.

You can *re-use the test cases* in regression testing to verify the existing functionalities. This includes running the same test cases and scripts to get the same output that validates the present features.

An example of regression testing is checking the Camera, Phone, and other applications in your mobile after a system update. Smartphones often receive regular system updates that can hamper the ongoing functions of the phone. Regression testing would assess if any OS or parts upgrade impacted your phone.

7 Behavior-Driven Development

8 terms from the youtube video

- decision coverage
- predicate coverage
- all path coverage
- boundary coverage
- black box testing
- white box testing

9 Continous Integration Systems

- BuildBot
- Jenkins

- [GitHub Actions](#)
- [AppVeyor](#)

Testing Bibliography

- [70] *Open Lecture by James Bach on Software Testing.* [Eesti Infotehnoloogia Kolledž YouTube Channel](#).
- [71] *Regression Testing.* [Wikipedia](#).
- [72] *Progression Testing.* [Wikipedia](#).
- [73] *Regression Testing vs Progression Testing.* [Test Sigma](#).
- [74] Kent Beck. *Simple Smalltalk Testing: With Patterns.* [webarchive](#).

Bibliography

- [1] Robert Sedgewick, Kevin Wayne - *Algorithms - 4th Edition*
- [2] Steven S. Skiena - *The Algorithm Design Manual*
- [3] Robert C. Martin (2014) - *Principles of OOD*
Retrieved from: [The OLD Object Mentor blog site](#)
- [4] Robert C. Martin (2009) - *Getting a SOLID start*
Retrieved from: [Uncle Bob Consulting LLC](#).
- [5] Sandu Metz (2009) - *SOLID Object-Oriented*
Retrieved from: [Ghost Archive - Gotham Ruby Conference](#)
- [6] Robert C. Martin (2000) - *Design Principles and Design Patterns*
Retrieved from: [Internet Archive - Objectmentor](#)
- [7] Robert C. Martin (2015) - *Single Responsibility Principle*
Retrieved from: [Internet Archive - Objectmentor](#)
- [8] Robert C. Martin (2003) - *Agile Software Development, Principles, Patterns, and Practices*
Retrieved from: [Internet Archive - Objectmentor](#)
- [9] *Open/Closed Principle*
Retrieved from: [Internet Archive - Objectmentor](#)
- [10] *Liskov Substitution Principle*
Retrieved from: [Internet Archive - Objectmentor](#)
- [11] *Interface Segregation Principle*
Retrieved from: [Internet Archive - Objectmentor](#)
- [12] *Dependency Inversion Principle*
Retrieved from: [Internet Archive - Objectmentor](#)
- [13] *Clean Architecture: A Craftman's Guide to Software Structure and Design*
Retrieved from: [GunterMueller user GitHub](#)
- [14] *What is SOLID? Principles for Better Software Design*
Retrieved from: [freecode camp website](#)

- [15] *Pimpl Idiom*
Retrieved from: [Wiki Wiki Web](#)
- [16] Herb Sutter - *The Fast Pimpl Idiom*
Retrieved from: [Guru of the Week - 28](#)
- [17] Herb Sutter - *Compilation Firewalls*
Retrieved from: [Guru of the Week - 24](#)
- [18] Herb Sutter - *Compilation Firewalls (C++11-updated version of GotW #24)*
Retrieved from: [Guru of the Week - 100](#)
- [19] Herb Sutter - *The Joy of Pimpls (or more about the Compiler-Firewall Idiom).*
Retrieved from: [More About Compiler Firewalls.](#)
- [20] *What is the copy and swap idiom?*
Retrieved from: [Stackoverflow.](#)
- [21] Herb Sutter - *Exception-Safe Class Design, Part I: Copy Assignment*
Retrieved from: [Guru of the Week - 59](#)
- [22] Herb Sutter - *Exception-Safe Class Design, Part II: Inheritance*
Retrieved from: [Guru of the Week - 60](#)
- [23] *What is the "Named Constructor Idiom"?*
Retrieved from: [ISOC++ Wiki](#)
- [24] *Construct On First Use Idiom*
Retrieved from: [ISOC++ Wiki](#)
- [25] *Nifty Counter Idiom*
Retrieved from: [ISOC++ Wiki](#)
- [26] *Nifty Counter Idiom*
Retrieved from: [Wikibook](#)
- [27] *Named Parameter Idiom*
Retrieved from: [ISOC++ Wiki](#)
- [28] *Virtual Friend Function Idiom*
Retrieved from: [ISOC++ Wiki](#)
- [29] *C++ Core Guidelines*
Retrieved from: [Standard Cpp Foundation Github.](#)
- [30] *Object-Oriented Programming*
Retrieved from: [Wikipedia - Object Oriented Programming](#)
- [31] *Object-Oriented Programming Using C++*
Retrieved from: [Weber State University - School of Computing - Computer Science](#)

- [32] *Allen Holub's UML Quick Reference*
Retreived from: [Allen Holub website](#)
- [33] *Class Diagram*
Retreived from: [WikiPedia - Class Diagram](#)
- [34] *State Diagram*
Retreived from: [WikiPedia - State Diagram](#)
- [35] *State Machine*
Retreived from: [WikiPedia - State Machine](#)
- [36] *Programming paradigm*
Retreived from: [WikiPedia - State Machine](#)
- [37] *Composition over Inheritance*
Retreived from: [WikiPedia - Composition over Inheritance](#)
- [38] *Delegation (object-oriented programming)*
Retreived from: [WikiPedia - Delegation](#)
- [39] *Role-oriented programming*
Retreived from: [WikiPedia - Programming paradigm](#)
- [40] James Rumbaugh (1991) - *Object-Oriented Modeling and Design*
Retreived from: [Internet Archive](#)
- [41] *Memory Management: Stack and Heap*
Retrieved from: [CppCon YouTube Video](#)
- [42] *Leak-freedom in C++ ... By Default - CppCon 2016*
Retrieved from: [CppCon YouTube Video](#)
- [43] Jonathan Boccara - *The Most Vexing Parse: How to Spot It and Fix It Quickly*
Retrieved from: [Fluent C++](#)
- [44] *Most Vexing Parse*
Retrieved from:

C++ chapter bibliography
- [45] R. Martinho Fernandes (2012) - *Rule of Zero*
Retrieved from: [ISOC++ Blog](#)
- [46] Scott Meyers (2014) - *A Concern about the Rule of Zero*
Retrieved from: [The View from Aristeia - Scoot Meyers' Activities and Interests](#)
- [47] *A polymorphic class should suppress public copy/move*
Retreived from: [Standard Cpp Foundation Github](#)

- [48] *If you define or =delete any copy, move, or destructor function, define or =delete them all*
Retrieved from: [Standard Cpp Foundation Github](#)
- [49] *References*
Retrieved from: [ISOC++ Wiki](#)
- [50] *Const Correctness*
Retrieved from: [ISOC++ Wiki](#)
- [51] (2016) - *Core C++ - lvalues and rvalues*
Retrieved from: [Just Software Solutions](#)
- [52] (2008) - *Rvalue References and Perfect Forwarding*
Retrieved from: [Just Software Solutions](#)
- [53] Peter Dimov, Howard E. Hinnant, Dave Abrahams - *The Forwarding Problem: Arguments*
Retrieved from: [open-std.org - N1385=02-0043](#)
- [54] Meyers, Sutter, Alexandrescu (2011) - *Session Announcement: Adventures in Perfect Forwarding*
Retrieved from: [C++ and Beyond](#)
- [55] *What are the main purposes of std::forward and which problems does it solve?*
Retrieved from: [StackOverflow Thread](#)
- [56] *What is the move semantic?*
Retrieved from: [StackOverflow Thread](#)
- [57] Howard Hinnant (2014) - *Everything you wanted to know about Move Semantics*
Retrieved from: [Slideshare](#)
- [58] *If your function must not throw declare it noexcept*
Retrieved from: [Cpp Core Guidelines](#)
- [59] *What is the meaning of auto keyword*
Retrieved from: [StackOverflow Thread](#)
- [60] Herb Sutter - *auto variables, Part 1*
Retrieved from: [Guru of the week - 92](#)
- [61] Herb Sutter - *auto variables, Part 2*
Retrieved from: [Guru of the week - 93](#)
- [62] Herb Sutter - *AAA style (Almost Always auto)*
Retrieved from: [Guru of the week - 94](#)
- [63] - *What's In a Class? - The Interface Principle*
Retrieved from: [Guru of the week - C++ Report, 10\(3\), March 1998](#)

- [64] (2016) *Declaring non-type template arguments with `auto`*
Retrieved from: [Programming Language C++, Evolution Working Group - P0127R1](#)
- [65] (2016) *Declaring non-type template parameters with `auto`*
Retrieved from: [Programming Language C++, Evolution Working Group - P0127R2](#)
- [66] *Pros and Cons of Alternative Function Syntax in C++*
Retrieved from: [Petr Zemek's Blog of a Software Engineer](#)
- [67] *ADL - Argument-dependent lookup*
Retrieved from: [Cplusplusreference](#)
- [68] *Lambda Expressions*
Retrieved from: [Cplusplusreference](#)
- [69] Larvi, Freeman, Cowl (2008) - *Lambda Expressions and Closures: Wording for Monomorphic Lambdas (Revision 4)*
Retrieved from: [open-std.org](#)
- [70] Vandevorde (2009) - *New wording for C++0x Lambdas*
Retrieved from: [open-std.org](#)
- [71] *Closure (Computer Programming)*
Retrieved from: [Wikipedia](#)
- [72] *Abbreviated function template*
Retrieved from: [Cplusplusreference](#)
- [73] *decltype specifier*
Retrieved from: [Cplusplusreference](#)
- [74] Stroustrup, Dos Reis (2007) - *Initializer list (Rev. 3)*
Retrieved from: [open-std.org](#)
- [75] Merrill, Vandevorde (2008) - *Initializer Lists - Alternative Mechanism and Rationale (v.2)*
Retrieved from: [Initializer Lists — Alternative Mechanism and Rationale \(v. 2](#)
- [76] Thorsten Ottosen (2007) - *Wording for range-based for-loop (revision 2)*
Retrieved from: [open-std.org](#)
- [77] *Range-based for statements and ADL*
Retrieved from: [open-std.org](#)
- [78] Svoboda (2010) - *Delete operators default to noexcept*
Retrieved from: [open-std.org](#)
- [79] Mauer (2010) - *Deducing "noexcept" for destructors*
Retrieved from: [open-std.org](#)

- [80] Abrahams, Sharoni, Gregor (2010) - *Allowing Move Constructors to Throw (Rev. 1)*
Retrieved from: open-std.org
- [81] *Bjarne Stroustrup's C++ Style and Technique FAQ*
Retrieved from: stroustrup.com
- [82] McKenney, Bohem, Cowl (2008) - *C++ Data-Dependency Ordering: Atomics and Memory Model*
Retrieved from: ISO/IEC JTC1 SC22 WG21 N2556
- [83] *Memory model synchronization modes*
Retrieved from: GNU gcc Wiki
- [84] *What do each memory_order mean?*
Retrieved from: StackOverflow Thread
- [85] *Python Glossary*
Retrieved from: docs.python.org
- [86] *Open Lecture by James Bach on Software Testing*
Retrieved from: Eesti Infotehnoloogia Kolledž YouTube Channel
- [87] *Does return-by-value mean extra copies and extra overhead?*
Retrieved from: ISOC++ Wiki
- [88] - *SFINAE - Substitution Failure Is Not An Error*
Retrieved from: Cppreference